



Khoa Công nghệ thông tin - Học viện Kỹ thuật quân sự
Trụ sở: Tầng 19, nhà S1, 236 Hoàng Quốc Việt, Cầu Giấy, Hà Nội
Tel: (+84) 069515329 - Email: fit@mta.edu.vn
Website: <https://fit.mta.edu.vn>

<VOCABOOST – HỆ THỐNG HỌC TỪ VỰNG TOEIC THÔNG MINH>

THIẾT KẾ CHI TIẾT

Mã dự án	<2026_VocaBoost_MTA>
Phiên bản	V1.0.0
Ngày	20/06/2026

<HÀ NỘI, 6-2026>

NỘI DUNG SỬA ĐỔI

*M- Mới S – Sửa X - Xóa

[illegible]

TRANG KÝ

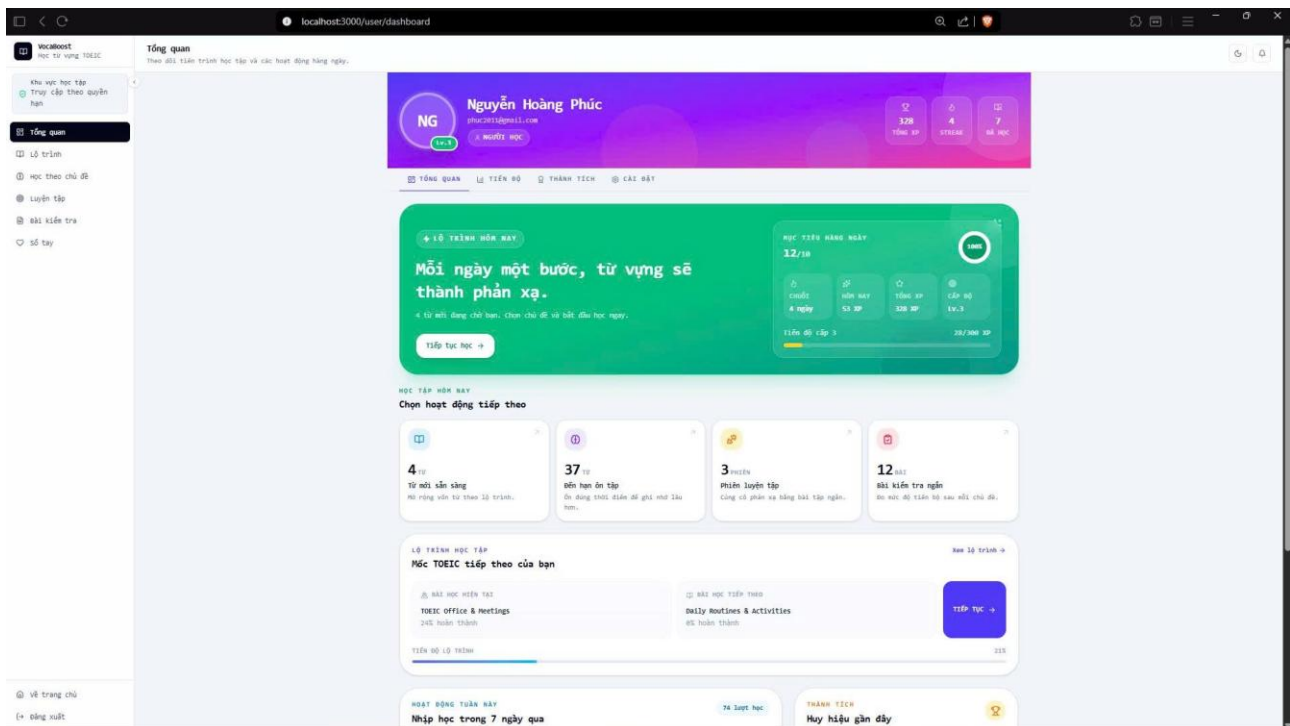
NGƯỜI LẬP: Nguyễn Hoàng Phúc _____ <Ngày> _____
<Vị trí: Backend, Architecture>

NGƯỜI KIỂM TRA: Lại Đức Tùng_____ <Ngày> _____
<Vị trí: Team Lead>

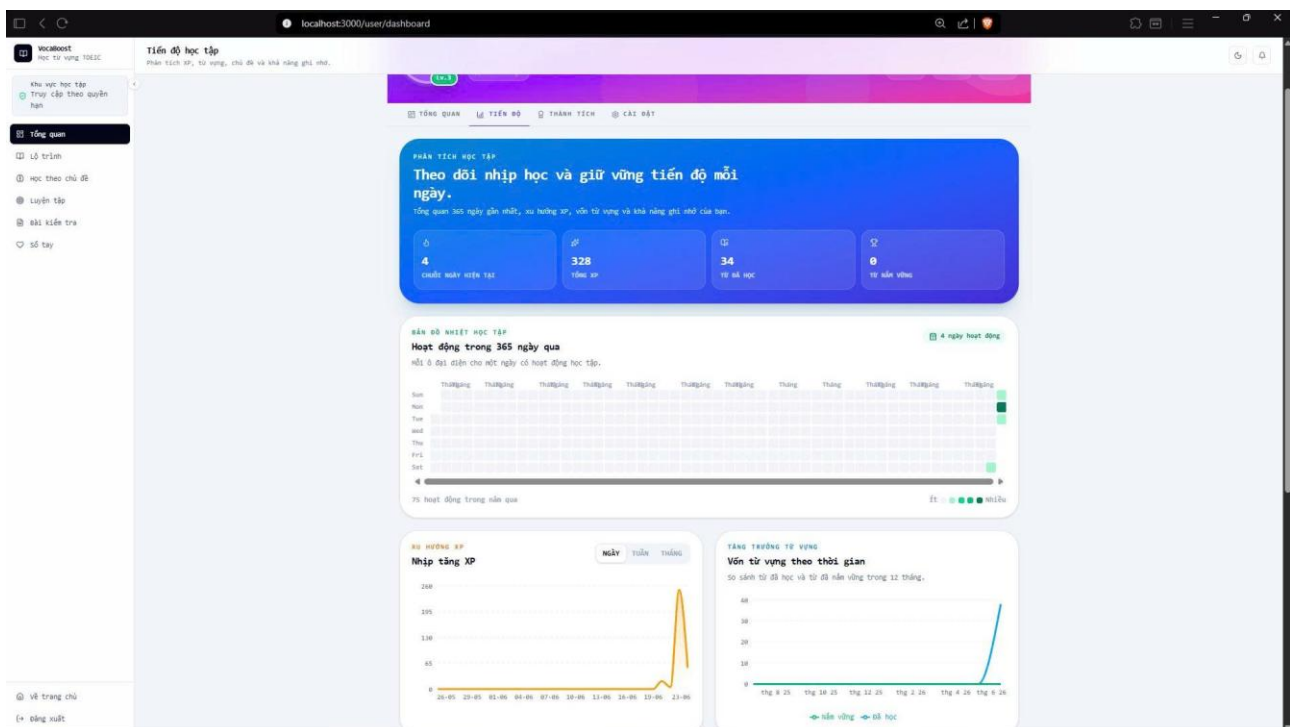
NGƯỜI PHÊ DUYỆT: Trần Tuấn Dũng _____ <Ngày> _____
 <Vị trí: Frontend>

MỤC LỤC

1. GIỚI THIỆU	15
1.1. Mục đích	15
1.2. Phạm vi.....	15
1.3. Vai trò và trách nhiệm.....	15
1.4. Tài liệu tham khảo	15
1.5. Từ và thuật ngữ	15
2. TỔNG QUAN HỆ THỐNG	17
2.1. Tổng quan.....	17
2.2. Các yêu cầu chức năng.....	17
3. KIẾN TRÚC HỆ THỐNG	18
3.1. Mô hình kiến trúc	18
4. THIẾT KẾ LỚP.....	24
4.1. Class Diagram	24
1. Module Auth - Xác thực và phân quyền.....	24
2. Module Categories - Danh mục và chủ đề học	26
3. Module User Learning - Học từ vựng, SRS, mini-test, notebook.....	27
4. Module Learning Path & Progress.....	37
5. Module Gamification - XP, Level, Achievement	39
6. Module Admin - Quản trị hệ thống và nội dung	41
7. Module Creator - Biên tập nội dung học	57
8. Module Review - Kiểm duyệt nội dung	62
9. Module AI - Hỗ trợ soạn nội dung TOEIC	63
10. Module Report & Notification - Báo cáo và thông báo	65
11. Module System/Common - Hạ tầng backend	67
5. THIẾT KẾ DỮ LIỆU	69
6. THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG.....	70
6.1 Giao diện đăng nhập, người dùng	70
Hình 6.1.1: Giao diện đăng nhập, đăng kí hệ thống	71



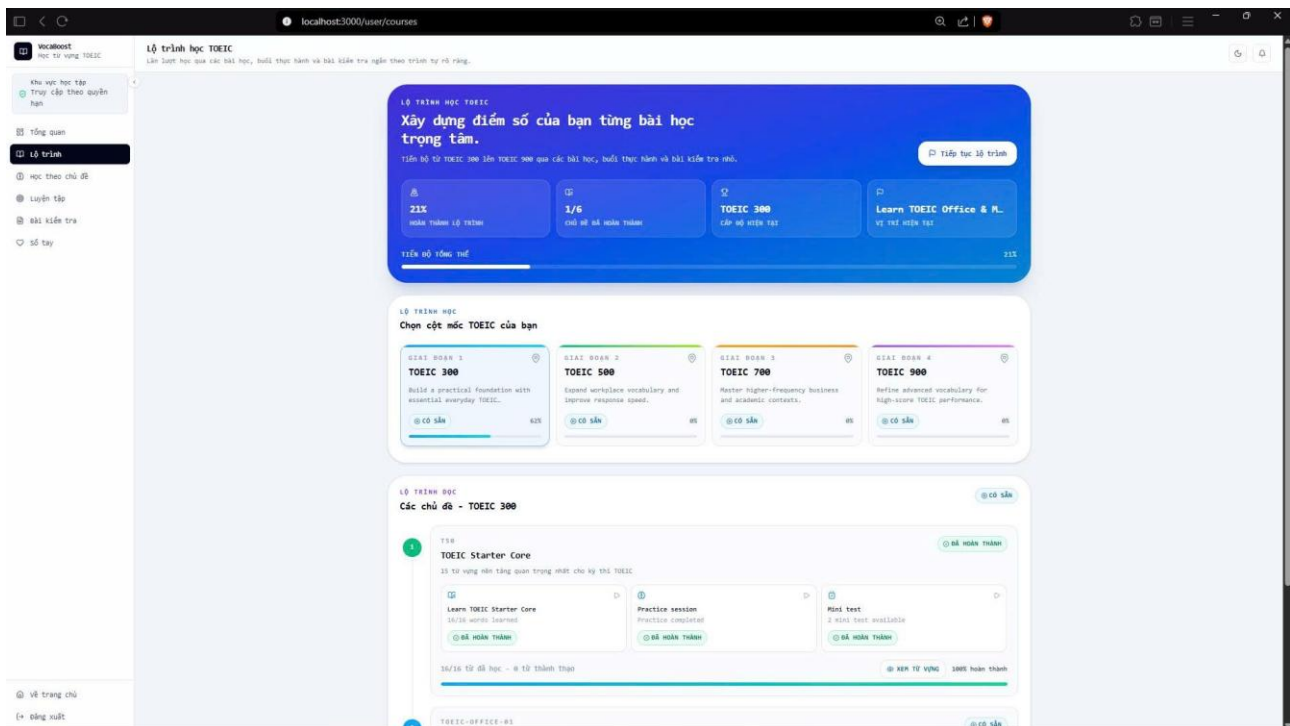
71



72

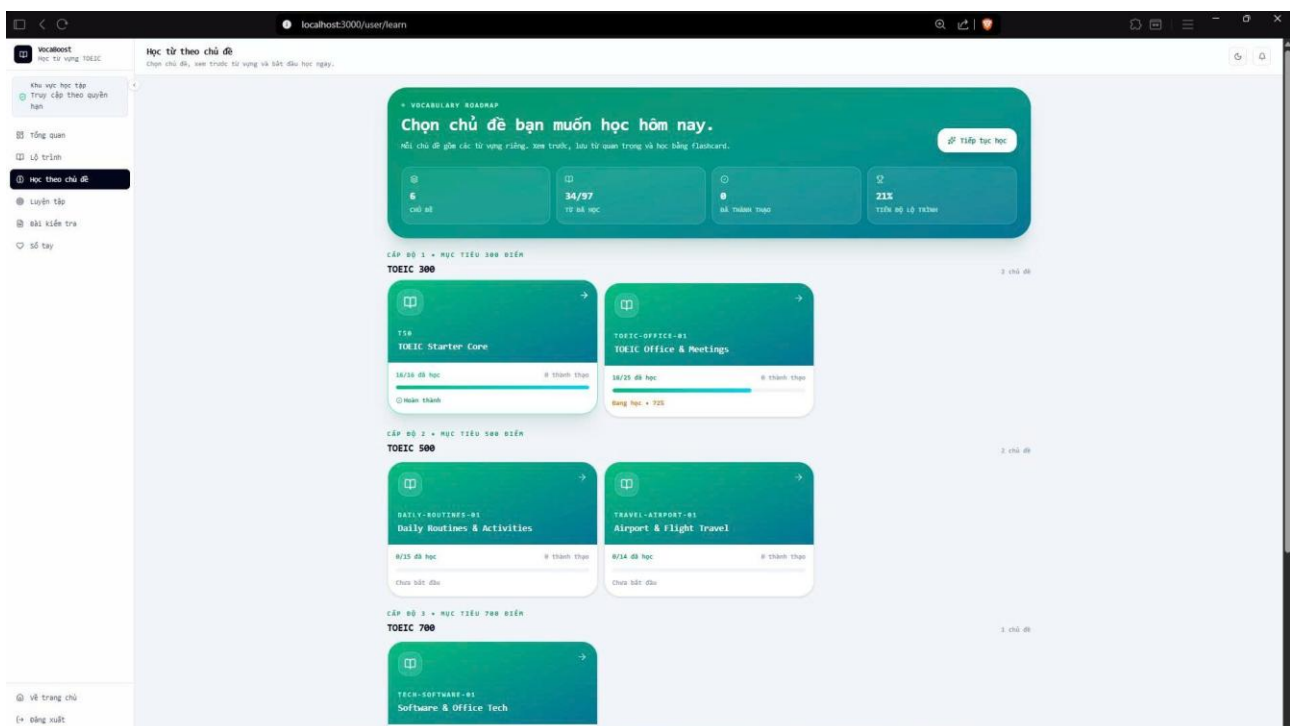
Hình 6.1.2: Giao diện bảng điều khiển học tập với mục tiêu ngày, XP, chuỗi học, hoạt động và lộ trình tiếp theo.

72



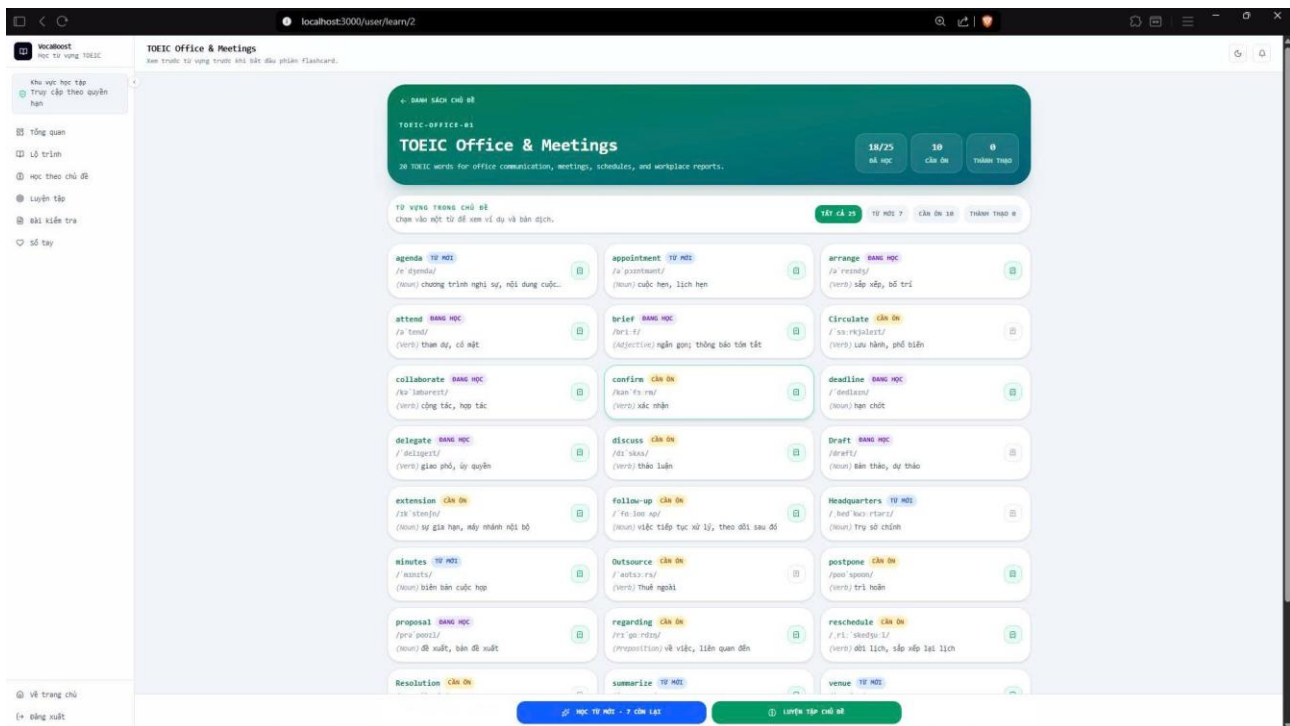
..... 72

Hình 6.1.3: Giao diện danh sách lộ trình/chủ đề giúp học viên chọn nội dung theo thứ tự học phù hợp. 72



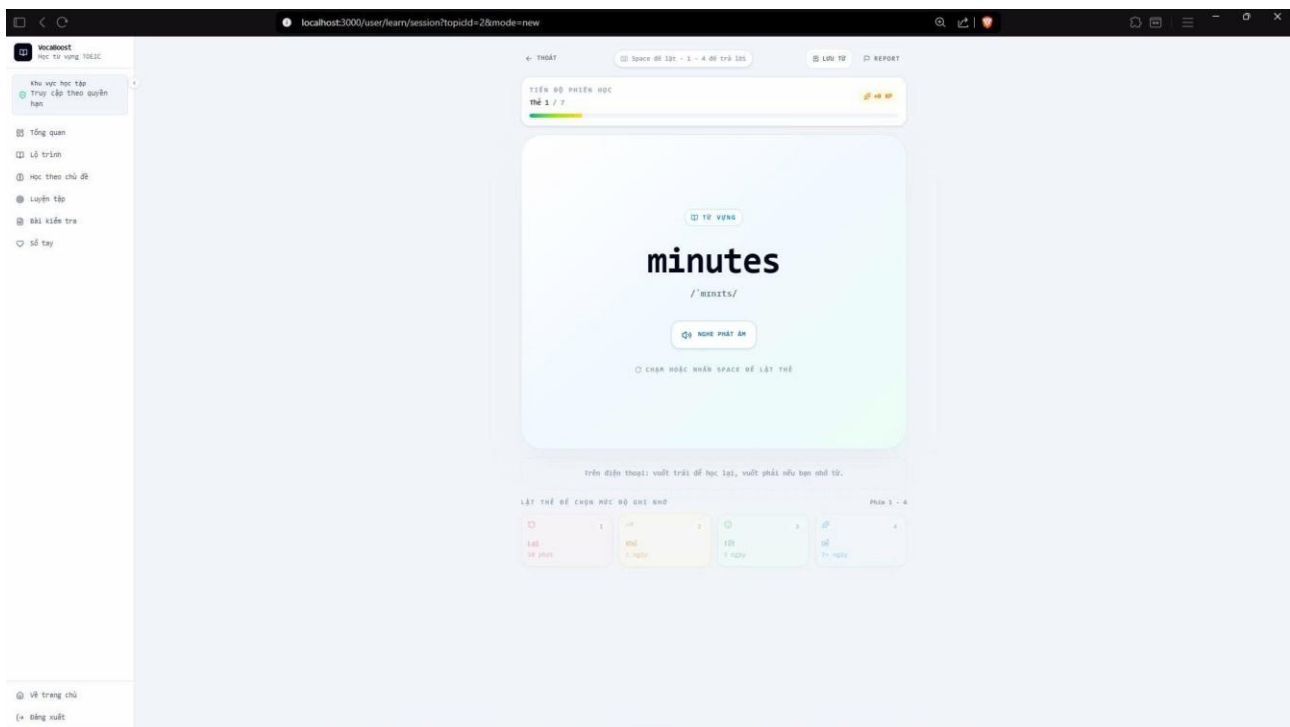
..... 73

Hình 6.1.4: Khu vực chọn chủ đề và bắt đầu phiên học từ vựng bằng flashcard..... 73

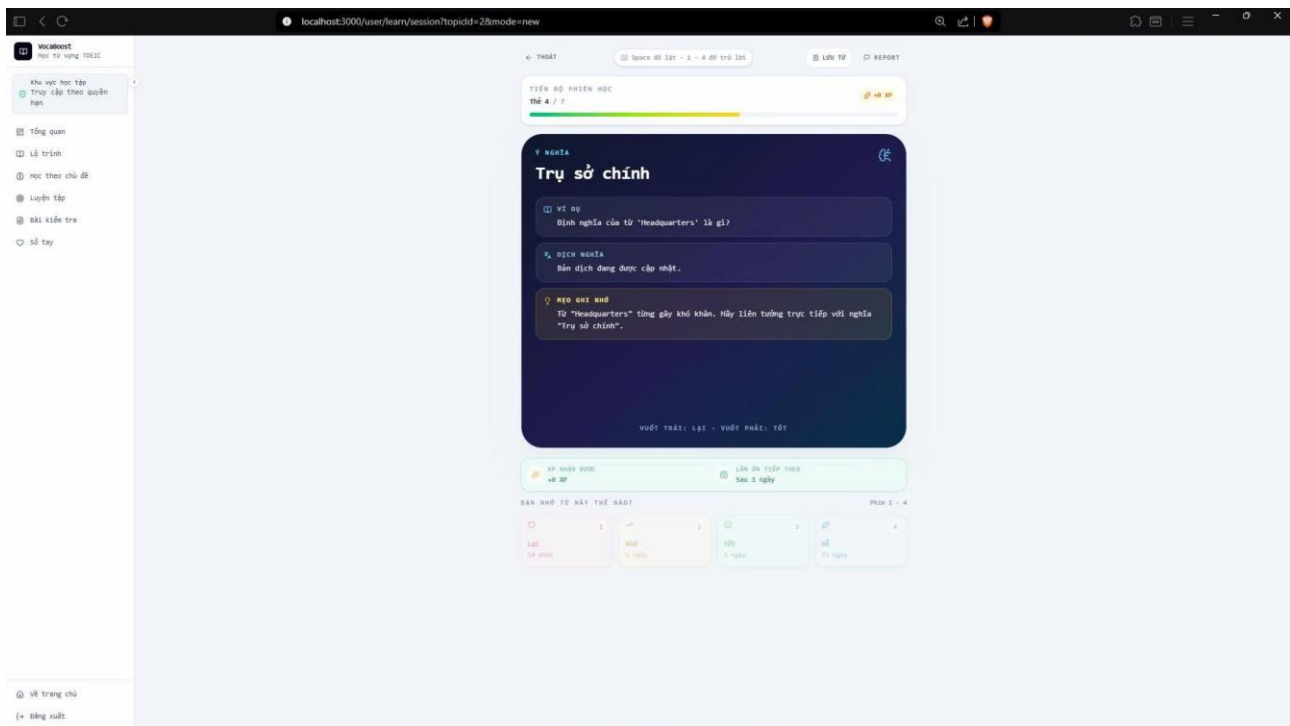


..... 73

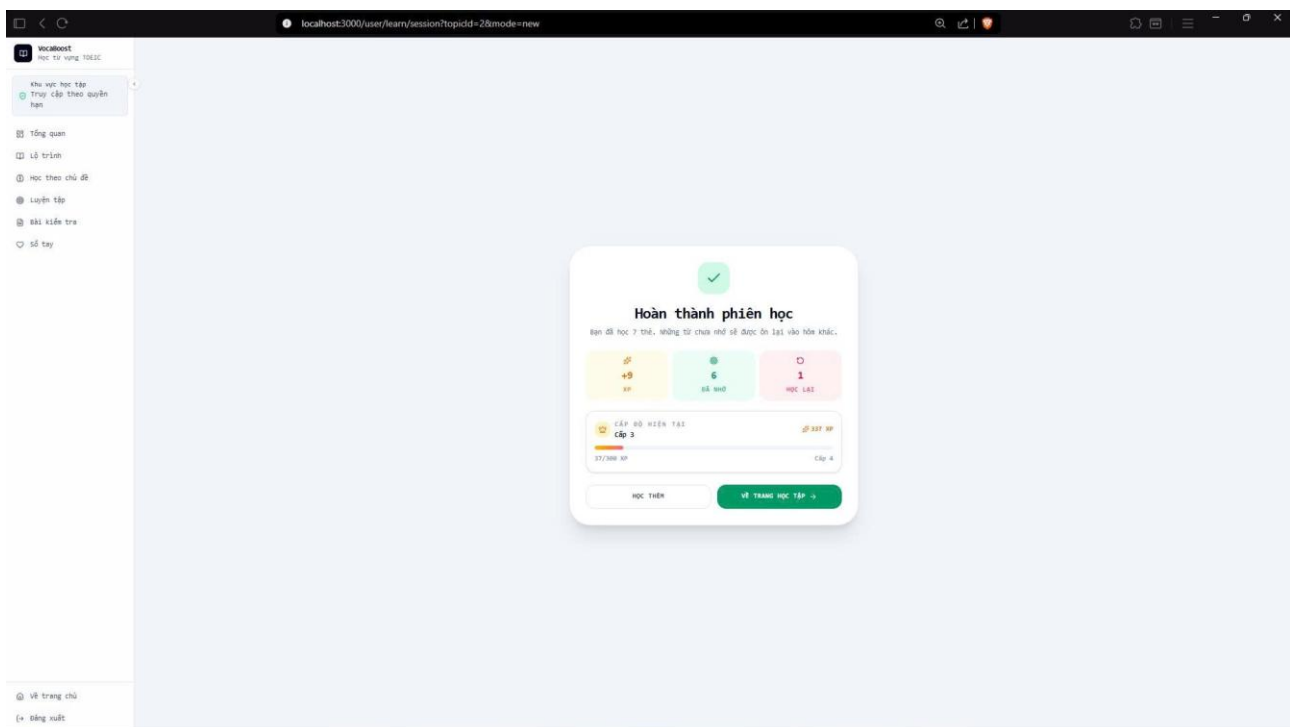
Hình 6.1.5: Giao diện xem các từ vựng trong chủ đề 73



..... 74

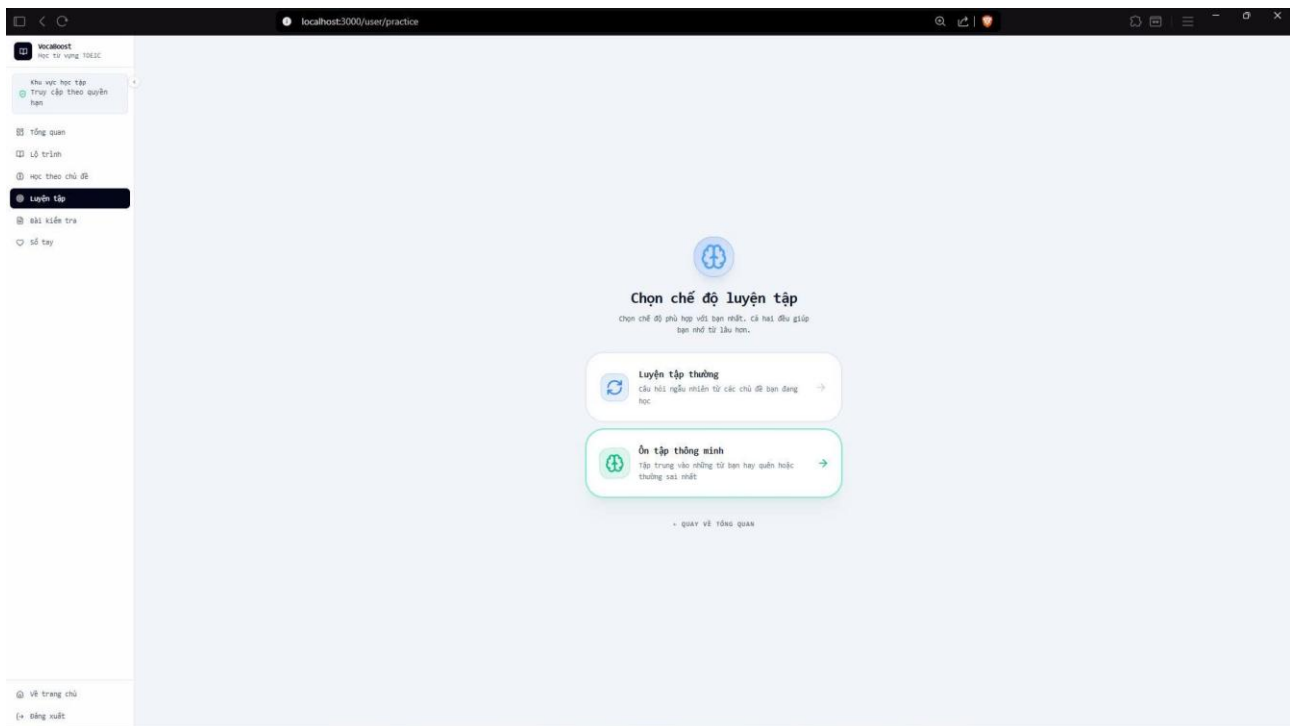


74

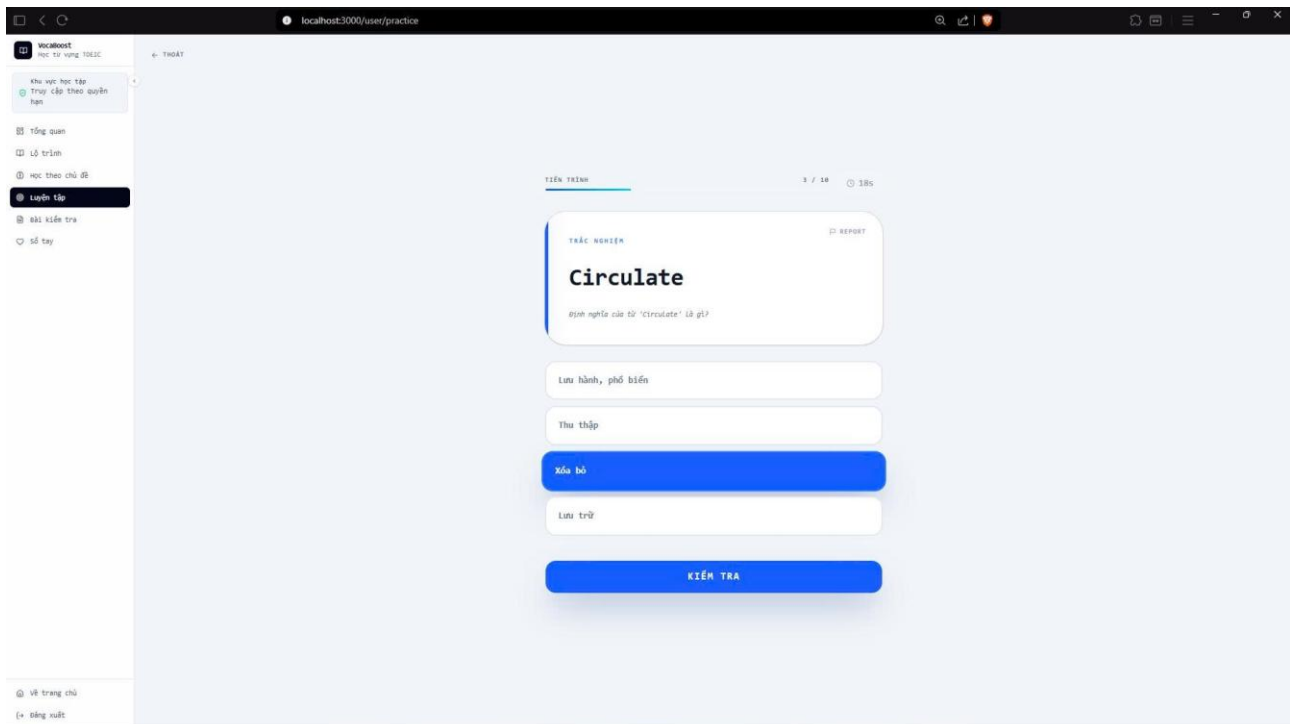


75

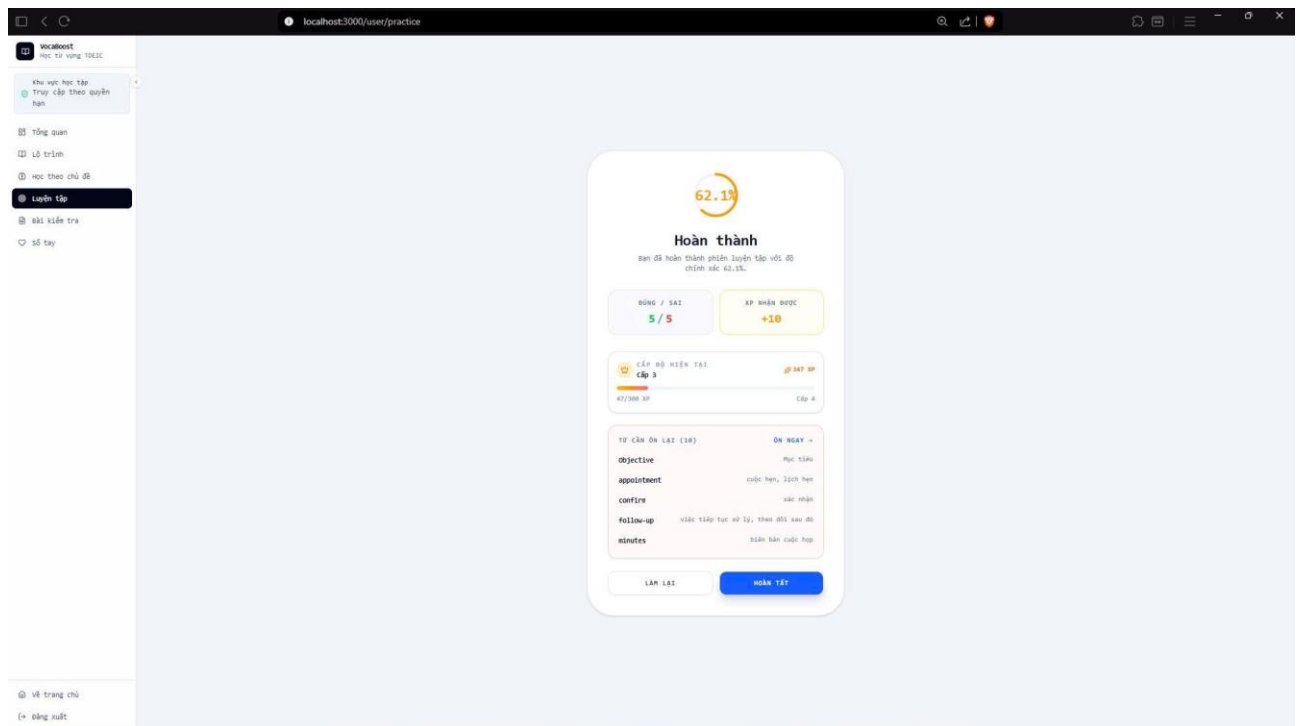
Hình 6.1.6: Giao diện học bằng flashcard..... 75



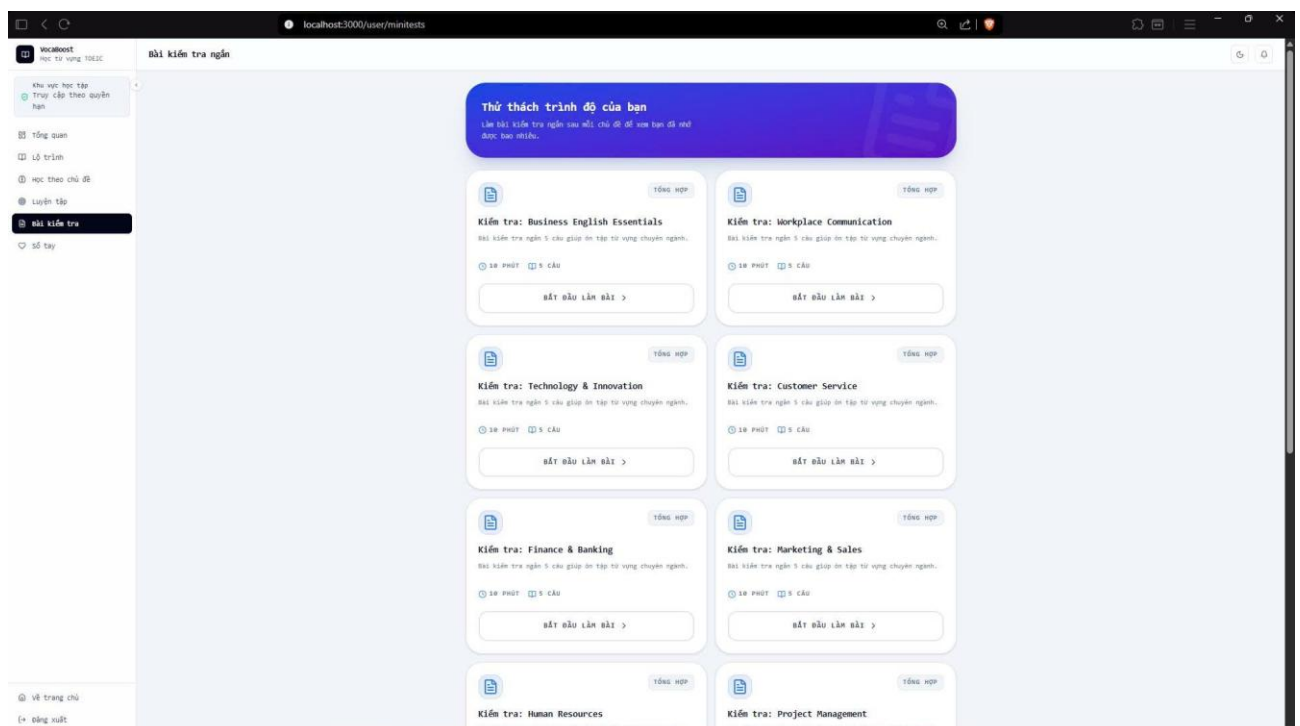
75



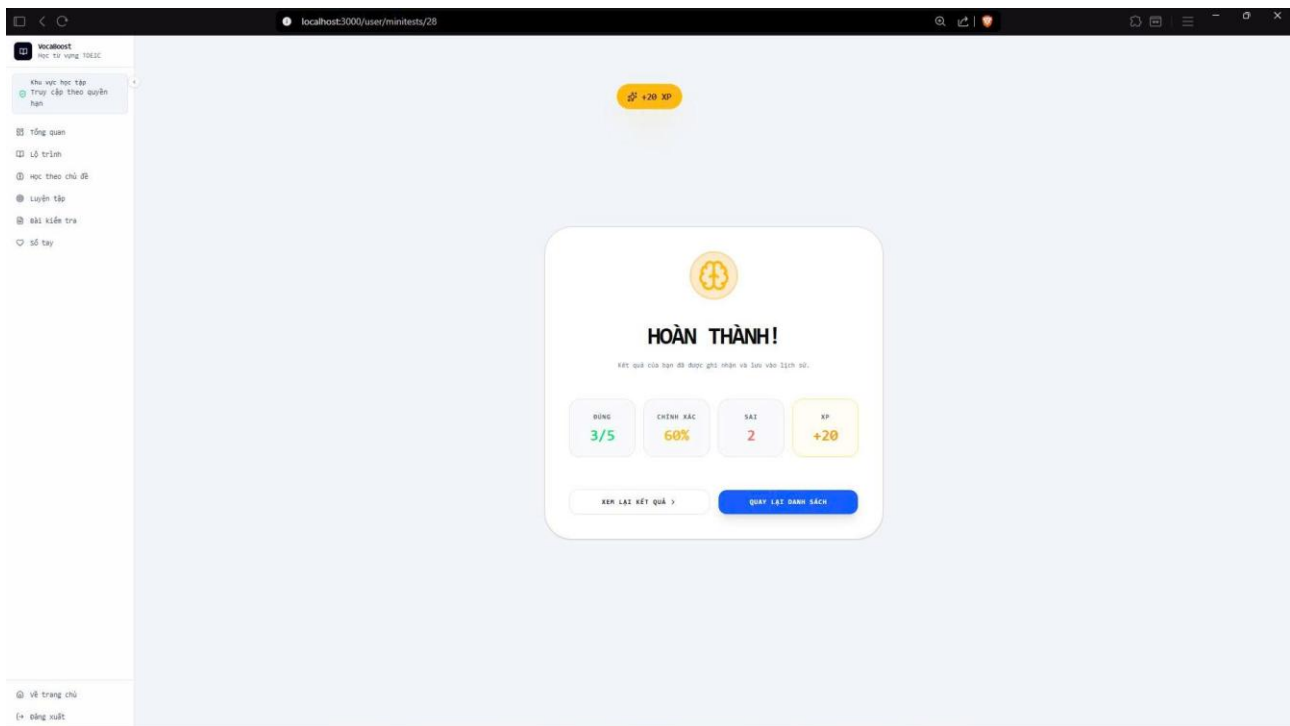
76



Hình 6.1.7: Màn hình chọn chế độ luyện tập và ôn tập thông minh theo dữ liệu SRS. 76

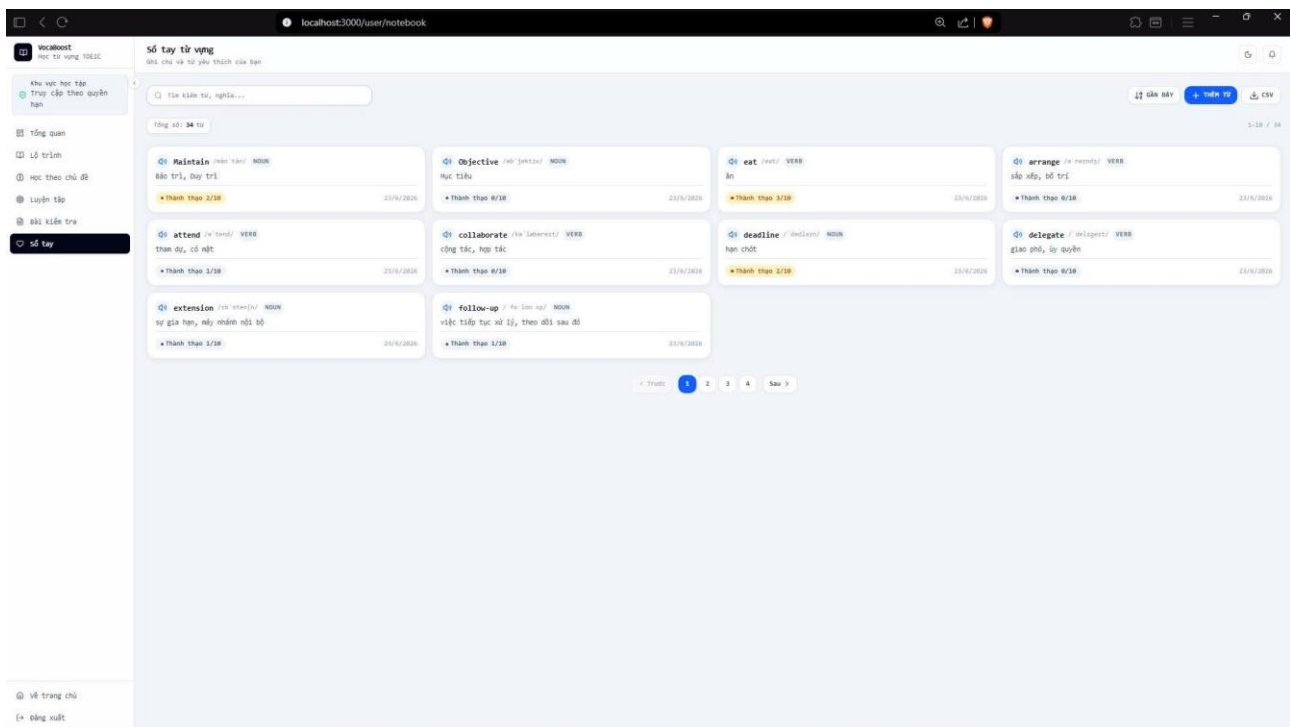


Hình 6.1.8: Danh sách mini test để đánh giá mức độ ghi nhớ và theo dõi kết quả. 77



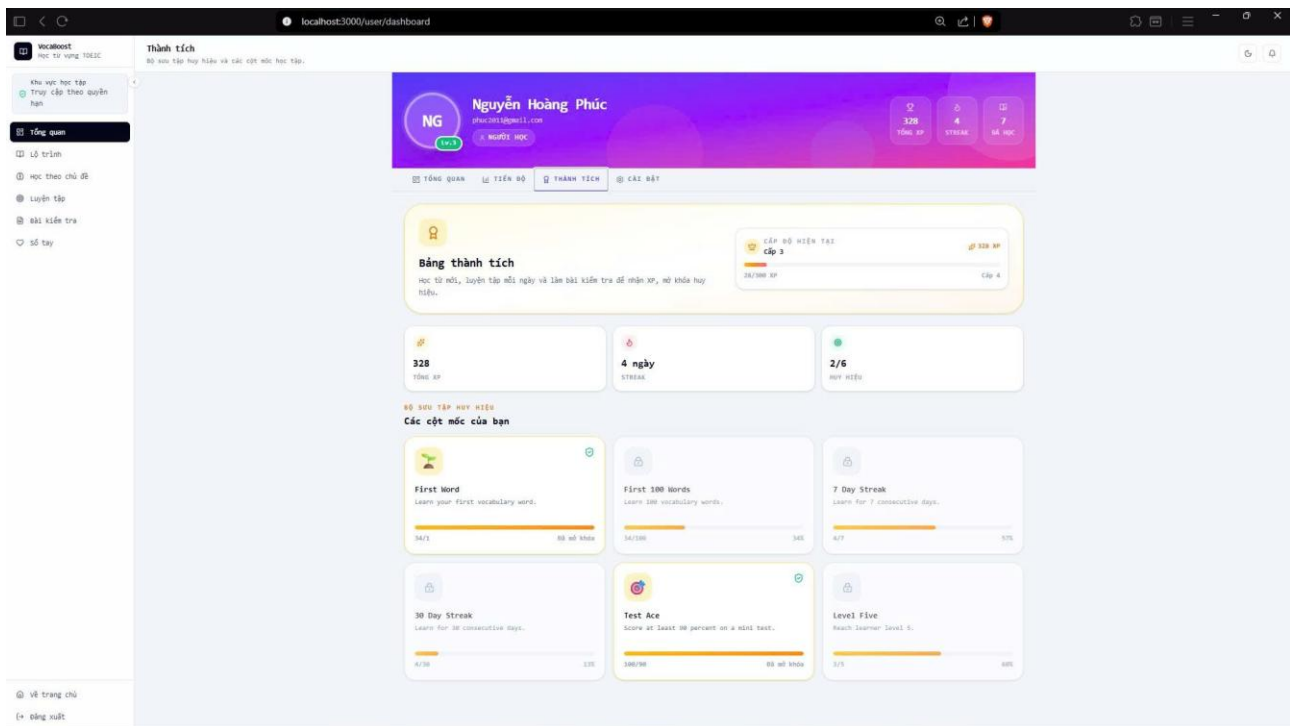
..... 78

Hình 6.1.9: Giao diện luyện tập minitest 78



..... 78

Hình 6.1.10: Nơi lưu, tìm kiếm và quản lý những từ vựng học viên muốn ghi nhớ riêng. 78

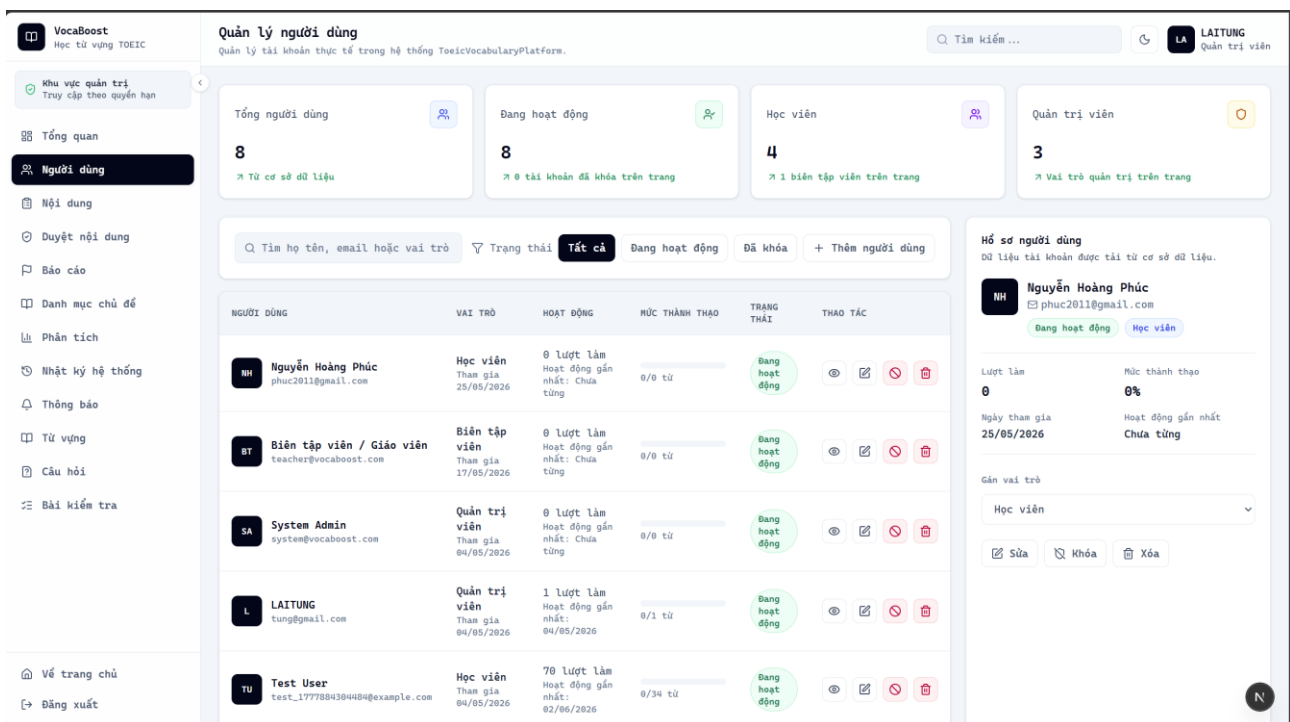


..... 79

Hình 6.1.11: Các huy hiệu, cột mốc và tiến độ gamification của học viên. 79

6.2 Giao diện dành cho quản trị (admin) 79

Hình 6.2.1: Dashboard hệ thống với KPI người dùng, nội dung, hoạt động học và trạng thái dịch vụ. 79



..... 80

Hình 6.2.2: Danh sách tài khoản, vai trò, trạng thái hoạt động và các thao tác quản trị. 80

Hình 6.2.3: Giao diện tổng hợp chủ đề và nội dung học để quản trị trạng thái xuất bản. 81

Hình 6.2.4: Giao diện hàng đợi kiểm duyệt nội dung do biên tập viên gửi lên. 81

Hình 6.2.5: Giao diện danh sách phản hồi/báo lỗi của học viên và quy trình xử lý. 82

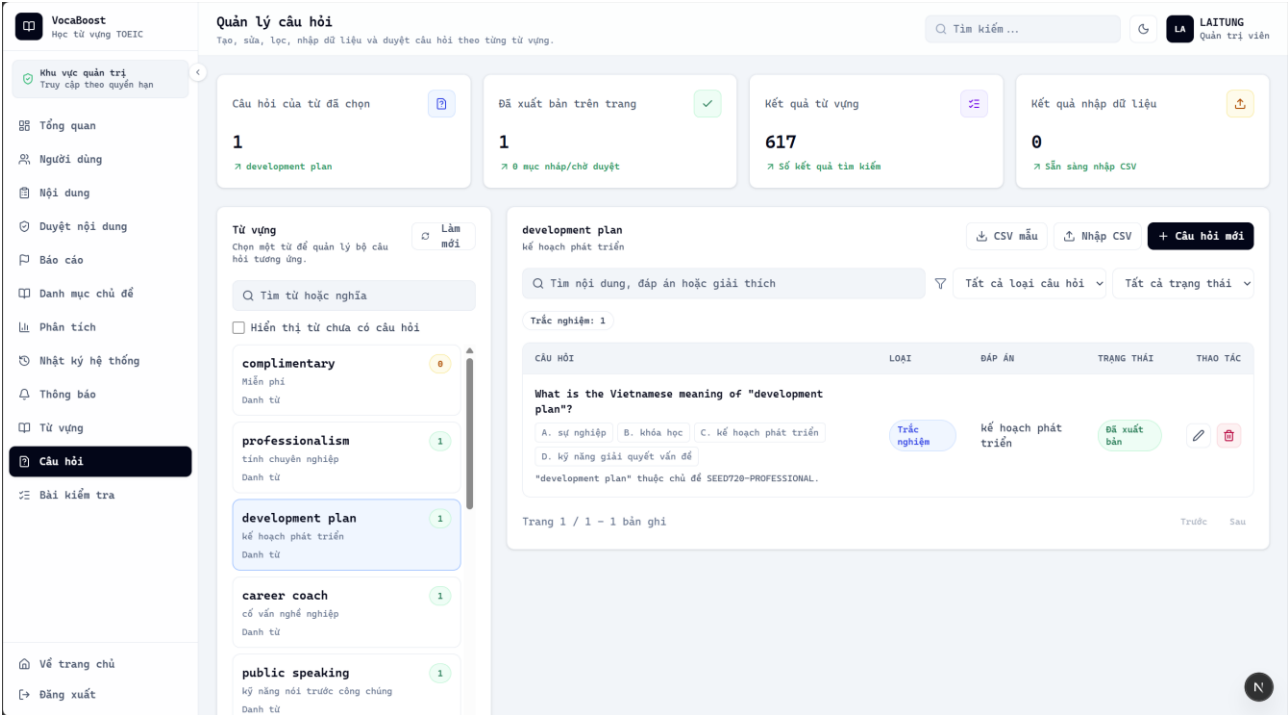
Hình 6.2.6: Giao diện Quản lý các nhóm danh mục dùng để phân loại chủ đề học. 82

Hình 6.2.7: Biểu đồ phân tích người dùng, học tập và dữ liệu nội dung toàn hệ thống. 83

Hình 6.2.8: Giao diện theo dõi các hành động quản trị và thay đổi dữ liệu quan trọng. 83

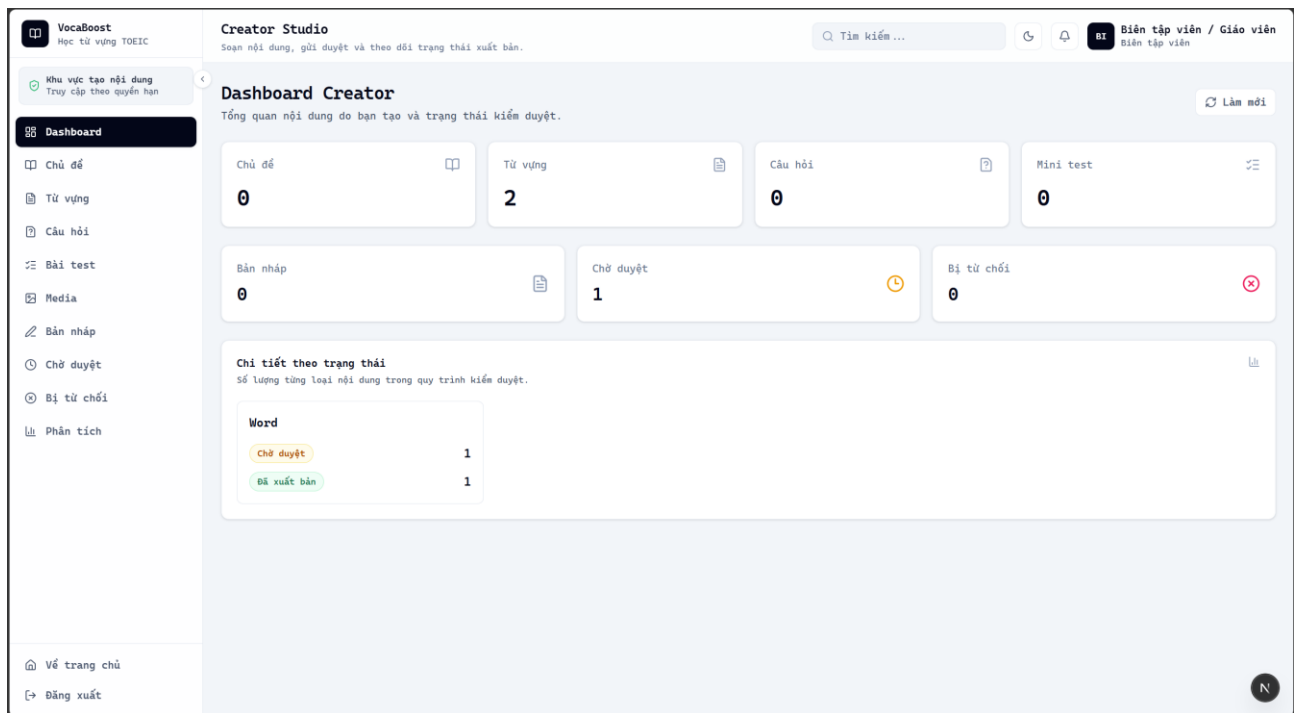
Hình 6.2.9: Giao diện gửi thông báo chung và tạo lời nhắc học hằng ngày..... 84

Hình 6.2.10: Giao diện bảng dữ liệu từ vựng với tìm kiếm, lọc, nhập hàng loạt và CRUD. 85



..... 85

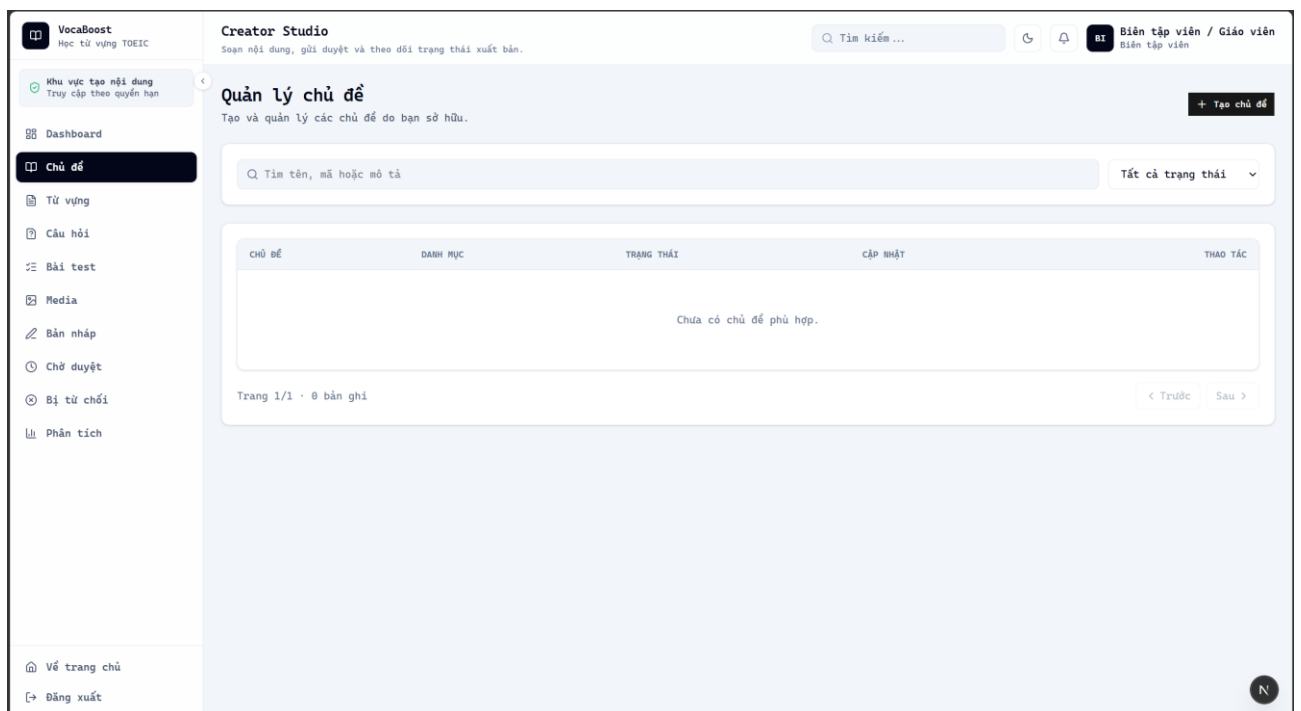
6.3 Giao diện dành cho Creator 86



86

Hình 6.3.1: Tổng quan số lượng chủ đề, từ vựng, câu hỏi, bài test và trạng thái nội dung.

86



86

Hình 6.3.9: Các chỉ số giúp đánh giá quy mô và hiệu quả nội dung đã tạo. 90

1. GIỚI THIỆU

1.1. Mục đích

Tài liệu này cung cấp thiết kế chi tiết về kiến trúc, dữ liệu cũng như các thành phần của hệ thống học từ vựng TOEIC thông minh VocaBoost.

Tài liệu này được sử dụng để:

Nhân viên kiểm thử: Lập kịch bản test và số liệu kiểm thử.

Thành viên đội phát triển: Xây dựng các phân hệ nghiệp vụ chức năng.

Nhân viên thiết kế UI/UX: Tham khảo để thiết kế giao diện.

Giảng viên hướng dẫn: Xác nhận tính đúng đắn của thiết kế.

1.2. Phạm vi

Tài liệu này phục vụ cho việc phát triển, bảo trì dự án VocaBoost.

1.3. Vai trò và trách nhiệm

Tài liệu này miêu tả chi tiết các quy trình phát triển các module và sự liên hệ giữa các module trong hệ thống VocaBoost.

Miêu tả chi tiết sự ràng buộc giữa các bảng trong CSDL, vai trò và mục đích các bảng được sử dụng để phát triển hệ thống VocaBoost.

Miêu tả quy trình thực hiện của hệ thống.

1.4. Tài liệu tham khảo

Liệt kê tất cả các tài liệu tham khảo như: các tài liệu khác của hệ thống, hoặc các bài báo kỹ thuật, ...

Loại tài liệu	Tiêu đề

1.5. Từ và thuật ngữ

Từ/thuật ngữ	Ý nghĩa	Diễn giải
<i>API</i>	<i>Application Programming Interface</i>	<i>Giao diện Lập trình Ứng dụng</i>

2. TỔNG QUAN HỆ THỐNG

VocaBoost là hệ thống học từ vựng TOEIC trực tuyến với hơn 720 từ vựng theo 4 cấp độ (300-500-700-900). Người học tiếp nhận và ôn tập từ vựng thông qua phương pháp SRS cùng cơ chế gamification (XP, Level, Streak, Badge).

Hệ thống sử dụng kiến trúc Frontend-Backend-Database với Next.js 16, React 19, Express.js 5 và SQL Server. Authentication dùng JWT kết hợp Bcrypt, phân quyền dựa trên Role-Based Access Control (RBAC) với 3 vai trò: Admin, Learner, ContentCreator.

Hệ thống VocaBoost cung cấp hơn 720 từ vựng TOEIC chia làm 4 cấp độ (300-500-700-900), học theo phương pháp Spaced Repetition System (SRS) với 4 mức đánh giá: Again (lại), Hard (khó), Good (tốt), Easy (dễ). Cơ chế gamification với XP, Level, Streak, Badge khuyến khích học tập hàng ngày.

Hệ thống hỗ trợ ba vai trò người dùng: Người học (Learner) học flashcard SRS, làm mini test, notebook từ vựng. Quản trị viên (Admin) quản lý Words, Questions, Topics, MiniTests, Students, Content Review. Biên tập viên (Content Creator) tạo nội dung và gửi duyệt.

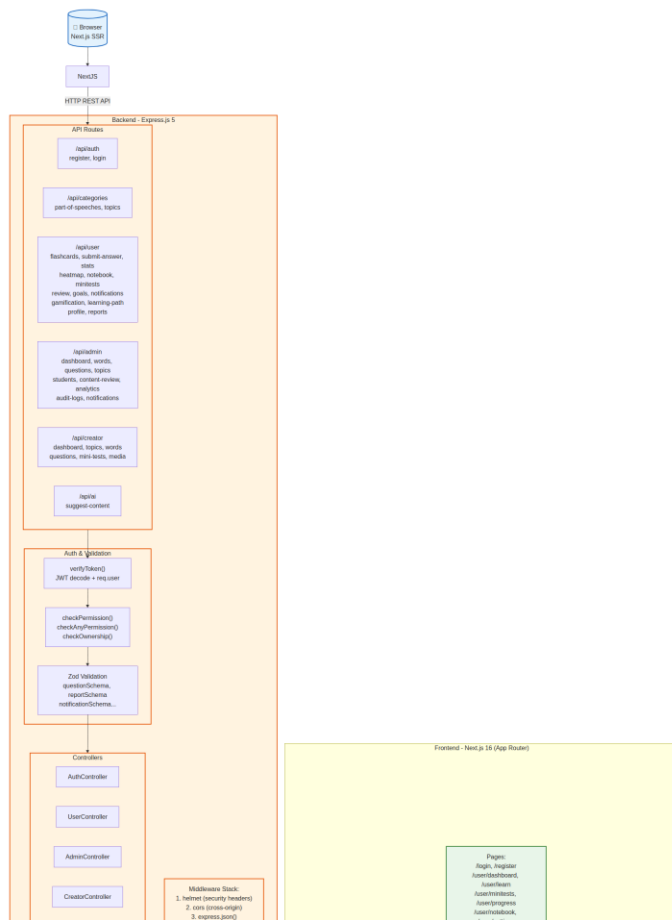
- +) Ảnh hưởng đến chất lượng dịch vụ học tập tại học tập của VocaBoost
- +) + Người học có thể luyện tập từ vựng mới và ôn tập từ đã học theo lịch trình SRS thông minh
- +) + Quản trị viên có thể quản lý từ vựng, câu hỏi, mini test, và duyệt nội dung từ biên tập viên
- + Biên tập viên có thể tạo nội dung từ vựng, câu hỏi, mini test và gửi cho admin duyệt (Draft > PendingReview > Published/Rejected)
- +) Dashboard admin cung cấp thống kê tổng quan: số lượng user, words, questions, mini tests, lượt làm bài, accuracy, và biểu đồ phân tích chi tiết.

2.1. Tổng quan

2.2. Các yêu cầu chức năng

3. KIẾN TRÚC HỆ THỐNG

3.1. Mô hình kiến trúc



Hình 5: Sơ đồ các thành phần của phần mềm

- **Client (Frontend)**

Giao diện người dùng được xây dựng bằng Next.js 16 (App Router), React 19, TypeScript và Tailwind CSS 4. Giao tiếp với backend thông qua REST API (Axios).

- Backend

Thành phần logic xử lý nghiệp vụ của hệ thống (Express.js 5), bao gồm:

- **API Routes (Express Router)**

Các route API tiếp nhận yêu cầu từ Client, xác thực JWT và điều hướng đến Controller tương ứng. Xác thực người dùng dựa trên JWT với phân quyền động.

- **Auth Controller & Auth Service**

Auth Controller & Auth Service xử lý đăng ký và đăng nhập người dùng. AuthService.register() tạo user mới, hash password bằng Bcrypt. AuthService.login() xác thực thông tin và trả về JWT token chứa userId, email, role, permissions.

- **AuthController**

Thành phần Controller xử lý xác thực người dùng. AuthController cung cấp endpoint POST /api/auth/register (đăng ký với fullName, email, password) và POST /api/auth/login (đăng nhập, trả về JWT token). Sử dụng AuthService để hash password bằng Bcrypt, tạo JWT token chứa userId, email, role, permissions với thời hạn 1 ngày. Sau đăng nhập thành công, tự động awardDailyLogin XP cho người học (Learner).

- **AdminController & AdminService**

Admin Controller & Admin Service xử lý logic quản trị: CRUD từ vựng (Words), câu hỏi (Questions), chủ đề (Topics), mini test (MiniTests); quản lý người dùng (Students); duyệt nội dung (Content Review); thống kê dashboard; import CSV; audit logs.

- **Creator Controller & Creator Service**

Xử lý logic cho biên tập viên: CRUD nội dung (từ vựng, câu hỏi, mini test), gửi duyệt nội dung (submit-review), xem thống kê (content-summary, analytics). Nội dung tạo ở trạng thái Draft, gửi admin duyệt qua PendingReview.

- **GamificationService**

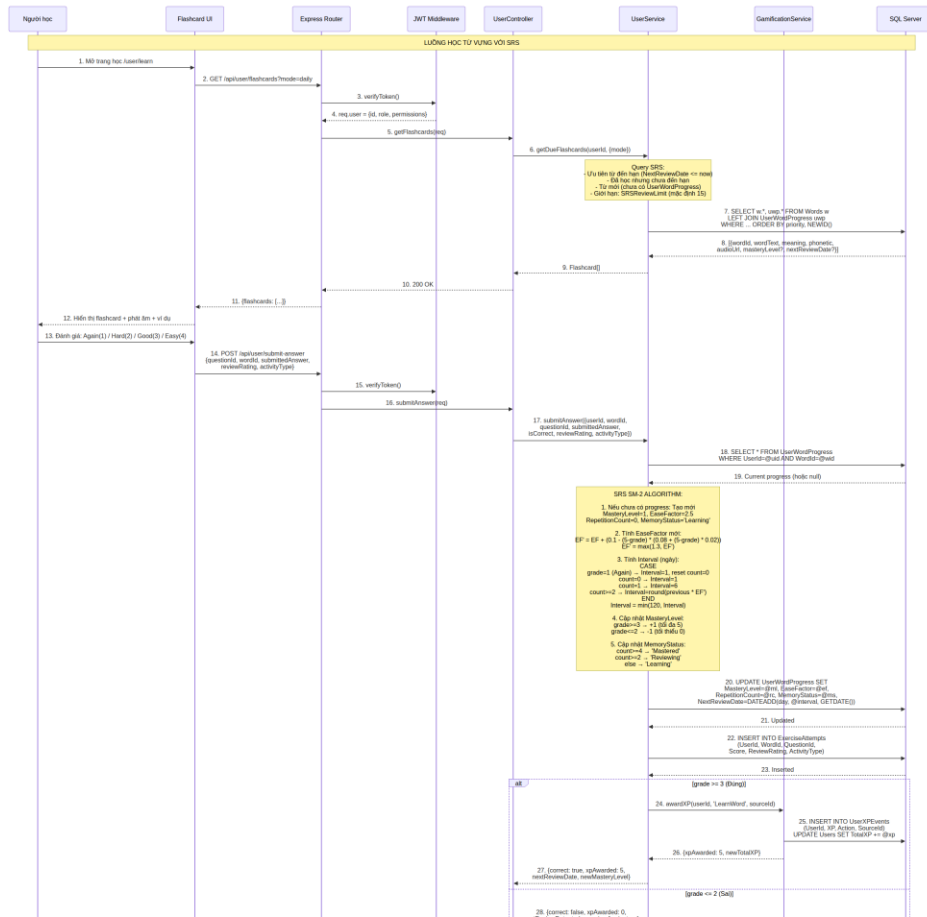
Dịch vụ gamification xử lý XP, Level, Streak, Achievement. Định nghĩa XP_REWARDS cho các hành động: LearnWord (10 XP), PracticeComplete (50 XP), MiniTestComplete (100 XP), DailyLogin (20 XP). Cung cấp getLevelState(totalXP) để tính cấp độ hiện tại ($\text{Level} = \text{floor}(\text{totalXP}/100) + 1$) và tiến độ XP trong cấp độ. Hàm awardXP(userId, action, sourceId) ghi nhận sự kiện XP. Theo dõi streak (số ngày học liên tiếp) và achievements (huy hiệu).

- **LearningPathService**

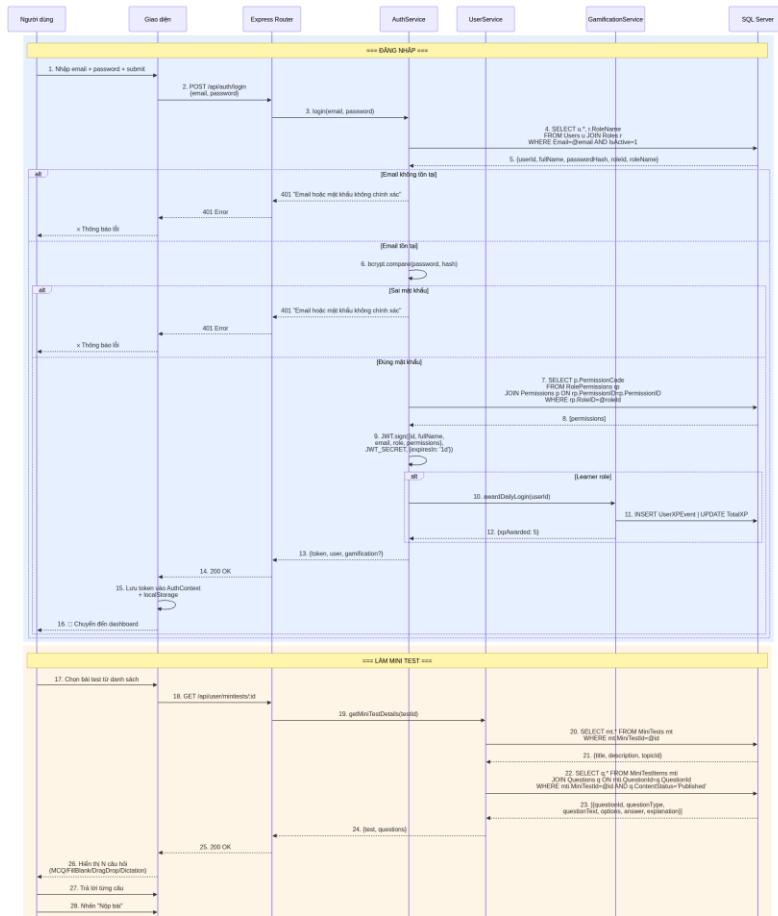
Dịch vụ quản lý lộ trình học tập (Learning Path). Tự động tạo schema với 2 bảng: dbo.LearningPathLevels (LevelId, Name, DisplayOrder) và dbo.LearningPathTopics (liên kết Level với Topics). Định nghĩa 4 cấp độ TOEIC: TOEIC 300, TOEIC 500, TOEIC 700, TOEIC 900. Mỗi cấp độ chứa nhiều chủ đề (Topics) tương ứng. getLearningPath(userId, levelId) trả về lộ trình cá nhân hóa với tiến độ học tập của người dùng tại từng chủ đề.

- **SQL Server Database (ToeicVocabularyPlatform)**

Cơ sở dữ liệu SQL Server (ToeicVocabularyPlatform) sử dụng mssql/msnodesqlv8 driver, hỗ trợ Windows Authentication và SQL Authentication. Tất cả queries được tham số hóa (parameterized queries) để chống SQL injection.



Hình 6: Sequence Diagram cho chức năng học từ vựng (SubmitAnswer)



Hình 7: Sequence Diagram cho chức năng đăng nhập và tạo Mini Test

[illegible]

Các module backend được mô tả theo cấu trúc tách lớp: Controller nhận request và điều phối, Service xử lý nghiệp vụ/truy vấn CSDL, Middleware/Config hỗ trợ xác thực, validate và vận hành. Phiên bản này mở rộng số lượng hàm trọng tâm để báo cáo đủ chi tiết hơn: mỗi hàm đều có mục đích, luồng xử lý và giả mã rút gọn theo đúng logic trong mã nguồn.

Mục đích	Xử lý đăng ký, đăng nhập, phát hành JWT và kiểm soát quyền truy cập API.
Thành phần	auth.routes.js; AuthController; AuthService; validate middleware; auth middleware; bảng Users, Roles, Permissions.
Luồng module	Client -> auth.routes -> validate(schema) -> AuthController -> AuthService -> SQL Server -> response. Với API nội bộ: Client -> verifyToken -> checkPermission/checkAnyPermission -> Controller.

Hàm Controller	Luồng xử lý	Giải mã
AuthController.register async register(req, res, next) <i>controllers/auth.controller.js</i>	Router gọi AuthController.register -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ	<pre> FUNCTION register(req, res, next) TRY { fullName, email, password } = req.body IF NOT fullName OR NOT email OR NOT password THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU </pre>

Mục đích: Nhận yêu cầu đăng ký tài khoản người học và trả kết quả tạo tài khoản.	-> trả HTTP response hoặc next(error)	<pre>{ message: "Vui lòng cung cấp đủ thông tin" } END IF user = GỌI BẮT ĐỒNG BỘ AuthService.register(fullName, email, password) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Đăng ký thành công", user: user } CATCH error IF error.message = "Email đã tồn tại" THEN ... END FUNCTION</pre>
AuthController.login async login(req, res, next) <i>controllers/auth.controller.js</i> Mục đích: Nhận thông tin đăng nhập, trả JWT và thông tin người dùng.	Router gọi AuthController.login -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre>FUNCTION login(req, res, next) TRY { email, password } = req.body IF NOT email OR NOT password THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Vui lòng cung cấp email và mật khẩu" } END IF data = GỌI BẮT ĐỒNG BỘ AuthService.login(email, password) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đăng nhập thành công", ...data } CATCH error IF error.message = "Email hoặc mật khẩu không chính xác" THEN ... END FUNCTION</pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
AuthService.register async register(fullName, email, password) <i>services/auth.service.js</i> Mục đích: Kiểm tra email trùng, mã hóa mật khẩu và tạo bản ghi người dùng.	Controller hoặc module gọi AuthService.register -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION register(fullName, email, password) pool = GỌI BẮT ĐỒNG BỘ poolPromise checkUser = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users] IF checkUser.recordset.length > 0 THEN THROW ERROR TẠO Error("Email đã tồn tại") END IF hashedPassword = GỌI BẮT ĐỒNG BỘ bcrypt.hash(password, 10) result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Roles, Users] user = result.recordset[0] RETURN { ...user, role: "Learner", permissions: ["VIEW_DASHBOARD", "LEARN VOCAB"] } END FUNCTION</pre>
AuthService.login async login(email, password) <i>services/auth.service.js</i> Mục đích: Xác thực mật khẩu, kiểm tra trạng thái tài khoản và phát hành JWT.	Controller hoặc module gọi AuthService.login -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION login(email, password) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users, Roles] user = result.recordset[0] IF NOT user THEN THROW ERROR TẠO Error("Email hoặc mật khẩu không chính xác") END IF isMatch = GỌI BẮT ĐỒNG BỘ bcrypt.compare(password, user.passwordHash) IF NOT isMatch THEN THROW ERROR TẠO Error("Email hoặc mật khẩu không chính xác") ... END FUNCTION</pre>
verifyToken verifyToken(req, res, next) <i>middlewares/auth.js</i> Mục đích: Xác thực JWT, nạp thông tin user và gắn vào req.user cho các API bảo vệ.	Express chạy middleware verifyToken -> đọc header/body/quyền user -> hợp lệ thì next() -> không hợp lệ trả lỗi 400/401/403	<pre>FUNCTION verifyToken(req, res, next) authHeader = req.headers.authorization IF NOT authHeader OR NOT authHeader.startsWith("Bearer ") THEN TRẢ VỀ HTTP 401 CÙNG DỮ LIỆU { message: "Missing or invalid token" } END IF token = authHeader.split(" ")[1] TRY req.user = jwt.verify(token, process.env.JWT_SECRET) THỰC HIỆN next() CATCH error</pre>

		... END FUNCTION
checkPermission checkPermission(permissionCode) <i>middlewares/auth.js</i> Mục đích: Kiểm tra người dùng có đúng quyền bắt buộc để gọi API quản trị.	Express chạy middleware checkPermission -> đọc header/body/quyền user -> hợp lệ thì next() -> không hợp lệ trả lỗi 400/401/403	FUNCTION checkPermission(permissionCode) RETURN HÀM CALLBACK(req, res, next) END FUNCTION
checkAnyPermission checkAnyPermission(permissionCodes) <i>middlewares/auth.js</i> Mục đích: Cho phép request đi tiếp nếu user có một trong các quyền được yêu cầu.	Express chạy middleware checkAnyPermission -> đọc header/body/quyền user -> hợp lệ thì next() -> không hợp lệ trả lỗi 400/401/403	FUNCTION checkAnyPermission(permissionCodes) RETURN HÀM CALLBACK(req, res, next) END FUNCTION
getUserPermissions getUserPermissions(req) <i>middlewares/auth.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Express chạy middleware getUserPermissions -> đọc header/body/quyền user -> hợp lệ thì next() -> không hợp lệ trả lỗi 400/401/403	FUNCTION getUserPermissions(req) RETURN req.user.permissions OR [].map(normalizePermission) END FUNCTION

2. Module Categories - Danh mục và chủ đề học

Mục đích	Cung cấp danh mục chủ đề và thống kê topic hiển thị cho người học.
Thành phần	categories.routes.js; CategoriesController; CategoriesService; bảng TopicCategories, Topics, Words.
Luồng module	Client -> categories.routes -> CategoriesController -> CategoriesService -> SQL Server -> danh sách category/topic kèm số lượng.

Controller

Hàm Controller	Luồng xử lý	Giả mã
CategoriesController.getPartOfSpeeches async getPartOfSpeeches(req, res, next) <i>controllers/categories.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi CategoriesController.getPartOfSpeeches -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getPartOfSpeeches(req, res, next) TRY pos = GỌI BẮT ĐỒNG BỘ CategoriesService.getPartOfSpeeches() TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU pos CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
CategoriesController.getTopics async getTopics(req, res, next) <i>controllers/categories.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi CategoriesController.getTopics -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getTopics(req, res, next) TRY topics = GỌI BẮT ĐỒNG BỘ CategoriesService.getTopics(req.user.id) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU topics CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
CategoriesService.getPartOfSpeeches async getPartOfSpeeches() <i>services/categories.service.js</i>	Controller hoặc module gọi CategoriesService.getPartOfSpeeches -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn,	FUNCTION getPartOfSpeeches() pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên PartOfSpeeches]

Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	RETURN result.recordset END FUNCTION
CategoriesService.getTopics async getTopics(userId = null) <i>services/categories.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi CategoriesService.getTopics -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION getTopics(userId = null) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, WordTopics, Words, UserWordProgress] RETURN result.recordset END FUNCTION

3. Module User Learning - Học từ vựng, SRS, mini-test, notebook

Mục đích	Phục vụ trải nghiệm học chính: flashcard, bài tập, SRS, mini-test, notebook, dashboard tiến độ.
Thành phần	user.routes.js; UserController; UserService; GamificationService; ReportService; bảng Words, Questions, UserWordProgress, TestAttempts, Notebook, Notifications.
Luồng module	Learner -> user.routes -> verifyToken -> UserController -> UserService -> SQL Server. Khi trả lời đúng/sai, UserService cập nhật UserWordProgress, lịch ôn SRS và gọi GamificationService cộng XP.

Controller

Hàm Controller	Luồng xử lý	Giải mã
UserController.submitAnswer async submitAnswer(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Gửi dữ liệu thao tác của người dùng, xử lý nghiệp vụ và lưu kết quả.	Router gọi UserController.submitAnswer -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION submitAnswer(req, res, next) TRY userId = req.user.id { questionId, wordId, submittedAnswer, isCorrect, reviewRating, activityType } = req.body IF NOT questionId AND NOT wordId THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Thiếu questionId" } END IF feedback = NULL IF questionId THEN feedback = GỌI BẮT ĐỒNG BỘ UserService.submitAnswer({ userId: userId, questionId: questionId, wordId: wordId, submittedAnswer: submittedAnswer, isCorrect: isCorrect, reviewRating: reviewRating, activityType: activityType }) ... END FUNCTION
UserController.getFlashcards async getFlashcards(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getFlashcards -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getFlashcards(req, res, next) TRY userId = req.user.id flashcards = GỌI BẮT ĐỒNG BỘ UserService.getDueFlashcards(userId, { topicId: req.query.topicId, mode: req.query.mode }) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU flashcards CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
UserController.getTopicWords async getTopicWords(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getTopicWords -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getTopicWords(req, res, next) TRY userId = req.user.id { topicId } = req.params words = GỌI BẮT ĐỒNG BỘ UserService.getTopicWords(userId, topicId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU words CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
UserController.getStats async getStats(req, res, next)	Router gọi UserController.getStats -> middleware auth/validate	FUNCTION getStats(req, res, next) TRY userId = req.user.id

<i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	stats = GỌI BẮT ĐỒNG BỘ UserService.getUserStats(userId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU stats CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
UserController.getMasteryTimeline async getMasteryTimeline(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getMasteryTimeline -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getMasteryTimeline(req, res, next) TRY userId = req.user.id projection = GỌI BẮT ĐỒNG BỘ UserService.getMasteryTimeline(userId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU projection CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
UserController.getMiniTests async getMiniTests(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getMiniTests -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getMiniTests(req, res, next) TRY page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page) END IF pageSize = req.query.pageSize IF pageSize không tồn tại hoặc không chuyển được sang số nguyên THEN ... END FUNCTION
UserController.getMiniTestDetails async getMiniTestDetails(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getMiniTestDetails -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getMiniTestDetails(req, res, next) TRY { id } = req.params questions = GỌI BẮT ĐỒNG BỘ UserService.getMiniTestDetails(id) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU questions CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
UserController.updateProfile async updateProfile(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi UserController.updateProfile -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION updateProfile(req, res, next) TRY userId = req.user.id { fullName } = req.body result = GỌI BẮT ĐỒNG BỘ UserService.updateProfile(userId, fullName) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật thành công", data: result } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
UserController.getTestHistory async getTestHistory(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getTestHistory -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getTestHistory(req, res, next) TRY userId = req.user.id page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page) END IF pageSize = req.query.pageSize ... END FUNCTION
UserController.getTestSessionDetails async getTestSessionDetails(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getTestSessionDetails -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getTestSessionDetails(req, res, next) TRY userId = req.user.id { testId, date } = req.query details = GỌI BẮT ĐỒNG BỘ UserService.getTestSessionDetails(userId, testId, date) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU details CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION

UserController.getDailyGoal async getDailyGoal(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getDailyGoal -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getDailyGoal(req, res, next) TRY userId = req.user.id data = GỌI BẮT ĐỒNG BỘ UserService.getDailyGoal(userId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
UserController.updateDailyGoal async updateDailyGoal(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi UserController.updateDailyGoal -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION updateDailyGoal(req, res, next) TRY userId = req.user.id { dailyGoal } = req.body IF NOT dailyGoal OR dailyGoal < 5 OR dailyGoal > 100 THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Mục tiêu phải từ 5 đến 100 từ" } END IF result = GỌI BẮT ĐỒNG BỘ UserService.updateDailyGoal(userId, dailyGoal) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU result CATCH error ... END FUNCTION </pre>
UserController.updateSRConfig async updateSRConfig(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi UserController.updateSRConfig -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION updateSRConfig(req, res, next) TRY userId = req.user.id { srsReviewLimit } = req.body IF NOT srsReviewLimit OR srsReviewLimit < 5 OR srsReviewLimit > 50 THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Số thẻ mỗi ngày phải từ 5 đến 50" } END IF result = GỌI BẮT ĐỒNG BỘ UserService.updateSRReviewLimit(userId, srsReviewLimit) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU result CATCH error ... END FUNCTION </pre>
UserController.submitMiniTest async submitMiniTest(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Gửi dữ liệu thao tác của người dùng, xử lý nghiệp vụ và lưu kết quả.	Router gọi UserController.submitMiniTest -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION submitMiniTest(req, res, next) TRY userId = req.user.id { id } = req.params { answers } = req.body IF NOT Array.isArray(answers) OR answers.length = 0 THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Thiếu danh sách câu trả lời" } END IF result = GỌI BẮT ĐỒNG BỘ UserService.submitMiniTestBatch(userId, id, answers) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU result ... END FUNCTION </pre>
UserController.getActivityHeatmap async getActivityHeatmap(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getActivityHeatmap -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> trả HTTP response hoặc next(error)	<pre> FUNCTION getActivityHeatmap(req, res, next) TRY userId = req.user.id year = req.query.year IF year không tồn tại hoặc không chuyển được sang số nguyên THEN year = TẠO Date().getFullYear() ELSE year = CHUYỂN SANG SỐ NGUYÊN(year) END IF data = GỌI BẮT ĐỒNG BỘ UserService.getActivityHeatmap(userId, year) ... END FUNCTION </pre>
UserController.getProgressAnalytics async getProgressAnalytics(req, res, next)	Router gọi UserController.getProgressAnalytics -> middleware auth/validate kiểm tra request -> controller đọc	<pre> FUNCTION getProgressAnalytics(req, res, next) TRY data = GỌI BẮT ĐỒNG BỘ UserService.getProgressAnalytics(req.user.id) END TRY </pre>

<i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	<i>params/query/body/user</i> -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
UserController.getDailyProgress async getDailyProgress(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getDailyProgress -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getDailyProgress(req, res, next) TRY userId = req.user.id progress = GỌI BẮT ĐỒNG BỘ UserService.getDailyProgress(userId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU progress CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
UserController.getSmartReviewQueue async getSmartReviewQueue(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getSmartReviewQueue -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getSmartReviewQueue(req, res, next) TRY userId = req.user.id limit = req.query.limit IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN limit = 20 ELSE limit = CHUYỂN SANG SỐ NGUYÊN(limit) END IF queue = GỌI BẮT ĐỒNG BỘ UserService.getSmartReviewQueue(userId, limit) ... END FUNCTION
UserController.getNotebook async getNotebook(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getNotebook -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getNotebook(req, res, next) TRY userId = req.user.id page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page) END IF pageSize = req.query.pageSize ... END FUNCTION
UserController.addNotebookEntry async addNotebookEntry(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Xử lý nghiệp vụ add notebook entry trong module user learning - học từ vựng, srs, mini-test, notebook.	Router gọi UserController.addNotebookEntry -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION addNotebookEntry(req, res, next) TRY userId = req.user.id { wordId, personalNote } = req.body IF NOT wordId THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Thiếu wordId" } END IF entry = GỌI BẮT ĐỒNG BỘ UserService.addNotebookEntry(userId, wordId, personalNote) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Đã thêm vào sổ tay", data: entry } CATCH error ... END FUNCTION
UserController.updateNotebookEntry async updateNotebookEntry(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi UserController.updateNotebookEntry -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION updateNotebookEntry(req, res, next) TRY userId = req.user.id { id } = req.params { personalNote, isFavorite } = req.body entry = GỌI BẮT ĐỒNG BỘ UserService.updateNotebookEntry(id, userId, { personalNote: personalNote, isFavorite: isFavorite }) IF NOT entry THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mục trong sổ tay" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã cập nhật sổ tay", data: entry } ... END FUNCTION

UserController.deleteNotebookEntry async deleteNotebookEntry(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Xóa hoặc lưu trữ mềm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	Router gọi UserController.deleteNotebookEntry -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION deleteNotebookEntry(req, res, next) TRY userId = req.user.id { id } = req.params entry = GỌI BẮT ĐỒNG BỘ UserService.deleteNotebookEntry(id, userId) IF NOT entry THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mục trong sổ tay" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã xóa khỏi sổ tay" } CATCH error ... END FUNCTION </pre>
UserController.checkNotebookEntry async checkNotebookEntry(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Kiểm tra điều kiện hoặc trạng thái hiện tại trước khi trả kết quả.	Router gọi UserController.checkNotebookEntry -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION checkNotebookEntry(req, res, next) TRY userId = req.user.id wordId = CHUYỂN SANG SỐ NGUYÊN(req.query.wordId) IF NOT wordId THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Thiếu wordId" } END IF entry = GỌI BẮT ĐỒNG BỘ UserService.checkNotebookEntry(userId, wordId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU entry OR { } CATCH error ... END FUNCTION </pre>
UserController.getNotifications async getNotifications(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getNotifications -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getNotifications(req, res, next) TRY userId = req.user.id limit = req.query.limit IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN limit = 20 ELSE limit = CHUYỂN SANG SỐ NGUYÊN(limit) END IF data = GỌI BẮT ĐỒNG BỘ UserService.getUserNotifications(userId, limit) ... END FUNCTION </pre>
UserController.markNotificationRead async markNotificationRead(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Đánh dấu trạng thái đã xử lý/đã đọc cho dữ liệu của người dùng.	Router gọi UserController.markNotificationRead -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION markNotificationRead(req, res, next) TRY userId = req.user.id { id } = req.params THỰC HIỆN GỌI BẮT ĐỒNG BỘ UserService.markNotificationRead(userId, id) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã đánh dấu đã đọc" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
UserController.markAllNotificationsRead async markAllNotificationsRead(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Đánh dấu trạng thái đã xử lý/đã đọc cho dữ liệu của người dùng.	Router gọi UserController.markAllNotificationsRead -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION markAllNotificationsRead(req, res, next) TRY userId = req.user.id result = GỌI BẮT ĐỒNG BỘ UserService.markAllNotificationsRead(userId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU result CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
UserController.getSessionSummary async getSessionSummary(req, res, next) <i>controllers/user.controller.js</i>	Router gọi UserController.getSessionSummary -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getSessionSummary(req, res, next) TRY userId = req.user.id summary = GỌI BẮT ĐỒNG BỘ UserService.getSessionSummary(userId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU summary CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>

Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.		<pre>END TRY END FUNCTION</pre>
UserController.getMistakeReviewQueue async getMistakeReviewQueue(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi UserController.getMistakeReviewQueue -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre>FUNCTION getMistakeReviewQueue(req, res, next) TRY userId = req.user.id limit = req.query.limit IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN limit = 10 ELSE limit = CHUYỂN SANG SỐ NGUYÊN(limit) END IF queue = GỌI BẤT ĐỒNG BỘ UserService.getMistakeReviewQueue(userId, limit) ... END FUNCTION</pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
UserService.getFlashcards async getFlashcards(userId) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getFlashcards -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION getFlashcards(userId) pool = GỌI BẤT ĐỒNG BỘ poolPromise result = GỌI BẤT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Questions, Words, UserWordProgress] RETURN result.recordset END FUNCTION</pre>
UserService.getDueFlashcards async getDueFlashcards(userId, { topicId = null, mode = null } = {}) <i>services/user.service.js</i> Mục đích: Tính danh sách flashcard đến hạn ôn theo Spaced Repetition.	Controller hoặc module gọi UserService.getDueFlashcards -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION getDueFlashcards(userId, { topicId = null, mode = null } = { }) pool = GỌI BẤT ĐỒNG BỘ poolPromise result = GỌI BẤT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users, UserTopicEnrollments, Words, Questions, ExampleSentences] RETURN result.recordset END FUNCTION</pre>
UserService.getTopicWords async getTopicWords(userId, topicId) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getTopicWords -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION getTopicWords(userId, topicId) pool = GỌI BẤT ĐỒNG BỘ poolPromise result = GỌI BẤT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên WordTopics, ExampleSentences, Words, PartOfSpeeches, UserWordProgress] RETURN result.recordset END FUNCTION</pre>
UserService.submitAnswer async submitAnswer({ userId, questionId, wordId, submittedAnswer, isCorrect, reviewRating, activityType }) <i>services/user.service.js</i> Mục đích: Ghi nhận câu trả lời, cập nhật SRS, cộng XP và trả phản hồi học tập.	Controller hoặc module gọi UserService.submitAnswer -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION submitAnswer({ userId, questionId, wordId, submittedAnswer, isCorrect, reviewRating, activityType }) pool = GỌI BẤT ĐỒNG BỘ poolPromise result = GỌI BẤT ĐỒNG BỘ THỰC THI STORED PROCEDURE "usp_SubmitQuestionAttempt" canonicalWordId = Number(result.recordset[0].WordID OR 0) IF NOT canonicalWordId THEN THROW ERROR TẠO Error("Question does not resolve to a vocabulary word") END IF reviewFeedback = { } IF ["Again", "Hard", "Good", "Easy"].includes(reviewRating) THEN feedbackResult = GỌI BẤT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên UserWordProgress] ... END FUNCTION</pre>
UserService.submitWordReview async submitWordReview({ userId, wordId, isCorrect, reviewRating, activityType }) <i>services/user.service.js</i>	Controller hoặc module gọi UserService.submitWordReview -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION submitWordReview({ userId, wordId, isCorrect, reviewRating, activityType }) pool = GỌI BẤT ĐỒNG BỘ poolPromise result = GỌI BẤT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH MERGE trên SET, UserWordProgress] gamification = NẾU activityType = "LearnWord" THÌ GỌI BẤT ĐỒNG BỘ GamificationService.awardXP(userId, {</pre>

Mục đích: Cập nhật lịch ôn tiếp theo theo đánh giá Again/Hard/Good/Easy.		<pre> eventType: "LearnWord", sourceKey: `learn-word: \${ wordId }: \${ GamificationService.getDateKey() }`, metadata: { wordId: wordId, reviewRating: reviewRating OR NULL } }) NGUỒN LẠI NULL RETURN { ...result.recordset[0] OR { }, xpGained: gamification.xpGained OR 0, reviewRating: reviewRating OR NULL, gamification: gamification } END FUNCTION </pre>
UserService.getUserStats async getUserStats(userId) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getUserStats -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getUserStats(userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise learnedResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT COUNT(*) AS total FROM UserWordProgress WHERE UserID = @UserID AND MasteryLevel >= 3" accuracyResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts] weakWordsResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên UserWordProgress, Words] recentAttemptsResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, Words] gamification = GỌI BẮT ĐỒNG BỘ GamificationService.getProfile(userId) stats = { totalLearned: learnedResult.recordset[0].total, accuracy: Math.round(accuracyResult.recordset[0].a ccuracy OR 0), correct: accuracyResult.recordset[0].correct OR 0, wrong: accuracyResult.recordset[0].wrong OR 0, weakWords: weakWordsResult.recordset, recentAttempts: recentAttemptsResult.recordset, streak: gamification.streak, totalXP: gamification.totalXP, currentLevel: gamification.currentLevel, currentLevelXP: gamification.currentLevelXP, xpForNextLevel: gamification.xpForNextLevel, xpToNextLevel: gamification.xpToNextLevel, levelProgress: gamification.levelProgress, todayXP: gamification.todayXP } stats.masteryTimeline = GỌI BẮT ĐỒNG BỘ this.getMasteryTimeline(userId) trendsResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts] ... END FUNCTION </pre>
UserService.getMasteryTimeline async getMasteryTimeline(userId) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getMasteryTimeline -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getMasteryTimeline(userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise viewExists = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT] IF viewExists.recordset[0].viewId THEN result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên vw_MasteryTimelineProjection] IF result.recordset.length > 0 THEN RETURN result.recordset[0] END IF END IF result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên UserWordProgress] ... END FUNCTION </pre>
UserService.getMiniTests async getMiniTests(page = 1, pageSize = 20) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getMiniTests -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getMiniTests(page = 1, pageSize = 20) pool = GỌI BẮT ĐỒNG BỘ poolPromise page = Math.max(1, page) pageSize = Math.min(100, Math.max(1, pageSize)) offset = page - 1 * pageSize countResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests] total = countResult.recordset[0].total result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests, Topics] RETURN { data: result.recordset, </pre>

		total: total, page: page, pageSize: pageSize, totalPages: Math.ceil(total / pageSize) } END FUNCTION
UserService.getMiniTestDetails async getMiniTestDetails(testId) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getMiniTestDetails -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getMiniTestDetails(testId) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTestItems, Questions, Words] RETURN result.recordset END FUNCTION </pre>
UserService.updateProfile async updateProfile(userId, fullName) <i>services/user.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi UserService.updateProfile -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateProfile(userId, fullName) pool = GỌI BẮT ĐỒNG BỘ poolPromise THỰC THI TRUY VẤN "UPDATE Users SET FullName = @FullName, UpdatedAt = SYSDATETIMEOFFSET() WHERE UserID = @UserID" RETURN { id: userId, fullName: fullName } END FUNCTION </pre>
UserService.getTestHistory async getTestHistory(userId, page = 1, pageSize = 20) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getTestHistory -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getTestHistory(userId, page = 1, pageSize = 20) pool = GỌI BẮT ĐỒNG BỘ poolPromise page = Math.max(1, page) pageSize = Math.min(100, Math.max(1, pageSize)) offset = page - 1 * pageSize countResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, Questions, MiniTestItems, MiniTests] total = countResult.recordset[0].total result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, Questions, MiniTestItems, MiniTests] RETURN { data: result.recordset, total: total, page: page, pageSize: pageSize, totalPages: Math.ceil(total / pageSize) } END FUNCTION </pre>
UserService.getTestSessionDetails async getTestSessionDetails(userId, testId, date) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getTestSessionDetails -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getTestSessionDetails(userId, testId, date) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, Questions, MiniTestItems, Words] RETURN result.recordset END FUNCTION </pre>
UserService.getActivityHeatmap async getActivityHeatmap(userId, year) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getActivityHeatmap -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getActivityHeatmap(userId, year) THỰC HIỆN GỌI BẮT ĐỒNG BỘ GamificationService.ensureSchema() pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, UserXPEvents, DailyAttempts, DailyXP] RETURN result.recordset END FUNCTION </pre>
UserService.getProgressAnalytics async getProgressAnalytics(userId) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getProgressAnalytics -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getProgressAnalytics(userId) THỰC HIỆN GỌI BẮT ĐỒNG BỘ GamificationService.ensureSchema() pool = GỌI BẮT ĐỒNG BỘ poolPromise [activityResult, growthResult, topicResult, retentionResult, gamification] = GỌI BẮT ĐỒNG BỘ Promise.all([THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên DateSeries, ExerciseAttempts, UserXPEvents, DailyAttempts, DailyRewards], THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MonthOffsets, MonthSeries, UserWordProgress], THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, WordTopics, UserWordProgress], THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, UserWordProgress], </pre>

		<pre> GamificationService.getMetrics(userId)) retentionRow = retentionResult.recordset[0] OR { } totalAnswers = Number(retentionRow.totalAnswers OR 0) correctAnswers = Number(retentionRow.correctAnswers OR 0) learnedWords = Number(retentionRow.learnedWords OR 0) forgottenWords = Number(retentionRow.forgottenWords OR 0) upToDateWords = Number(retentionRow.upToDateWords OR 0) ... END FUNCTION </pre>
UserService.getDailyProgress async getDailyProgress(userId) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getDailyProgress -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getDailyProgress(userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts] RETURN { todayCount: result.recordset[0].count OR 0 } END FUNCTION </pre>
UserService.getSmartReviewQueue async getSmartReviewQueue(userId, limit = 20) <i>services/user.service.js</i> Mục đích: Tạo hàng đợi ôn thông minh, ưu tiên từ đến hạn và từ yếu.	Controller hoặc module gọi UserService.getSmartReviewQueue -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getSmartReviewQueue(userId, limit = 20) pool = GỌI BẮT ĐỒNG BỘ poolPromise limit = Math.min(50, Math.max(1, limit)) result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words, PartOfSpeeches, UserWordProgress] RETURN result.recordset END FUNCTION </pre>
UserService.submitMiniTestBatch async submitMiniTestBatch(userId, testId, answers) <i>services/user.service.js</i> Mục đích: Gửi dữ liệu thao tác của người dùng, xử lý nghiệp vụ và lưu kết quả.	Controller hoặc module gọi UserService.submitMiniTestBatch -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION submitMiniTestBatch(userId, testId, answers) pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) committed = false TRY BẮT ĐẦU TRANSACTION testQuestionsResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTestItems, Questions] testQuestions = TẠO Map(testQuestionsResult.recordset.map (HÀM CALLBACK(question))) IF testQuestions.size = 0 THEN THROW ERROR TẠO Error('Mini test \${ testId } does not contain questions') ... END FUNCTION </pre>
UserService.getDailyGoal async getDailyGoal(userId) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getDailyGoal -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getDailyGoal(userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users] RETURN result.recordset[0] OR { dailyGoal: 20, srsReviewLimit: 15 } END FUNCTION </pre>
UserService.updateDailyGoal async updateDailyGoal(userId, dailyGoal) <i>services/user.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi UserService.updateDailyGoal -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateDailyGoal(userId, dailyGoal) pool = GỌI BẮT ĐỒNG BỘ poolPromise goal = Math.min(100, Math.max(5, Number(dailyGoal) OR 20)) THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Users] RETURN { dailyGoal: goal } END FUNCTION </pre>
UserService.updateSRSReviewLimit async updateSRSReviewLimit(userId, limit) <i>services/user.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi UserService.updateSRSReviewLimit -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateSRSReviewLimit(userId, limit) pool = GỌI BẮT ĐỒNG BỘ poolPromise newLimit = Math.min(50, Math.max(5, Number(limit) OR 15)) THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Users] RETURN { srsReviewLimit: newLimit } END FUNCTION </pre>

UserService.getUserNotifications s async getUserNotifications(userId, limit = 20) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getUserNotifications ns -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getUserNotifications(userId, limit = 20) pool = GỌI BẮT ĐỒNG BỘ poolPromise limit = Math.min(50, Math.max(1, limit)) tableCheck = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT] IF NOT tableCheck.recordset[0].tableId THEN RETURN { notifications: [], unreadCount: 0 } END IF result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Notifications] countResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Notifications] RETURN { notifications: result.recordset, unreadCount: countResult.recordset[0].unread OR 0, total: countResult.recordset[0].total OR 0 } END FUNCTION </pre>
UserService.markNotificationRead async markNotificationRead(userId, notificationId) <i>services/user.service.js</i> Mục đích: Đánh dấu trạng thái đã xử lý/đã đọc cho dữ liệu của người dùng.	Controller hoặc module gọi UserService.markNotificationRead ad -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION markNotificationRead(userId, notificationId) pool = GỌI BẮT ĐỒNG BỘ poolPromise tableCheck = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT] IF NOT tableCheck.recordset[0].tableId THEN RETURN { success: false } END IF THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Notifications] RETURN { success: true } END FUNCTION </pre>
UserService.markAllNotificationsRead async markAllNotificationsRead(userId) <i>services/user.service.js</i> Mục đích: Đánh dấu trạng thái đã xử lý/đã đọc cho dữ liệu của người dùng.	Controller hoặc module gọi UserService.markAllNotificationsRead nsRead -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION markAllNotificationsRead(userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise tableCheck = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT] IF NOT tableCheck.recordset[0].tableId THEN RETURN { success: false, count: 0 } END IF result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Notifications] RETURN { success: true, count: result.rowsAffected[0] OR 0 } END FUNCTION </pre>
UserService.addNotebookEntry async addNotebookEntry(userId, wordId, personalNote) <i>services/user.service.js</i> Mục đích: Xử lý nghiệp vụ add notebook entry trong module user learning - học từ vựng, srs, mini-test, notebook.	Controller hoặc module gọi UserService.addNotebookEntry ns -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION addNotebookEntry(userId, wordId, personalNote) pool = GỌI BẮT ĐỒNG BỘ poolPromise existing = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên UserVocabularyNotebook] IF existing.recordset.length > 0 THEN result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên UserVocabularyNotebook] RETURN result.recordset[0] END IF result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên UserVocabularyNotebook] RETURN result.recordset[0] END FUNCTION </pre>
UserService.updateNotebookEntry async updateNotebookEntry(notebookId, userId, { personalNote, isFavorite }) <i>services/user.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi UserService.updateNotebookEntry ns -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateNotebookEntry(notebookId, userId, { personalNote, isFavorite }) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên UserVocabularyNotebook] RETURN result.recordset[0] OR NULL END FUNCTION </pre>
UserService.deleteNotebookEntry async	Controller hoặc module gọi UserService.deleteNotebookEntry ns -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn,	<pre> FUNCTION deleteNotebookEntry(notebookId, userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH DELETE trên UserVocabularyNotebook] </pre>

deleteNotebookEntry(notebookId, userId) <i>services/user.service.js</i> Mục đích: Xóa hoặc lưu trữ mềm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	RETURN result.recordset[0] OR NULL END FUNCTION
UserService.getSessionSummary async getSessionSummary(userId) <i>services/user.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi UserService.getSessionSummary -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION getSessionSummary(userId) THỰC HIỆN GỌI BẤT ĐỒNG BỘ GamificationService.ensureSchema() pool = GỌI BẤT ĐỒNG BỘ poolPromise result = GỌI BẤT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, Users, UserXPEvents, SessionStats, XPInfo] row = result.recordset[0] OR { totalAttempts: 0, correctCount: 0, wrongCount: 0, accuracy: 0, xpEarned: 0, TotalXP: 0, CurrentLevel: 1 } weakResult = GỌI BẤT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, Words] RETURN { totalAttempts: row.totalAttempts, correctCount: row.correctCount, wrongCount: row.wrongCount, accuracy: Number(row.accuracy), xpEarned: Number(row.xpEarned), totalXP: Number(row.TotalXP), currentLevel: Number(row.CurrentLevel), weakWords: weakResult.recordset } END FUNCTION
UserService.getMistakeReviewQueue async getMistakeReviewQueue(userId, limit = 10) <i>services/user.service.js</i> Mục đích: Lấy danh sách từ/câu hỏi người học hay sai để luyện lại.	Controller hoặc module gọi UserService.getMistakeReviewQueue -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION getMistakeReviewQueue(userId, limit = 10) pool = GỌI BẤT ĐỒNG BỘ poolPromise limit = Math.min(30, Math.max(1, limit)) result = GỌI BẤT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, Words, PartOfSpeeches, UserWordProgress] RETURN result.recordset END FUNCTION
UserService.checkNotebookEntry async checkNotebookEntry(userId, wordId) <i>services/user.service.js</i> Mục đích: Kiểm tra điều kiện hoặc trạng thái hiện tại trước khi trả kết quả.	Controller hoặc module gọi UserService.checkNotebookEntry -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION checkNotebookEntry(userId, wordId) pool = GỌI BẤT ĐỒNG BỘ poolPromise result = GỌI BẤT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên UserVocabularyNotebook] RETURN result.recordset[0] OR NULL END FUNCTION

4. Module Learning Path & Progress

Mục đích	Xây dựng lộ trình học theo topic và tính tiến độ tổng quát của người học.
Thành phần	learning-path.routes.js; progress.routes.js; LearningPathController; ProgressController; LearningPathService; ProgressService.
Luồng module	Learner -> learning/progress routes -> Controller -> Service -> đồng bộ topic đã publish -> tính roadmap/progress từ UserWordProgress.

Controller

Hàm Controller	Luồng xử lý	Giải mã
LearningPathController.getRoadmap async getRoadmap(req, res, next) <i>controllers/learning-path.controller.js</i>	Router gọi LearningPathController.getRoadmap -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ	FUNCTION getRoadmap(req, res, next) TRY roadmap = GỌI BẤT ĐỒNG BỘ LearningPathService.getRoadmap(req.userId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU roadmap CATCH error

Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	-> trả HTTP response hoặc next(error)	CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
ProgressController.getProgress async getProgress(req, res, next) <i>controllers/progress.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi ProgressController.getProgress -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getProgress(req, res, next) TRY userId = req.user.id progress = GỌI BẮT ĐỒNG BỘ ProgressService.getProgress(userId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU progress CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION
ProgressController.getStats async getStats(req, res, next) <i>controllers/progress.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi ProgressController.getStats -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	FUNCTION getStats(req, res, next) TRY userId = req.user.id stats = GỌI BẮT ĐỒNG BỘ ProgressService.getStats(userId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU stats CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giải mã
LearningPathService.syncPublishedTopics async syncPublishedTopics() <i>services/learning-path.service.js</i> Mục đích: Đồng bộ dữ liệu nguồn với cấu trúc hiển thị hoặc lộ trình học.	Controller hoặc module gọi LearningPathService.syncPublishedTopics -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION syncPublishedTopics() pool = GỌI BẮT ĐỒNG BỘ poolPromise THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, RankedTopics, LearningPathTopics, TopicsToMap, LearningPathLevels] END FUNCTION
LearningPathService.getRoadmap ap async getRoadmap(userId) <i>services/learning-path.service.js</i> Mục đích: Trả lộ trình học theo chủ đề đã xuất bản và tiến độ người dùng.	Controller hoặc module gọi LearningPathService.getRoadmap -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION getRoadmap(userId) THỰC HIỆN GỌI BẮT ĐỒNG BỘ Promise.all([this.ensureSchema(), GamificationService.ensureSchema()]) THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.syncPublishedTopics() pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên LearningPathLevels, LearningPathTopics, WordTopics, UserXPEvents, MiniTests] RETURN this.buildRoadmap(result.recordsets[0], result.recordsets[1]) END FUNCTION
LearningPathService.buildRoadmap buildRoadmap(levelRows, topicRows) <i>services/learning-path.service.js</i> Mục đích: Tổng hợp dữ liệu nguồn thành cấu trúc kết quả phục vụ giao diện.	Controller hoặc module gọi LearningPathService.buildRoadmap -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION buildRoadmap(levelRows, topicRows) allTopics = [] levels = levelRows.map(HÀM CALLBACK(level)) completedTopics = allTopics.filter(HÀM CALLBACK(topic)).length currentTopicIndex = allTopics.findIndex(HÀM CALLBACK(topic)) currentTopic = NẾU currentTopicIndex >= 0 THÌ allTopics[currentTopicIndex] NGƯỢC LẠI allTopics.at(-1) OR NULL nextTopic = NẾU currentTopicIndex >= 0 THÌ allTopics[currentTopicIndex + 1] OR NULL NGƯỢC LẠI NULL currentActivity = currentTopic.activities.find(HÀM CALLBACK(activity)) OR NULL RETURN { completionPercentage: NẾU allTopics.length > 0 THÌ Math.round(allTopics.reduce(HÀM CALLBACK(sum, topic), 0) / allTopics.length) NGƯỢC LẠI 0, completedTopics: completedTopics, totalTopics: allTopics.length, currentPosition: NẾU currentTopic THÌ { levelTitle: levels.find(HÀM CALLBACK(level)).title, topicId: currentTopic.topicId, topicTitle: currentTopic.title, topicStatus: currentTopic.status, activityTitle: currentActivity.title OR "Roadmap

		<pre> completed", activityRoute: currentActivity.route OR "/user/courses", completionPercentage: currentTopic.completionPercentage } NGUỘC LẠI NULL, currentLesson: NẾU currentTopic THÌ { topicId: currentTopic.topicId, title: currentTopic.title, route: `/user/learn/\${ currentTopic.topicId }`, status: currentTopic.status, completionPercentage: currentTopic.completionPercentage } NGUỘC LẠI NULL, nextLesson: NẾU nextTopic THÌ { topicId: nextTopic.topicId, title: nextTopic.title, route: `/user/learn/\${ nextTopic.topicId }`, status: nextTopic.status, completionPercentage: nextTopic.completionPercentage } NGUỘC LẠI NULL, levels: levels } END FUNCTION </pre>
ProgressService.getProgress async getProgress(userId) <i>services/progress.service.js</i> Mục đích: Tính tiến độ học tập theo topic và trạng thái ghi nhớ từng từ.	Controller hoặc module gọi ProgressService.getProgress -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getProgress(userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise totalResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên UserWordProgress] statsResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts] weakResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, Words] streakResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, DailyActivity, RankedDates] totalLearned = totalResult.recordset[0].totalLearned OR 0 correct = statsResult.recordset[0].correct OR 0 wrong = statsResult.recordset[0].wrong OR 0 streak = streakResult.recordset[0].streak OR 0 ... END FUNCTION </pre>

5. Module Gamification - XP, Level, Achievement

Mục đích	Tạo động lực học bằng XP, level, streak và achievement.
Thành phần	gamification.routes.js; GamificationController; GamificationService; bảng UserGamification, UserAchievements.
Luồng module	Learner action -> UserService/GamificationController -> GamificationService -> tính XP/level/achievement -> lưu DB -> trả profile/thành tích.

Controller

Hàm Controller	Luồng xử lý	Giả mã
GamificationController.getProfile async getProfile(req, res, next) <i>controllers/gamification.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi GamificationController.getProfile -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getProfile(req, res, next) TRY profile = GỌI BẮT ĐỒNG BỘ GamificationService.getProfile(req.user. id) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU profile CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
GamificationController.completePractice async completePractice(req, res, next) <i>controllers/gamification.controller.js</i>	Router gọi GamificationController.completePractice -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ	<pre> FUNCTION completePractice(req, res, next) TRY { sessionId, topicId = null, correctCount = 0, totalAttempts = 0 } = req.body IF NOT sessionId OR typeofsessionId ≠ "string" THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU </pre>

Mục đích: Xử lý nghiệp vụ complete practice trong module gamification - xp, level, achievement.	-> trả HTTP response hoặc next(error)	<pre> { message: "Missing practice session key" } END IF reward = GỌI BẮT ĐỒNG BỘ GamificationService.awardXP(req.user.id, { eventType: "PracticeComplete", sourceKey: `practice: \${ sessionKey.slice(0, 120) }`, metadata: { topicId: Number(topicId) OR NULL, correctCount: Number(correctCount) OR 0, totalAttempts: Number(totalAttempts) OR 0 } }) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU reward CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
--	---------------------------------------	--

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giải mã
GamificationService.awardXP async awardXP(userId, { eventType, sourceKey = null, metadata = null } = { }) <i>services/gamification.service.js</i> Mục đích: Cộng XP, cập nhật level và tránh cộng trùng theo sourceKey.	Controller hoặc module gọi GamificationService.awardXP -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION awardXP(userId, { eventType, sourceKey = null, metadata = null } = { }) THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema() amount = XP_REWARDS[eventType] IF NOT amount THEN THROW ERROR TẠO Error('Unsupported XP event type: \${ eventType }') END IF pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) BẮT ĐẦU TRANSACTION TRY ... END FUNCTION </pre>
GamificationService.awardDailyLogin async awardDailyLogin(userId) <i>services/gamification.service.js</i> Mục đích: Cộng thưởng đăng nhập hằng ngày và duy trì streak học tập.	Controller hoặc module gọi GamificationService.awardDailyLogin -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION awardDailyLogin(userId) RETURN this.awardXP(userId, { eventType: "DailyLogin", sourceKey: `daily-login: \${ this.getDateKey() }` }) END FUNCTION </pre>
GamificationService.checkAchievements async checkAchievements(userId) <i>services/gamification.service.js</i> Mục đích: Kiểm tra điều kiện đạt huy hiệu sau các hoạt động học.	Controller hoặc module gọi GamificationService.checkAchievements -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION checkAchievements(userId) THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema() profile = GỌI BẮT ĐỒNG BỘ this.getMetrics(userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Achievements, UserAchievements] unlockedIds = result.recordset.map(HÀM CALLBACK(row)) IF unlockedIds.length = 0 THEN RETURN [] END IF allAchievements = GỌI BẮT ĐỒNG BỘ this.getAchievements(userId, profile) RETURN allAchievements.filter(HÀM CALLBACK(achievement)) END FUNCTION </pre>
GamificationService.getProfile async getProfile(userId) <i>services/gamification.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi GamificationService.getProfile -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getProfile(userId) THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema() THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.checkAchievements(userId) metrics = GỌI BẮT ĐỒNG BỘ this.getMetrics(userId) achievements = GỌI BẮT ĐỒNG BỘ this.getAchievements(userId, metrics) RETURN { ...metrics, achievements: achievements, unseenAchievements: achievements.filter(HÀM CALLBACK(achievement)) } END FUNCTION </pre>

6. Module Admin - Quản trị hệ thống và nội dung

Mục đích	Quản trị toàn bộ nội dung học, người dùng, câu hỏi, mini-test, kiểm duyệt, analytics và thông báo.
Thành phần	admin.routes.js; AdminController; AdminService; auth permissions; bảng Topics, Words, Questions, MiniTests, Users, AuditLogs, Notifications.
Luồng module	Admin -> admin.routes -> verifyToken -> checkPermission/checkAnyPermission -> AdminController -> AdminService -> SQL Server/transaction -> audit log -> response.

Controller

Hàm Controller	Luồng xử lý	Giả mã
AdminController.getTopics async getTopics(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getTopics -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getTopics(req, res, next) TRY page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10) END IF limit = req.query.limit IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN ... END FUNCTION </pre>
AdminController.getTopicCategories async getTopicCategories(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getTopicCategories -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getTopicCategories(req, res, next) TRY categories = GỌI BẮT ĐỒNG BỘ AdminService.getTopicCategories() TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU categories CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.createTopic async createTopic(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Router gọi AdminController.createTopic -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION createTopic(req, res, next) TRY adminId = req.user.id result = GỌI BẮT ĐỒNG BỘ AdminService.createTopic(req.body, adminId) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo chủ đề thành công", data: result } CATCH error IF error.message = "Invalid topic data" OR error.message = "Topic already exists" OR error.message = "Topic code already exists" THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } END IF CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.updateTopic async updateTopic(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi AdminController.updateTopic -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION updateTopic(req, res, next) TRY success = GỌI BẮT ĐỒNG BỘ AdminService.updateTopic(req.params.id, req.body, req.user.id) IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy chủ đề" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật chủ đề thành công" } CATCH error IF ["Invalid topic data", "Topic already exists", "Topic code already exists"].includes(error.message) THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } ... END FUNCTION </pre>
AdminController.deleteTopic async deleteTopic(req, res, next) <i>controllers/admin.controller.js</i>	Router gọi AdminController.deleteTopic -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user	<pre> FUNCTION deleteTopic(req, res, next) TRY result = GỌI BẮT ĐỒNG BỘ AdminService.deleteTopic(req.params.id, req.user.id) IF NOT result.success THEN </pre>

Mục đích: Xóa hoặc lưu trữ tạm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	-> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy chủ đề" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: NẾU result.archived THÌ "Đã lưu trữ chủ đề vì đang có nội dung liên quan" NGƯỢC LẠI "Xóa chủ đề thành công", archived: result.archived } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.createTopicCategory async createTopicCategory(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Router gọi AdminController.createTopicCategory -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION createTopicCategory(req, res, next) TRY result = GỌI BẮT ĐỒNG BỘ AdminService.createTopicCategory(req.body, req.user.id) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo danh mục chủ đề thành công", data: result } CATCH error IF ["Invalid topic category data", "Topic category code already exists"].includes(error.message) THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } END IF CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.updateTopicCategory async updateTopicCategory(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi AdminController.updateTopicCategory -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION updateTopicCategory(req, res, next) TRY success = GỌI BẮT ĐỒNG BỘ AdminService.updateTopicCategory(req.params.id, req.body, req.user.id) IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy danh mục chủ đề" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật danh mục chủ đề thành công" } CATCH error IF ["Invalid topic category data", "Topic category code already exists"].includes(error.message) THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } ... END FUNCTION </pre>
AdminController.deleteTopicCategory async deleteTopicCategory(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Xóa hoặc lưu trữ tạm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	Router gọi AdminController.deleteTopicCategory -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION deleteTopicCategory(req, res, next) TRY success = GỌI BẮT ĐỒNG BỘ AdminService.deleteTopicCategory(req.params.id, req.user.id) IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy danh mục chủ đề" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã tắt danh mục chủ đề" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.getWords async getWords(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getWords -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getWords(req, res, next) TRY page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page) END IF limit = req.query.limit IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN ... END FUNCTION </pre>
AdminController.createWord async createWord(req, res, next) <i>controllers/admin.controller.js</i>	Router gọi AdminController.createWord -> middleware auth/validate kiểm tra request -> controller đọc	<pre> FUNCTION createWord(req, res, next) TRY adminId = req.user.id wordData = req.body result = GỌI BẮT ĐỒNG BỘ </pre>

Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	<pre>params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)</pre>	<pre>AdminService.createWord(wordData, adminId) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo từ vựng thành công", data: result } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION</pre>
AdminController.getWordDetail async getWordDetail(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	<pre>Router gọi AdminController.getWordDetail -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)</pre>	<pre>FUNCTION getWordDetail(req, res, next) TRY word = GỌI BẮT ĐỒNG BỘ AdminService.getWordDetail(req.params.id) IF NOT word THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy từ vựng" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU word CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION</pre>
AdminController.bulkImportWords async bulkImportWords(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Xử lý nhập hàng loạt, kiểm tra dữ liệu và tổng hợp kết quả nhập.	<pre>Router gọi AdminController.bulkImportWords -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)</pre>	<pre>FUNCTION bulkImportWords(req, res, next) TRY adminId = req.user.id result = GỌI BẮT ĐỒNG BỘ AdminService.bulkInsertWords(req.body, adminId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Import từ vựng hoàn tất", ...result } CATCH error IF error.message = "Invalid import payload" OR error.message.startsWith("CSV must include") THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } END IF CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION</pre>
AdminController.previewWordImport async previewWordImport(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Đọc dữ liệu nhập thử, kiểm tra lỗi và trả về bản xem trước.	<pre>Router gọi AdminController.previewWordImport -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)</pre>	<pre>FUNCTION previewWordImport(req, res, next) TRY result = GỌI BẮT ĐỒNG BỘ AdminService.previewWordImport(req.body) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU result CATCH error IF error.message = "Invalid import payload" OR error.message.startsWith("CSV must include") THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } END IF CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION</pre>
AdminController.updateWord async updateWord(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	<pre>Router gọi AdminController.updateWord -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)</pre>	<pre>FUNCTION updateWord(req, res, next) TRY { id } = req.params wordData = req.body success = GỌI BẮT ĐỒNG BỘ AdminService.updateWord(id, wordData, req.user.id) IF success THEN TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật thành công" } ELSE TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy từ vựng" } END IF ... END FUNCTION</pre>
AdminController.deleteWord async deleteWord(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Xóa hoặc lưu trữ mềm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	<pre>Router gọi AdminController.deleteWord -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)</pre>	<pre>FUNCTION deleteWord(req, res, next) TRY { id } = req.params success = GỌI BẮT ĐỒNG BỘ AdminService.archiveWord(id, req.user.id) IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy từ vựng" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa từ vựng thành công" }</pre>

		<pre> CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.hardDeleteWord async hardDeleteWord(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Xử lý nghiệp vụ hard delete word trong module admin - quản trị hệ thống và nội dung.	Router gọi AdminController.hardDeleteWord -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION hardDeleteWord(req, res, next) TRY { id } = req.params success = GỌI BẮT ĐỒNG BỘ AdminService.deleteWord(id, req.user.id) IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy từ vựng" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa vĩnh viễn từ vựng thành công" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.getQuestionsByWord async getQuestionsByWord(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getQuestionsByWord -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getQuestionsByWord(req, res, next) TRY { wordId } = req.params page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10) END IF limit = req.query.limit ... END FUNCTION </pre>
AdminController.createQuestion async createQuestion(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Router gọi AdminController.createQuestion -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION createQuestion(req, res, next) TRY adminId = req.user.id result = GỌI BẮT ĐỒNG BỘ AdminService.createQuestion(req.body, adminId) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo câu hỏi thành công", data: result } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.updateQuestion async updateQuestion(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi AdminController.updateQuestion -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION updateQuestion(req, res, next) TRY success = GỌI BẮT ĐỒNG BỘ AdminService.updateQuestion(req.params.i d, req.body, req.user.id) IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy câu hỏi" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật câu hỏi thành công" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.deleteQuestion async deleteQuestion(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Xóa hoặc lưu trữ mềm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	Router gọi AdminController.deleteQuestion -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION deleteQuestion(req, res, next) TRY success = GỌI BẮT ĐỒNG BỘ AdminService.deleteQuestion(req.params.i d, req.user.id) IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy câu hỏi" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa câu hỏi thành công" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.bulkImportQuestions async bulkImportQuestions(req, res, next) <i>controllers/admin.controller.js</i>	Router gọi AdminController.bulkImportQuestions -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION bulkImportQuestions(req, res, next) TRY adminId = req.user.id result = GỌI BẮT ĐỒNG BỘ AdminService.bulkInsertQuestions(req.bod y, adminId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Import câu hỏi hoàn tất", ...result } CATCH error </pre>

Mục đích: Xử lý nhập hàng loạt, kiểm tra dữ liệu và tổng hợp kết quả nhập.		<pre> IF error.message = "Invalid import payload" OR error.message.startsWith("CSV must include") THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } END IF CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.getMiniTests async getMiniTests(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getMiniTests -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getMiniTests(req, res, next) TRY page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10) END IF limit = req.query.limit IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN ... END FUNCTION </pre>
AdminController.createMiniTest async createMiniTest(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Router gọi AdminController.createMiniTest -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION createMiniTest(req, res, next) TRY adminId = req.user.id result = GỌI BẮT ĐỒNG BỘ AdminService.createMiniTest(req.body, adminId) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo Mini Test thành công", data: result } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.updateMiniTest async updateMiniTest(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi AdminController.updateMiniTest -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION updateMiniTest(req, res, next) TRY success = GỌI BẮT ĐỒNG BỘ AdminService.updateMiniTest(req.params.i d, req.body, req.user.id) IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mini test" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật mini test thành công" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.publishMiniTest async publishMiniTest(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Chuyển nội dung sang trạng thái công bố để người học có thể sử dụng.	Router gọi AdminController.publishMiniTest -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION publishMiniTest(req, res, next) TRY success = GỌI BẮT ĐỒNG BỘ AdminService.setMiniTestStatus(req.param s.id, "Published", req.user.id, "Published from mini test manager") IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mini test" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xuất bản mini test thành công" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.archiveMiniTest async archiveMiniTest(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Chuyển nội dung sang trạng thái lưu trữ, không xóa vật lý dữ liệu.	Router gọi AdminController.archiveMiniTest -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION archiveMiniTest(req, res, next) TRY success = GỌI BẮT ĐỒNG BỘ AdminService.setMiniTestStatus(req.param s.id, "Archived", req.user.id, "Archived from mini test manager") IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mini test" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Lưu trữ mini test thành công" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>

AdminController.getStats async getStats(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getStats -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getStats(req, res, next) TRY stats = GỌI BẮT ĐỒNG BỘ AdminService.getDashboardStats() TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU stats CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.getStudents async getStudents(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getStudents -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getStudents(req, res, next) TRY page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10) END IF limit = req.query.limit IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN ... END FUNCTION </pre>
AdminController.createUser async createUser(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Router gọi AdminController.createUser -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION createUser(req, res, next) TRY result = GỌI BẮT ĐỒNG BỘ AdminService.createUser(req.body) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo user thành công", data: result } CATCH error IF error.message = "Email already exists" OR error.message = "Invalid role" OR error.message = "Invalid user data" THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } END IF CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.updateUser async updateUser(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi AdminController.updateUser -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION updateUser(req, res, next) TRY { id } = req.params success = GỌI BẮT ĐỒNG BỘ AdminService.updateUser(id, req.body) IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy user" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật user thành công" } CATCH error IF error.message = "Email already exists" OR error.message = "Invalid role" OR error.message = "Invalid user data" THEN ... END FUNCTION </pre>
AdminController.toggleStudent Status async toggleStudentStatus(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Xử lý nghiệp vụ toggle student status trong module admin - quản trị hệ thống và nội dung.	Router gọi AdminController.toggleStudentS tatus -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION toggleStudentStatus(req, res, next) TRY { id } = req.params THỰC HIỆN GỌI BẮT ĐỒNG BỘ AdminService.toggleUserStatus(id) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật trạng thái thành công" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.updateUserRole async updateUserRole(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi AdminController.updateUserRole -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION updateUserRole(req, res, next) TRY { id } = req.params { role } = req.body THỰC HIỆN GỌI BẮT ĐỒNG BỘ AdminService.updateUserRole(id, role) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật vai trò thành công" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>

AdminController.getAnalytics async getAnalytics(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getAnalytics -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getAnalytics(req, res, next) TRY data = GỌI BẮT ĐỒNG BỘ AdminService.getAnalyticsData() TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.getContentManagement async getContentManagement(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getContentManagement -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getContentManagement(req, res, next) TRY data = GỌI BẮT ĐỒNG BỘ AdminService.getContentManagementData() TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.updateContentStatus async updateContentStatus(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Router gọi AdminController.updateContentStatus -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION updateContentStatus(req, res, next) TRY success = GỌI BẮT ĐỒNG BỘ AdminService.updateContentStatus(req.body, req.userId) IF NOT success THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy nội dung" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật trạng thái nội dung thành công" } CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.getPendingContent async getPendingContent(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getPendingContent -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getPendingContent(req, res, next) TRY data = GỌI BẮT ĐỒNG BỘ AdminService.getPendingContent() TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data CATCH error CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.getContentReviewLogs async getContentReviewLogs(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getContentReviewLogs -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getContentReviewLogs(req, res, next) TRY data = GỌI BẮT ĐỒNG BỘ AdminService.getContentReviewLogs(req.params.entityType, req.params.entityId) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data CATCH error IF error.message = "Invalid entity type" THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } END IF CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.getNotifications async getNotifications(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getNotifications -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getNotifications(req, res, next) TRY page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10) END IF limit = req.query.limit IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN ... END FUNCTION </pre>
AdminController.getAuditLogs async getAuditLogs(req, res, next)	Router gọi AdminController.getAuditLogs -> middleware auth/validate kiểm tra request -> controller đọc	<pre> FUNCTION getAuditLogs(req, res, next) TRY page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN </pre>

<i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	<i>params/query/body/user</i> -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10) END IF limit = req.query.limit IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN ... END FUNCTION </pre>
--	---	---

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giải mã
AdminService.normalizePagination normalizePagination(page = 1, limit = 20, maxLimit = 100) <i>services/admin.service.js</i> Mục đích: Xử lý nghiệp vụ normalize pagination trong module admin - quản trị hệ thống và nội dung.	Controller hoặc module gọi AdminService.normalizePaginat on -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION normalizePagination(page = 1, limit = 20, maxLimit = 100) normalizedPage = Math.max(CHUYỂN SANG SỐ NGUYÊN(page, 10) OR 1, 1) normalizedLimit = Math.min(Math.max(CHUYỂN SANG SỐ NGUYÊN(limit, 10) OR 20, 1), maxLimit) RETURN { page: normalizedPage, limit: normalizedLimit, offset: normalizedPage - 1 * normalizedLimit } END FUNCTION </pre>
AdminService.logAdminAction async logAdminAction(adminId, action, entityType, entityId, details = null) <i>services/admin.service.js</i> Mục đích: Ghi audit log cho thao tác quản trị quan trọng.	Controller hoặc module gọi AdminService.logAdminAction -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION logAdminAction(adminId, action, entityType, entityId, details = null) TRY pool = GỌI BẮT ĐỒNG BỘ poolPromise THỰC THI TRUY VẤN [CÂU LỆNH CREATE] THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên AdminAuditLogs] CATCH error THỰC HIỆN console.warn("Failed to write admin audit log", error.message) END TRY END FUNCTION </pre>
AdminService.logContentReview async logContentReview(entityType, entityId, oldStatus, newStatus, adminId, comment = null) <i>services/admin.service.js</i> Mục đích: Ghi log kiểm duyệt khi nội dung được duyệt/từ chối/lưu trữ.	Controller hoặc module gọi AdminService.logContentReview -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION logContentReview(entityType, entityId, oldStatus, newStatus, adminId, comment = null) pool = GỌI BẮT ĐỒNG BỘ poolPromise THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ContentReviewLogs] END FUNCTION </pre>
AdminService.parseDelimitedImport parseDelimitedImport(input) <i>services/admin.service.js</i> Mục đích: Phân tích file CSV/plain text nhập liệu thành các dòng dữ liệu chuẩn.	Controller hoặc module gọi AdminService.parseDelimitedImp ort -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION parseDelimitedImport(input) IF Array.isArray(input) THEN RETURN input END IF IF typeofinput = "object" AND input THEN IF Array.isArray(input.words) THEN RETURN input.words END IF IF Array.isArray(input.questions) THEN RETURN input.questions ... END FUNCTION </pre>
AdminService.getTopics async getTopics(page = 1, limit = 50, filters = { }) <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getTopics -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getTopics(page = 1, limit = 50, filters = { }) pool = GỌI BẮT ĐỒNG BỘ poolPromise paging = this.normalizePagination(page, limit, 200) search = String(filters.search NẾU RỒNG THÌ "").trim() status = String(filters.status NẾU RỒNG THÌ "").trim() categoryId = Number(filters.categoryId) OR NULL conditions = [] request = pool.request().input("Offset", sql.Int, paging.offset).input("Limit", sql.Int, paging.limit) IF search THEN THỰC HIỆN </pre>

		<pre> request.input("Search", sql.NVarChar(250), ` \${ search } `) ... END FUNCTION </pre>
AdminService.getTopicCategories async getTopicCategories() <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getTopicCategories -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getTopicCategories() pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên TopicCategories, Topics, WordTopics] RETURN result.recordset END FUNCTION </pre>
AdminService.createTopic async createTopic(topicData, adminId) <i>services/admin.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Controller hoặc module gọi AdminService.createTopic -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION createTopic(topicData, adminId) name = String(topicData.name NẾU RỖNG THÌ "").trim() description = String(topicData.description NẾU RỖNG THÌ "").trim() code = this.buildTopicCode(topicData.code OR name) topicCategoryId = NẾU topicData.topicCategoryId THÌ Number(topicData.topicCategoryId) NGƯỢC LẠI NULL status = topicData.status OR "Published" IF NOT name OR NOT adminId THEN THROW ERROR TẠO Error("Invalid topic data") END IF THỰC HIỆN this.assertContentStatus(status) ... END FUNCTION </pre>
AdminService.updateTopic async updateTopic(topicId, topicData, adminId) <i>services/admin.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi AdminService.updateTopic -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateTopic(topicId, topicData, adminId) name = NẾU topicData.name THÌ String(topicData.name).trim() NGƯỢC LẠI NULL description = NẾU topicData.description ≠ undefined THÌ String(topicData.description NẾU RỖNG THÌ "").trim() NGƯỢC LẠI NULL code = NẾU topicData.code THÌ this.buildTopicCode(topicData.code) NGƯỢC LẠI NULL topicCategoryId = NẾU topicData.topicCategoryId = undefined THÌ undefined NGƯỢC LẠI NẾU topicData.topicCategoryId THÌ Number(topicData.topicCategoryId) NGƯỢC LẠI NULL status = topicData.status OR NULL IF NOT name AND description = NULL AND NOT code AND topicCategoryId = undefined AND NOT status THEN THROW ERROR TẠO Error("Invalid topic data") END IF IF status THEN ... END FUNCTION </pre>
AdminService.deleteTopic async deleteTopic(topicId, adminId) <i>services/admin.service.js</i> Mục đích: Xóa hoặc lưu trữ tạm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	Controller hoặc module gọi AdminService.deleteTopic -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION deleteTopic(topicId, adminId) pool = GỌI BẮT ĐỒNG BỘ poolPromise dependencies = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, WordTopics, MiniTests] row = dependencies.recordset[0] IF NOT row.existsCount THEN RETURN { success: false, archived: false } END IF IF row.wordCount > 0 OR row.miniTestCount > 0 THEN THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.updateTopic(topicId, { status: "Archived" }, adminId) RETURN { success: true, archived: true } ... END FUNCTION </pre>
AdminService.createTopicCategory async	Controller hoặc module gọi AdminService.createTopicCategory -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn,	<pre> FUNCTION createTopicCategory(categoryData, adminId) name = String(categoryData.name NẾU RỖNG THÌ "").trim() code = </pre>

createTopicCategory(categoryData, adminId) <i>services/admin.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> this.buildTopicCode(categoryData.code OR name) description = String(categoryData.description NẾU RỖNG THÌ "").trim() iconUrl = String(categoryData.iconUrl NẾU RỖNG THÌ "").trim() displayOrder = Number(categoryData.displayOrder) OR 1 isActive = NẾU categoryData.isActive = undefined THÌ true NGƯỢC LẠI Boolean(categoryData.isActive) IF NOT name THEN THROW ERROR TẠO Error("Invalid topic category data") END IF ... END FUNCTION </pre>
AdminService.updateTopicCategory async updateTopicCategory(categoryId, categoryData, adminId) <i>services/admin.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi AdminService.updateTopicCategory -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateTopicCategory(categoryId, categoryData, adminId) name = String(categoryData.name NẾU RỖNG THÌ "").trim() code = this.buildTopicCode(categoryData.code OR name) description = String(categoryData.description NẾU RỖNG THÌ "").trim() iconUrl = String(categoryData.iconUrl NẾU RỖNG THÌ "").trim() displayOrder = Number(categoryData.displayOrder) OR 1 isActive = NẾU categoryData.isActive = undefined THÌ true NGƯỢC LẠI Boolean(categoryData.isActive) IF NOT name THEN THROW ERROR TẠO Error("Invalid topic category data") END IF ... END FUNCTION </pre>
AdminService.deleteTopicCategory async deleteTopicCategory(categoryId, adminId) <i>services/admin.service.js</i> Mục đích: Xóa hoặc lưu trữ mềm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	Controller hoặc module gọi AdminService.deleteTopicCategory -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION deleteTopicCategory(categoryId, adminId) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Topics, TopicCategories] success = result.rowsAffected.some(HÀM CALLBACK(count)) IF success THEN THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "DISABLE_TOPIC_CATEGORY", "TopicCategory", categoryId) END IF RETURN success END FUNCTION </pre>
AdminService.getWords async getWords(page = 1, limit = 20, filters = { }) <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getWords -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getWords(page = 1, limit = 20, filters = { }) pool = GỌI BẮT ĐỒNG BỘ poolPromise paging = this.normalizePagination(page, limit, 100) topicId = Number(filters.topicId) OR NULL partOfSpeechId = Number(filters.partOfSpeechId) OR NULL search = String(filters.search NẾU RỖNG THÌ "").trim() status = String(filters.status NẾU RỖNG THÌ "").trim() missingExamples = filters.missingExamples = true OR filters.missingExamples = "true" missingQuestions = filters.missingQuestions = true OR filters.missingQuestions = "true" sortBy = NẾU ["term", "createdAt", "updatedAt", "questionCount", "exampleCount"].includes(filters.sortBy) THÌ filters.sortBy NGƯỢC LẠI "createdAt" ... END FUNCTION </pre>
AdminService.attachWordRelations async	Controller hoặc module gọi AdminService.attachWordRelations -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn,	<pre> FUNCTION attachWordRelations(pool, words) IF NOT words.length THEN RETURN words END IF wordIds = words.map(HÀM </pre>

attachWordRelations(pool, words) <i>services/admin.service.js</i> Mục đích: Xử lý nghiệp vụ attach word relations trong module admin - quản trị hệ thống và nội dung.	transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>CALLBACK(word)).filter(Boolean) idList = wordIds.join(", ") relatedResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên WordTopics, ExampleSentences, Topics] topicsByWord = TẠO Map() examplesByWord = TẠO Map() FOR EACH topic IN relatedResult.recordsets[0] OR [] ... END FUNCTION</pre>
AdminService.getWordDetail async getWordDetail(wordId) <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getWordDetail -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION getWordDetail(wordId) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words, Questions, ExampleSentences, WordTopics, AdminAuditLogs] word = result.recordsets[0][0] IF NOT word THEN RETURN NULL END IF RETURN { ...word, topics: result.recordsets[1] OR [], examples: result.recordsets[2] OR [], questions: result.recordsets[3] OR [], auditLogs: result.recordsets[4] OR [] } END FUNCTION</pre>
AdminService.createWord async createWord(wordData, adminId) <i>services/admin.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Controller hoặc module gọi AdminService.createWord -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION createWord(wordData, adminId) { term, meaning, phonetic = '', partOfSpeechId, topicIds, examples, status = 'Published' } = wordData THỰC HIỆN this.assertContentStatus(status) normalizedTopicIds = NẾU Array.isArray(topicIds) THÌ [...new Set(topicIds.map((topicId) => Number(topicId)).filter(Boolean))] NGƯỢC LẠI [] validExamples = NẾU Array.isArray(examples) THÌ examples.filter(HÀM CALLBACK(ex)) NGƯỢC LẠI [] pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION request = TẠO sql.Request(transaction) ... END FUNCTION</pre>
AdminService.updateWord async updateWord(wordId, wordData, adminId = null) <i>services/admin.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi AdminService.updateWord -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION updateWord(wordId, wordData, adminId = null) { term, meaning, phonetic = '', partOfSpeechId, topicIds, examples, status = 'Published' } = wordData THỰC HIỆN this.assertContentStatus(status) normalizedTopicIds = NẾU Array.isArray(topicIds) THÌ [...new Set(topicIds.map((topicId) => Number(topicId)).filter(Boolean))] NGƯỢC LẠI NULL validExamples = NẾU Array.isArray(examples) THÌ examples.filter(HÀM CALLBACK(ex)) NGƯỢC LẠI NULL pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION updateReq = TẠO sql.Request(transaction) ... END FUNCTION</pre>
AdminService.deleteWord async deleteWord(wordId, adminId = null) <i>services/admin.service.js</i> Mục đích: Xóa hoặc lưu trữ mềm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	Controller hoặc module gọi AdminService.deleteWord -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION deleteWord(wordId, adminId = null) pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION request = TẠO sql.Request(transaction) result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words, MiniTestItems, Questions, MiniTests, ExerciseAttempts] COMMIT TRANSACTION success = result.recordset[0].deleted > 0</pre>

		<pre> IF success AND adminId THEN ... END FUNCTION </pre>
AdminService.bulkInsertWords async bulkInsertWords(input, adminId) <i>services/admin.service.js</i> Mục đích: Nhập nhiều từ vựng vào CSDL bằng transaction và trả thông kê kết quả.	Controller hoặc module gọi AdminService.bulkInsertWords -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION bulkInsertWords(input, adminId) rows = this.parseWordImport(input) results = { success: 0, failed: 0, errors: [] } pool = GỌI BẮT ĐỒNG BỘ poolPromise referenceData = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên PartOfSpeeches, Topics] partsOfSpeech = referenceData.recordsets[0] OR [] topics = referenceData.recordsets[1] OR [] partOfSpeechById = TẠO Map(partsOfSpeech.map(HÀM CALLBACK(item))) partOfSpeechByName = TẠO Map() topicById = TẠO Map(topics.map(HÀM CALLBACK(item))) ... END FUNCTION </pre>
AdminService.previewWordImport async previewWordImport(input) <i>services/admin.service.js</i> Mục đích: Đọc dữ liệu nhập thử, kiểm tra lỗi và trả về bản xem trước.	Controller hoặc module gọi AdminService.previewWordImport -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION previewWordImport(input) rows = this.parseWordImport(input) pool = GỌI BẮT ĐỒNG BỘ poolPromise referenceData = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên PartOfSpeeches, Topics] partsOfSpeech = referenceData.recordsets[0] OR [] topics = referenceData.recordsets[1] OR [] partOfSpeechById = TẠO Map(partsOfSpeech.map(HÀM CALLBACK(item))) partOfSpeechByName = TẠO Map() topicById = TẠO Map(topics.map(HÀM CALLBACK(item))) topicByName = TẠO Map() ... END FUNCTION </pre>
AdminService.getQuestionsByWord async getQuestionsByWord(wordId, page = 1, limit = 20, filters = { }) <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getQuestionsByWord -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getQuestionsByWord(wordId, page = 1, limit = 20, filters = { }) pool = GỌI BẮT ĐỒNG BỘ poolPromise paging = this.normalizePagination(page, limit, 100) search = String(filters.search NẾU RỖNG THÌ "").trim() type = String(filters.type NẾU RỖNG THÌ "").trim() status = String(filters.status NẾU RỖNG THÌ "").trim() conditions = ["WordID = @WordID"] request = pool.request().input("WordID", sql.BigInt, wordId).input("Offset", sql.Int, paging.offset).input("Limit", sql.Int, paging.limit) IF search THEN THỰC HIỆN request.input("Search", sql.NVarChar(250), `%\${ search }%`) ... END FUNCTION </pre>
AdminService.createQuestion async createQuestion(questionData, adminId) <i>services/admin.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Controller hoặc module gọi AdminService.createQuestion -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION createQuestion(questionData, adminId) { wordId, questionType, questionText, optionsJson = '[]', correctAnswer, explanation, status = 'Published' } = questionData THỰC HIỆN this.assertContentStatus(status) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Questions] created = result.recordset[0] THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "CREATE QUESTION", "Question", created.id, { wordId: wordId, questionType: questionType, status: status }) RETURN created END FUNCTION </pre>
AdminService.updateQuestion async	Controller hoặc module gọi AdminService.updateQuestion -> chuẩn hóa tham số đầu vào	<pre> FUNCTION updateQuestion(questionId, questionData, adminId) { wordId, questionType, </pre>

updateQuestion(questionId, questionData, adminId) <i>services/admin.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	-> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> questionText, optionsJson = '[]', correctAnswer, explanation, status = 'Published' } = questionData THỰC HIỆN this.assertContentStatus(status) pool = GỌI BẮT ĐỒNG BỘ poolPromise oldStatusResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT ContentStatus FROM Questions WHERE QuestionID = @QuestionID" IF oldStatusResult.recordset.length = 0 THEN RETURN false END IF oldStatus = oldStatusResult.recordset[0].ContentStat us result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Questions] ... END FUNCTION </pre>
AdminService.deleteQuestion async deleteQuestion(questionId, adminId) <i>services/admin.service.js</i> Mục đích: Xóa hoặc lưu trữ tạm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	Controller hoặc module gọi AdminService.deleteQuestion -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION deleteQuestion(questionId, adminId) pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION request = TẠO sql.Request(transaction) result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Questions, MiniTestItems, ExerciseAttempts, MiniTests, mt] COMMIT TRANSACTION success = result.recordset[0].deleted > 0 IF success THEN ... END FUNCTION </pre>
AdminService.bulkInsertQuestions async bulkInsertQuestions(input, adminId) <i>services/admin.service.js</i> Mục đích: Nhập nhiều câu hỏi liên kết từ vùng bằng transaction.	Controller hoặc module gọi AdminService.bulkInsertQuestions -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION bulkInsertQuestions(input, adminId) rows = this.parseQuestionImport(input) results = { success: 0, failed: 0, errors: [] } FOR let index = 0; WHILE index < rows.length; index TĂNG 1 row = rows[index] wordId = Number(row.wordId NẾU RỖNG THÌ row.WordID NẾU RỖNG THÌ row.wordID) questionType = row.questionType NẾU RỖNG THÌ row.QuestionType questionText = row.questionText NẾU RỖNG THÌ row.QuestionText correctAnswer = row.correctAnswer NẾU RỖNG THÌ row.CorrectAnswer optionsJson = row.optionsJson NẾU RỖNG THÌ row.OptionsJson NẾU RỖNG THÌ "[]" ... END FUNCTION </pre>
AdminService.getMiniTests async getMiniTests(page = 1, limit = 20, filters = { }) <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getMiniTests -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getMiniTests(page = 1, limit = 20, filters = { }) pool = GỌI BẮT ĐỒNG BỘ poolPromise paging = this.normalizePagination(page, limit, 100) search = String(filters.search NẾU RỖNG THÌ "").trim() status = String(filters.status NẾU RỖNG THÌ "").trim() topicId = Number(filters.topicId) OR NULL conditions = [] request = pool.request().input("Offset", sql.Int, paging.offset).input("Limit", sql.Int, paging.limit) IF search THEN THỰC HIỆN request.input("Search", sql.NVarChar(250), ` \${ search }`) ... END FUNCTION </pre>
AdminService.createMiniTest async createMiniTest(testData, adminId)	Controller hoặc module gọi AdminService.createMiniTest -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool	<pre> FUNCTION createMiniTest(testData, adminId) { title, description, topicId, questionIds } = testData </pre>

<i>services/admin.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	-> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION request = TẠO sql.Request(transaction) testResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên MiniTests] testId = testResult.recordset[0].id FOR let i = 0; WHILE i < questionIds.length; i TĂNG 1 ... END FUNCTION</pre>
AdminService.updateMiniTest async updateMiniTest(testId, testData, adminId) <i>services/admin.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi AdminService.updateMiniTest -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION updateMiniTest(testId, testData, adminId) { title, description, topicId, questionIds } = testData pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION updateReq = TẠO sql.Request(transaction) result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên MiniTests] IF result.rowsAffected[0] = 0 THEN ROLLBACK TRANSACTION ... END FUNCTION</pre>
AdminService.deleteMiniTest async deleteMiniTest(testId, adminId) <i>services/admin.service.js</i> Mục đích: Xóa hoặc lưu trữ mềm dữ liệu theo đúng quyền và quy tắc nghiệp vụ.	Controller hoặc module gọi AdminService.deleteMiniTest -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION deleteMiniTest(testId, adminId) pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION request = TẠO sql.Request(transaction) result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests, MiniTestAttempts, MiniTestItems] COMMIT TRANSACTION success = result.recordset[0].deleted > 0 IF success THEN ... END FUNCTION</pre>
AdminService.setMiniTestStatus async setMiniTestStatus(testId, status, adminId, comment = null) <i>services/admin.service.js</i> Mục đích: Xử lý nghiệp vụ set mini test status trong module admin - quản trị hệ thống và nội dung.	Controller hoặc module gọi AdminService.setMiniTestStatus -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION setMiniTestStatus(testId, status, adminId, comment = null) THỰC HIỆN this.assertContentStatus(status) pool = GỌI BẮT ĐỒNG BỘ poolPromise oldStatusResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT ContentStatus FROM MiniTests WHERE MiniTestID = @MiniTestID" IF oldStatusResult.recordset.length = 0 THEN RETURN false END IF oldStatus = oldStatusResult.recordset[0].ContentStat us result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên MiniTests] IF result.rowsAffected[0] > 0 THEN ... END FUNCTION</pre>
AdminService.getDashboardStats async getDashboardStats() <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getDashboardStats -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION getDashboardStats() pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users, Words, Topics, Questions, ExerciseAttempts] RETURN { ...result.recordsets[0][0], systemHealth: { apiStatus: "OK", databaseStatus: "Connected", environment: process.env.NODE_ENV OR "development", uptimeSeconds: Math.round(process.uptime()) }, userGrowth: result.recordsets[1], weeklyActivity: result.recordsets[2], userTypes: result.recordsets[3], recentActivity: result.recordsets[4] } END FUNCTION</pre>

AdminService.getStudents async getStudents(page = 1, limit = 20, filters = { }) <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getStudents -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getStudents(page = 1, limit = 20, filters = { }) pool = GỌI BẮT ĐỒNG BỘ poolPromise paging = this.normalizePagination(page, limit, 100) search = String(filters.search NẾU RÕNG THÌ "").trim() status = String(filters.status NẾU RÕNG THÌ "").trim() role = String(filters.role NẾU RÕNG THÌ "").trim() conditions = [] request = pool.request().input("Offset", sql.Int, paging.offset).input("Limit", sql.Int, paging.limit) IF search THEN THỰC HIỆN request.input("Search", sql.NVarChar(250), `\${ search }%`) ... END FUNCTION </pre>
AdminService.createUser async createUser(userData) <i>services/admin.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Controller hoặc module gọi AdminService.createUser -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION createUser(userData) { fullName, email, password, role = 'Learner', isActive = true } = userData THỰC HIỆN assertValidUserRole(role) IF NOT fullName OR NOT email OR NOT password OR password.length < 6 THEN THROW ERROR TẠO Error("Invalid user data") END IF pool = GỌI BẮT ĐỒNG BỘ poolPromise existing = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT UserID FROM Users WHERE Email = @Email" IF existing.recordset.length > 0 THEN THROW ERROR TẠO Error("Email already exists") ... END FUNCTION </pre>
AdminService.updateUser async updateUser(userId, userData) <i>services/admin.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi AdminService.updateUser -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateUser(userId, userData) { fullName, email, password, role = 'Learner', isActive = true } = userData THỰC HIỆN assertValidUserRole(role) IF NOT fullName OR NOT email THEN THROW ERROR TẠO Error("Invalid user data") END IF pool = GỌI BẮT ĐỒNG BỘ poolPromise existing = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT UserID FROM Users WHERE Email = @Email AND UserID <> @UserID" IF existing.recordset.length > 0 THEN THROW ERROR TẠO Error("Email already exists") ... END FUNCTION </pre>
AdminService.toggleUserStatus async toggleUserStatus(userId) <i>services/admin.service.js</i> Mục đích: Xử lý nghiệp vụ toggle user status trong module admin - quản trị hệ thống và nội dung.	Controller hoặc module gọi AdminService.toggleUserStatus -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION toggleUserStatus(userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise THỰC THI TRUY VẤN "UPDATE Users SET IsActive = CASE WHEN IsActive = 1 THEN 0 ELSE 1 END, UpdatedAt = SYSDATETIMEOFFSET() WHERE UserID = @UserID" RETURN true END FUNCTION </pre>
AdminService.updateUserRole async updateUserRole(userId, roleName) <i>services/admin.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi AdminService.updateUserRole -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateUserRole(userId, roleName) THỰC HIỆN assertValidUserRole(roleName) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Roles, Users] RETURN result.rowsAffected.some(HÀM CALLBACK(count)) END FUNCTION </pre>
AdminService.updateContentStatus async updateContentStatus({ entityType, entityId, status, comment }, adminId) <i>services/admin.service.js</i>	Controller hoặc module gọi AdminService.updateContentStatus -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateContentStatus({ entityType, entityId, status, comment }, adminId) THỰC HIỆN this.assertContentStatus(status) tableMap = { Topic: { table: "Topics", id: "TopicID" }, Word: { table: "Words", id: "WordID" }, Question: { table: "Questions", id: "QuestionID" }, MiniTest: { table: </pre>

Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.		<pre> "MiniTests", id: "MiniTestID", publishColumn: true } } target = tableMap[entityType] IF NOT target THEN THROW ERROR TẠO Error("Invalid entity type") END IF pool = GỌI BẮT ĐỒNG BỘ poolPromise oldStatusResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT] IF oldStatusResult.recordset.length = 0 THEN ... END FUNCTION </pre>
AdminService.getPendingContent async getPendingContent() <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getPendingContent -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getPendingContent() pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, Words, Questions, MiniTests, Users] RETURN result.recordset END FUNCTION </pre>
AdminService.getContentReviewLogs async getContentReviewLogs(entityType, entityId) <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getContentReviewLogs -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getContentReviewLogs(entityType, entityId) validEntityTypes = ["Topic", "Word", "Question", "MiniTest"] IF NOT validEntityTypes.includes(entityType) THEN THROW ERROR TẠO Error("Invalid entity type") END IF pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ContentReviewLogs, Users] RETURN result.recordset END FUNCTION </pre>
AdminService.getAnalyticsData async getAnalyticsData() <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getAnalyticsData -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getAnalyticsData() pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, Questions, PartOfSpeeches, Words, WordTopics] RETURN { summary: result.recordsets[0][0], popularQuizzes: result.recordsets[1], questionAccuracy: result.recordsets[2], studyActivity: result.recordsets[3], wordDistribution: result.recordsets[4], difficultTopics: result.recordsets[5] } END FUNCTION </pre>
AdminService.getContentManagementData async getContentManagementData() <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getContentManagementData -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getContentManagementData() pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, Words, Questions, MiniTests, TopicCategories] RETURN { summary: result.recordsets[0][0], content: result.recordsets[1], categories: result.recordsets[2], reviewLogs: result.recordsets[3] } END FUNCTION </pre>
AdminService.getNotifications async getNotifications(page = 1, limit = 50, filters = { }) <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getNotifications -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getNotifications(page = 1, limit = 50, filters = { }) pool = GỌI BẮT ĐỒNG BỘ poolPromise paging = this.normalizePagination(page, limit, 100) tableExists = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT] IF NOT tableExists.recordset[0].tableId THEN RETURN this.paginate([], 0, paging.page, paging.limit) END IF search = String(filters.search NẾU RÕNG THÌ "").trim() type = String(filters.type NẾU RÕNG THÌ "").trim() deliveryChannel = String(filters.deliveryChannel NẾU RÕNG THÌ "").trim() ... END FUNCTION </pre>

AdminService.getAuditLogs async getAuditLogs(page = 1, limit = 50, filters = { }) <i>services/admin.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi AdminService.getAuditLogs -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getAuditLogs(page = 1, limit = 50, filters = { }) pool = GỌI BẮT ĐỒNG BỘ poolPromise paging = this.normalizePagination(page, limit, 100) tableExists = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT] IF NOT tableExists.recordset[0].tableId THEN RETURN this.paginate([], 0, paging.page, paging.limit) END IF search = String(filters.search NẾU RỖNG THÌ "").trim() action = String(filters.action NẾU RỖNG THÌ "").trim() entityType = String(filters.entityType NẾU RỖNG THÌ "").trim() ... END FUNCTION </pre>
--	---	---

7. Module Creator - Biên tập nội dung học

Mục đích	Cho phép content creator tạo topic, từ vựng, câu hỏi, mini-test, media và gửi kiểm duyệt.
Thành phần	creator.routes.js; CreatorController; CreatorService; ReviewService; bảng Topics, Words, Questions, MiniTests, Media.
Luồng module	Creator -> creator.routes -> verifyToken/permission -> CreatorController -> CreatorService -> kiểm tra quyền sở hữu -> SQL Server -> submitForReview nếu gửi duyệt.

Controller

Hàm Controller	Luồng xử lý	Giả mã
CreatorController.getDashboard async getDashboard(req, res, next) <i>controllers/creator.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi CreatorController.getDashboard -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getDashboard(req, res, next) TRY stats = GỌI BẮT ĐỒNG BỘ CreatorService.getDashboardStats(req.user.id) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU stats CATCH err THỰC HIỆN next(err) END TRY END FUNCTION </pre>
CreatorController.getTopics async getTopics(req, res, next) <i>controllers/creator.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi CreatorController.getTopics -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getTopics(req, res, next) TRY data = GỌI BẮT ĐỒNG BỘ CreatorService.getMyTopics(req.user.id, req.query) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data CATCH err THỰC HIỆN next(err) END TRY END FUNCTION </pre>
CreatorController.createTopic async createTopic(req, res, next) <i>controllers/creator.controller.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Router gọi CreatorController.createTopic -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION createTopic(req, res, next) TRY result = GỌI BẮT ĐỒNG BỘ CreatorService.createTopic(req.body, req.user.id) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo chủ đề thành công", data: result } CATCH err IF ["Topic name or code already exists", "Topic category not found"].includes(err.message) THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err) } END IF THỰC HIỆN next(err) END TRY END FUNCTION </pre>
CreatorController.submitTopicForReview	Router gọi CreatorController.submitTopicForReview	<pre> FUNCTION submitTopicForReview(req, res, next) TRY </pre>

async submitTopicForReview(req, res, next) <i>controllers/creator.controller.js</i> Mục đích: Gửi dữ liệu thảo tác của người dùng, xử lý nghiệp vụ và lưu kết quả.	-> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> ok = GỌI BẮT ĐỒNG BỘ CreatorService.submitForReview("Topics", "TopicID", req.params.id, req.user.id) IF NOT ok THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Không thể gửi duyệt (sai trạng thái hoặc không có quyền)" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã gửi duyệt" } CATCH err THỰC HIỆN next(err) END TRY END FUNCTION </pre>
CreatorController.getWords async getWords(req, res, next) <i>controllers/creator.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi CreatorController.getWords -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getWords(req, res, next) TRY data = GỌI BẮT ĐỒNG BỘ CreatorService.getMyWords(req.user.id, req.query) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data CATCH err THỰC HIỆN next(err) END TRY END FUNCTION </pre>
CreatorController.createWord async createWord(req, res, next) <i>controllers/creator.controller.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Router gọi CreatorController.createWord -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION createWord(req, res, next) TRY result = GỌI BẮT ĐỒNG BỘ CreatorService.createWord(req.body, req.user.id) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo từ vựng thành công", data: result } CATCH err IF ["Invalid part of speech", "Word already exists for this part of speech", "Topic not found or unavailable"].includes(err.message) THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err) } END IF THỰC HIỆN next(err) END TRY END TRY END FUNCTION </pre>
CreatorController.submitWord ForReview async submitWordForReview(req, res, next) <i>controllers/creator.controller.js</i> Mục đích: Gửi dữ liệu thảo tác của người dùng, xử lý nghiệp vụ và lưu kết quả.	Router gọi CreatorController.submitWordFo rReview -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION submitWordForReview(req, res, next) TRY ok = GỌI BẮT ĐỒNG BỘ CreatorService.submitWordForReview(req.p arams.id, req.user.id) IF NOT ok THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Không thể gửi duyệt" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã gửi duyệt" } CATCH err IF err.message = "Word topics must be published before review" THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Các chủ đề gắn với từ vựng phải được xuất bản trước khi gửi duyệt." } ... END FUNCTION </pre>
CreatorController.getQuestions async getQuestions(req, res, next) <i>controllers/creator.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi CreatorController.getQuestions -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getQuestions(req, res, next) TRY data = GỌI BẮT ĐỒNG BỘ CreatorService.getMyQuestions(req.user.i d, req.query) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data CATCH err THỰC HIỆN next(err) END TRY END FUNCTION </pre>
CreatorController.createMiniT est async createMiniTest(req, res, next) <i>controllers/creator.controller.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Router gọi CreatorController.createMiniTe st -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION createMiniTest(req, res, next) TRY result = GỌI BẮT ĐỒNG BỘ CreatorService.createMiniTest(req.body, req.user.id) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo bài test thành công", data: result } CATCH err IF ["Mini test contains duplicate questions", "Mini test requires at least one question", "Topic not found or unavailable", "Mini test questions must be owned and </pre>

		<pre>published"].includes(err.message) THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err) } END IF THỰC HIỆN next(err) END TRY END FUNCTION</pre>
CreatorController.submitMiniTestForReview async submitMiniTestForReview(req, res, next) <i>controllers/creator.controller.js</i> Mục đích: Gửi dữ liệu thao tác của người dùng, xử lý nghiệp vụ và lưu kết quả.	Router gọi CreatorController.submitMiniTestForReview -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre>FUNCTION submitMiniTestForReview(req, res, next) TRY ok = GỌI BẮT ĐỒNG BỘ CreatorService.submitMiniTestForReview(req.params.id, req.user.id) IF NOT ok THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Không thể gửi duyệt" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã gửi duyệt" } CATCH err IF err.message.includes("Published") OR err.message.includes("it nhất một câu hỏi") THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: err.message } ... END FUNCTION</pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
CreatorService.getDashboardStats async getDashboardStats(userId) <i>services/creator.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi CreatorService.getDashboardStats -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION getDashboardStats(userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, Words, Questions, MiniTests] RETURN result.recordset[0] OR { } END FUNCTION</pre>
CreatorService.getMyTopics async getMyTopics(userId, filters = { }) <i>services/creator.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi CreatorService.getMyTopics -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION getMyTopics(userId, filters = { }) pool = GỌI BẮT ĐỒNG BỘ poolPromise page = Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.page) OR 1) pageSize = Math.min(100, Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.pageSize) OR 50)) offset = page - 1 * pageSize req = pool.request().input("UserID", sql.BigInt, userId) where = "t.CreatedByUserID = @UserID" IF filters.status THEN THỰC HIỆN req.input("Status", sql.NVarChar(20), filters.status) where = " AND t.ContentStatus = @Status" ... END FUNCTION</pre>
CreatorService.createTopic async createTopic(data, userId) <i>services/creator.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Controller hoặc module gọi CreatorService.createTopic -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION createTopic(data, userId) { topicName, topicCode, description, topicCategoryId } = data pool = GỌI BẮT ĐỒNG BỘ poolPromise IF topicCategoryId THEN category = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT TopicCategoryId FROM TopicCategories WHERE TopicCategoryId = @TopicCategoryId AND IsActive = 1" IF NOT category.recordset.length THEN THROW ERROR TẠO Error("Topic category not found") END IF END IF duplicate = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT TopicID FROM Topics WHERE TopicName = @TopicName OR TopicCode = @TopicCode" ... END FUNCTION</pre>

CreatorService.updateTopic async updateTopic(id, data, userId) <i>services/creator.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi CreatorService.updateTopic -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateTopic(id, data, userId) { topicName, topicCode, description, topicCategoryId } = data pool = GỌI BẮT ĐỒNG BỘ poolPromise IF topicCategoryId THEN category = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT TopicCategoryId FROM TopicCategories WHERE TopicCategoryId = @TopicCategoryId AND IsActive = 1" IF NOT category.recordset.length THEN THROW ERROR TẠO Error("Topic category not found") END IF END IF duplicate = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics] ... END FUNCTION </pre>
CreatorService.submitForReview async submitForReview(tableName, idColumn, id, userId) <i>services/creator.service.js</i> Mục đích: Chuyển nội dung của creator sang hàng đợi kiểm duyệt.	Controller hoặc module gọi CreatorService.submitForReview -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION submitForReview(tableName, idColumn, id, userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION req1 = TẠO sql.Request(transaction) upd = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE] IF upd.rowsAffected[0] = 0 THEN ROLLBACK TRANSACTION RETURN false ... END FUNCTION </pre>
CreatorService.getMyWords async getMyWords(userId, filters = { }) <i>services/creator.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi CreatorService.getMyWords -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getMyWords(userId, filters = { }) pool = GỌI BẮT ĐỒNG BỘ poolPromise page = Math.max(1, CHUYÊN SANG SỐ NGUYÊN(filters.page) OR 1) pageSize = Math.min(100, Math.max(1, CHUYÊN SANG SỐ NGUYÊN(filters.pageSize) OR 50)) offset = page - 1 * pageSize req = pool.request().input("UserID", sql.BigInt, userId) where = "w.CreatedByUserID = @UserID" IF filters.status THEN THỰC HIỆN req.input("Status", sql.NVarChar(20), filters.status) where = " AND w.ContentStatus = @Status" ... END FUNCTION </pre>
CreatorService.createWord async createWord(data, userId) <i>services/creator.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Controller hoặc module gọi CreatorService.createWord -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION createWord(data, userId) { term, meaning, phonetic, partOfSpeechId, topicIds, examples } = data normalizedTopicIds = [...new Set(Array.isArray(topicIds) ? topicIds: [])] pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION ownershipRequest = TẠO sql.Request(transaction) references = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT PartOfSpeechID FROM PartOfSpeeches WHERE PartOfSpeechID = @PartOfSpeechID" IF NOT references.recordset.length THEN ... END FUNCTION </pre>
CreatorService.updateWord async updateWord(id, data, userId) <i>services/creator.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi CreatorService.updateWord -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateWord(id, data, userId) { term, meaning, phonetic, partOfSpeechId, topicIds = [], examples = [] } = data normalizedTopicIds = [...new Set(topicIds)] pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION partOfSpeechRequest = TẠO sql.Request(transaction) partOfSpeech = GỌI BẮT ĐỒNG BỘ </pre>

		THỰC THI TRUY VẤN "SELECT PartOfSpeechID FROM PartOfSpeeches WHERE PartOfSpeechID = @PartOfSpeechID" IF NOT partOfSpeech.recordset.length THEN ... END FUNCTION
CreatorService.createQuestion async createQuestion(data, userId) <i>services/creator.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Controller hoặc module gọi CreatorService.createQuestion -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION createQuestion(data, userId) { wordId, questionType, questionText, optionsJson, correctAnswer, explanation } = data pool = GỌI BẮT ĐỒNG BỘ poolPromise normalizedOptionsJson = JSON.stringify(this.parseQuestionOptions (questionType, optionsJson, correctAnswer)) word = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words] IF NOT word.recordset.length THEN THROW ERROR TẠO Error("Word not found or unavailable") END IF result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Questions] RETURN result.recordset[0] END FUNCTION
CreatorService.getMyMiniTests async getMyMiniTests(userId, filters = { }) <i>services/creator.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi CreatorService.getMyMiniTests -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION getMyMiniTests(userId, filters = { }) pool = GỌI BẮT ĐỒNG BỘ poolPromise page = Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.page) OR 1) pageSize = Math.min(100, Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.pageSize) OR 50)) offset = page - 1 * pageSize req = pool.request().input("UserID", sql.BigInt, userId) where = "mt.CreatedByUserID = @UserID" IF filters.status THEN THỰC HIỆN req.input("Status", sql.NVarChar(20), filters.status) where = " AND mt.ContentStatus = @Status" ... END FUNCTION
CreatorService.createMiniTest async createMiniTest(data, userId) <i>services/creator.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Controller hoặc module gọi CreatorService.createMiniTest -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION createMiniTest(data, userId) { title, description, topicId, questionIds } = data pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION normalizedQuestionIds = GỌI BẮT ĐỒNG BỘ this.validateMiniTestReferences(transact ion, topicId, questionIds, userId) req = TẠO sql.Request(transaction) testResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên MiniTests] testId = testResult.recordset[0].id ... END FUNCTION
CreatorService.addMiniTestItem async addMiniTestItem(miniTestId, questionId, userId) <i>services/creator.service.js</i> Mục đích: Xử lý nghiệp vụ add mini test item trong module creator - biên tập nội dung học.	Controller hoặc module gọi CreatorService.addMiniTestItem -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	FUNCTION addMiniTestItem(miniTestId, questionId, userId) pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION checkRequest = TẠO sql.Request(transaction) check = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests, Questions, MiniTestItems] IF NOT check.recordset.length THEN ROLLBACK TRANSACTION RETURN false ... END FUNCTION

8. Module Review - Kiểm duyệt nội dung

Mục đích	Quản lý hàng đợi duyệt nội dung và nhật ký duyệt/từ chối/lưu trữ.
Thành phần	review.routes.js; ReviewController; ReviewService; AdminService review endpoints; bảng ContentReviewLogs.
Luồng module	Reviewer/Admin -> review routes hoặc admin content-review routes -> Controller -> ReviewService/AdminService -> cập nhật ContentStatus -> ghi ContentReviewLogs.

Controller

Hàm Controller	Luồng xử lý	Giả mã
ReviewController.getPending async getPending(req, res, next) <i>controllers/review.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi ReviewController.getPending -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getPending(req, res, next) TRY data = GỌI BẮT ĐỒNG BỘ ReviewService.getPendingContent() TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data CATCH err THỰC HIỆN next(err) END TRY END FUNCTION </pre>
ReviewController.approve async approve(req, res, next) <i>controllers/review.controller.js</i> Mục đích: Duyệt nội dung đang chờ kiểm tra và cập nhật trạng thái xuất bản.	Router gọi ReviewController.approve -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION approve(req, res, next) TRY { entityType, entityId } = req.params ok = GỌI BẮT ĐỒNG BỘ ReviewService.approve(entityType, entityId, req.user.id) IF NOT ok THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy nội dung chờ duyệt" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã duyệt thành công" } CATCH err IF err.message = "EntityType không hợp lệ" THEN ... END FUNCTION </pre>
ReviewController.reject async reject(req, res, next) <i>controllers/review.controller.js</i> Mục đích: Từ chối nội dung, lưu lý do và trả về trạng thái cần chỉnh sửa.	Router gọi ReviewController.reject -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION reject(req, res, next) TRY { entityType, entityId } = req.params { reason } = req.body ok = GỌI BẮT ĐỒNG BỘ ReviewService.reject(entityType, entityId, req.user.id, reason) IF NOT ok THEN TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy nội dung chờ duyệt" } END IF TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã từ chối" } CATCH err ... END FUNCTION </pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
ReviewService.getPendingContent async getPendingContent() <i>services/review.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi ReviewService.getPendingContent -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getPendingContent() pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, Words, Questions, MiniTests, Users] RETURN result.recordset END FUNCTION </pre>
ReviewService._resolveEntity _resolveEntity(entityType) <i>services/review.service.js</i>	Controller hoặc module gọi ReviewService._resolveEntity -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan	<pre> FUNCTION _resolveEntity(entityType) input = String(entityType OR "").toLowerCase() map = { topic: "Topic", table: "Topics", idCol: "TopicID", publishedAtCol: "PublishedAt", reviewedByCol: "ReviewedByUserID", reviewedAtCol: "ReviewedAt" }, word: { </pre>

Mục đích: Xử lý nghiệp vụ _resolve entity trong module review - kiểm duyệt nội dung.	-> trả dữ liệu đã tổng hợp về controller	<pre> type: "Word", table: "Words", idCol: "WordID", publishedAtCol: "PublishedAt", reviewedByCol: "ReviewedByUserID", reviewedAtCol: "ReviewedAt" }, question: { type: "Question", table: "Questions", idCol: "QuestionID", publishedAtCol: "PublishedAt", reviewedByCol: "ReviewedByUserID", reviewedAtCol: "ReviewedAt" }, minitest: { type: "MiniTest", table: "MiniTests", idCol: "MiniTestID", publishedAtCol: "PublishedAt", reviewedByCol: "ReviewedByUserID", reviewedAtCol: "ReviewedAt" } } entity = map[input] IF NOT entity THEN THROW ERROR TẠO Error("EntityType không hợp lệ") END IF RETURN entity END FUNCTION </pre>
ReviewService.approve async approve(entityType, entityId, adminId) <i>services/review.service.js</i> Mục đích: Duyệt nội dung và ghi nhật ký kiểm duyệt.	Controller hoặc module gọi ReviewService.approve -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION approve(entityType, entityId, adminId) e = this._resolveEntity(entityType) pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION extraSet = "" IF e.publishedAtCol THEN extraSet = `, \${ e.publishedAtCol } = SYSDATETIMEOFFSET(` END IF ... END FUNCTION </pre>
ReviewService.reject async reject(entityType, entityId, adminId, reason) <i>services/review.service.js</i> Mục đích: Từ chối nội dung kèm ghi chú phản hồi cho người tạo.	Controller hoặc module gọi ReviewService.reject -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION reject(entityType, entityId, adminId, reason) e = this._resolveEntity(entityType) pool = GỌI BẮT ĐỒNG BỘ poolPromise transaction = TẠO sql.Transaction(pool) TRY BẮT ĐẦU TRANSACTION extraSet = "" IF e.reviewedByCol THEN extraSet = `, \${ e.reviewedByCol } = @AdminID` END IF ... END FUNCTION </pre>

9. Module AI - Hỗ trợ soạn nội dung TOEIC

Mục đích	Gợi ý nội dung từ vựng, ví dụ TOEIC, dịch nghĩa và tra cứu từ điển để hỗ trợ biên tập.
Thành phần	ai.routes.js; AiController; AiService; OpenAI client/dictionary fallback.
Luồng module	Creator/Admin -> ai.routes -> AiController -> AiService -> OpenAI/dictionary -> chuẩn hóa JSON/text -> response.

Controller

Hàm Controller	Luồng xử lý	Giả mã
AiController.suggestWordContent async suggestWordContent(req, res, next) <i>controllers/ai.controller.js</i> Mục đích: Xử lý nghiệp vụ suggest word content trong module ai - hỗ trợ soạn nội dung toEIC.	Router gọi AiController.suggestWordContent -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION suggestWordContent(req, res, next) TRY suggestion = GỌI BẮT ĐỒNG BỘ AiService.suggestWordContent(req.body) TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU suggestion CATCH error IF error.statusCode THEN TRẢ VỀ HTTP error.statusCode CÙNG DỮ LIỆU { message: error.message } END IF CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>

Service / Middleware / Config

Hàm Service	Lưu đồ xử lý	Giả mã
withTimeout withTimeout(ms = 12000) <i>services/ai.service.js</i> Mục đích: Xử lý nghiệp vụ with timeout trong module ai - hỗ trợ soạn nội dung toEIC.	Controller hoặc module gọi withTimeout -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION withTimeout(ms = 12000) controller = TẠO AbortController() timeout = setTimeout(HÀM CALLBACK(), ms) RETURN { signal: controller.signal, clear: HÀM CALLBACK() } END FUNCTION </pre>
extractResponseText extractResponseText(response) <i>services/ai.service.js</i> Mục đích: Xử lý nghiệp vụ extract response text trong module ai - hỗ trợ soạn nội dung toEIC.	Controller hoặc module gọi extractResponseText -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION extractResponseText(response) IF typeof response.output_text = "string" THEN RETURN response.output_text END IF chunks = [] FOR EACH item IN response.output OR [] IF item.type ≠ "message" THEN CONTINUE END IF FOR EACH content IN item.content OR [] ... END FUNCTION </pre>
parseJsonText parseJsonText(text) <i>services/ai.service.js</i> Mục đích: Xử lý nghiệp vụ parse json text trong module ai - hỗ trợ soạn nội dung toEIC.	Controller hoặc module gọi parseJsonText -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION parseJsonText(text) raw = cleanText(text) IF NOT raw THEN THROW ERROR TẠO Error("AI response is empty") END IF TRY RETURN JSON.parse(raw) CATCH error match = raw.match(/\{ [\s\S]*\ }/) IF NOT match THEN ... END FUNCTION </pre>
AiService.suggestWordContent async suggestWordContent({ term, meaning, partOfSpeech, exampleCount = 3 }) <i>services/ai.service.js</i> Mục đích: Gọi AI để gợi ý nghĩa, ví dụ, phát âm và nội dung TOEIC cho từ mới.	Controller hoặc module gọi AiService.suggestWordContent -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION suggestWordContent({ term, meaning, partOfSpeech, exampleCount = 3 }) cleanTerm = cleanText(term) IF NOT cleanTerm THEN THROW ERROR TẠO Error("Missing term") END IF [dictionary, translatedTerm] = GỌI BẮT ĐỒNG BỘ Promise.all([this.lookupDictionary(clean Term).catch(HÀM CALLBACK()), this.translateToVietnamese(cleanTerm).ca tch(HÀM CALLBACK())]) aiSuggestion = GỌI BẮT ĐỒNG BỘ this.generateToEICExamples({ term: cleanTerm, meaning: cleanText(meaning) OR translatedTerm OR dictionary.meaning OR "", partOfSpeech: cleanText(partOfSpeech) OR dictionary.partOfSpeech OR "", exampleCount: exampleCount }) RETURN { term: cleanTerm, meaning: aiSuggestion.meaning OR translatedTerm OR dictionary.meaning OR "", phonetic: dictionary.phonetic OR aiSuggestion.phonetic OR "", partOfSpeech: aiSuggestion.partOfSpeech OR dictionary.partOfSpeech OR "", examples: NẾU aiSuggestion.examples.length > 0 THÌ aiSuggestion.examples NGƯỢC LẠI dictionary.examples OR [], sources: { ai: "openai", translate: NẾU translatedTerm THÌ "google-translate" NGƯỢC LẠI NULL, dictionary: NẾU dictionary THÌ "dictionaryapi.dev" NGƯỢC LẠI NULL } } END FUNCTION </pre>
AiService.generateToEICExamples async generateToEICExamples({ term, meaning, partOfSpeech, exampleCount }) <i>services/ai.service.js</i>	Controller hoặc module gọi AiService.generateToEICExamples -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan	<pre> FUNCTION generateToEICExamples({ term, meaning, partOfSpeech, exampleCount }) apiKey = process.env.OPENAI_API_KEY IF NOT apiKey THEN error = TẠO Error("OPENAI_API_KEY is not configured") error.statusCode = 503 THROW ERROR error </pre>

Mục đích: Xử lý nghiệp vụ generate toEIC examples trong module ai - hỗ trợ soạn nội dung toEIC.	-> trả dữ liệu đã tổng hợp về controller	<pre> END IF model = process.env.OPENAI_MODEL OR process.env.AI_MODEL OR "gpt-5.4-mini" { signal, clear } = withTimeout(20000) TRY ... END FUNCTION </pre>
--	--	--

10. Module Report & Notification - Báo cáo và thông báo

Mục đích	Tiếp nhận báo cáo từ người dùng và gửi/đọc thông báo hệ thống.
Thành phần	ReportService; UserController notification endpoints; AdminController notification/report endpoints; bảng Reports, Notifications.
Luồng module	User tạo report/thông báo -> UserController/ReportService. Admin xem/xử lý report hoặc gửi announcement -> AdminController/AdminService -> Notifications/Reports.

Controller

Hàm Controller	Luồng xử lý	Giả mã
UserController.createReport async createReport(req, res, next) <i>controllers/user.controller.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Router gọi UserController.createReport -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION createReport(req, res, next) TRY result = GỌI BẮT ĐỒNG BỘ ReportService.createReport(req.user.id, req.body) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Report submitted", data: result } CATCH error IF ["Invalid report type", "Invalid entity type", "Report description is too short"].includes(error.message) THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } END IF CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>
AdminController.getReports async getReports(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Router gọi AdminController.getReports -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION getReports(req, res, next) TRY page = req.query.page IF page không tồn tại hoặc không chuyển được sang số nguyên THEN page = 1 ELSE page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10) END IF limit = req.query.limit IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN ... END FUNCTION </pre>
AdminController.sendAnnouncement async sendAnnouncement(req, res, next) <i>controllers/admin.controller.js</i> Mục đích: Tạo và gửi thông báo đến nhóm người dùng phù hợp.	Router gọi AdminController.sendAnnouncement -> middleware auth/validate kiểm tra request -> controller đọc params/query/body/user -> gọi service xử lý nghiệp vụ -> trả HTTP response hoặc next(error)	<pre> FUNCTION sendAnnouncement(req, res, next) TRY result = GỌI BẮT ĐỒNG BỘ AdminService.sendAnnouncement(req.body) TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Announcement queued", data: result } CATCH error IF error.message = "Missing title or message" THEN TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message } END IF CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ END TRY END FUNCTION </pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
-------------	-------------	--------

ReportService.createReport async createReport(userId, reportData) <i>services/report.service.js</i> Mục đích: Tạo báo cáo từ người dùng cho nội dung hoặc sự cố trong hệ thống.	Controller hoặc module gọi ReportService.createReport -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION createReport(userId, reportData) THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema() reportType = String(reportData.reportType OR "").trim() entityType = this.inferEntityType({ entityType: String(reportData.entityType OR "").trim(), questionId: reportData.questionId, wordId: reportData.wordId, reportType: reportType }) wordId = Number(reportData.wordId) OR NULL questionId = Number(reportData.questionId) OR NULL title = String(reportData.title OR reportType).trim().slice(0, 200) description = String(reportData.description OR "").trim().slice(0, 2000) THỰC HIỆN this.assertOneOf(reportType, REPORT_TYPES, "report type") THỰC HIỆN this.assertOneOf(entityType, ENTITY_TYPES, "entity type") ... END FUNCTION </pre>
ReportService.getReports async getReports(page = 1, limit = 20, filters = { }) <i>services/report.service.js</i> Mục đích: Lấy dữ liệu, áp dụng điều kiện lọc/phân quyền và trả về cho màn hình sử dụng.	Controller hoặc module gọi ReportService.getReports -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION getReports(page = 1, limit = 20, filters = { }) THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema() pool = GỌI BẮT ĐỒNG BỘ poolPromise paging = this.normalizePagination(page, limit, 100) search = String(filters.search NẾU RỒNG THÌ "").trim() status = String(filters.status NẾU RỒNG THÌ "").trim() reportType = String(filters.reportType NẾU RỒNG THÌ "").trim() entityType = String(filters.entityType NẾU RỒNG THÌ "").trim() priority = String(filters.priority NẾU RỒNG THÌ "").trim() conditions = [] ... END FUNCTION </pre>
ReportService.updateReport async updateReport(reportId, data, adminId) <i>services/report.service.js</i> Mục đích: Cập nhật dữ liệu hiện có, bảo toàn ràng buộc và ghi nhận trạng thái mới.	Controller hoặc module gọi ReportService.updateReport -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION updateReport(reportId, data, adminId) THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema() status = String(data.status OR "").trim() priority = String(data.priority OR "").trim() adminResponse = NẾU data.adminResponse = undefined THÌ undefined NGƯỢC LẠI String(data.adminResponse OR "").trim().slice(0, 2000) IF status THEN THỰC HIỆN this.assertOneOf(status, REPORT_STATUSES, "report status") END IF IF priority THEN THỰC HIỆN this.assertOneOf(priority, PRIORITIES, "priority") ... END FUNCTION </pre>
AdminService.sendAnnouncement async sendAnnouncement({ audience = 'All users', title, message, deliveryChannel = 'InApp', actionUrl = null }) <i>services/admin.service.js</i> Mục đích: Tạo và gửi thông báo đến nhóm người dùng phù hợp.	Controller hoặc module gọi AdminService.sendAnnouncement -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre> FUNCTION sendAnnouncement({ audience = 'All users', title, message, deliveryChannel = 'InApp', actionUrl = null }) IF NOT title OR NOT message THEN THROW ERROR TẠO Error("Missing title or message") END IF pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Users, Notifications] RETURN { inserted: result.recordset[0].inserted } END FUNCTION </pre>

AdminService.createDailyReminders async createDailyReminders() <i>services/admin.service.js</i> Mục đích: Tạo mới dữ liệu nghiệp vụ sau khi kiểm tra đầu vào và quyền thực hiện.	Controller hoặc module gọi AdminService.createDailyReminders -> chuẩn hóa tham số đầu vào -> lấy kết nối SQL Server/pool -> thực hiện truy vấn, transaction hoặc gọi service liên quan -> trả dữ liệu đã tổng hợp về controller	<pre>FUNCTION createDailyReminders() pool = GỌI BẮT ĐỒNG BỘ poolPromise result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Users, UserWordProgress, Notifications] RETURN { inserted: result.recordset[0].inserted } END FUNCTION</pre>
--	---	---

11. Module System/Common - Hạ tầng backend

Mục đích	Kết nối CSDL, validate request, xử lý lỗi, health-check, integration-test và shutdown an toàn.
Thành phần	config/db.js; validate.js; errorHandler.js; health-check.js; integration-test.js; index.js.
Luồng module	Express app -> middleware JSON/CORS -> routes -> middleware auth/validate -> controller -> service -> DB. Lỗi đi qua next(error) về errorHandler.

Controller

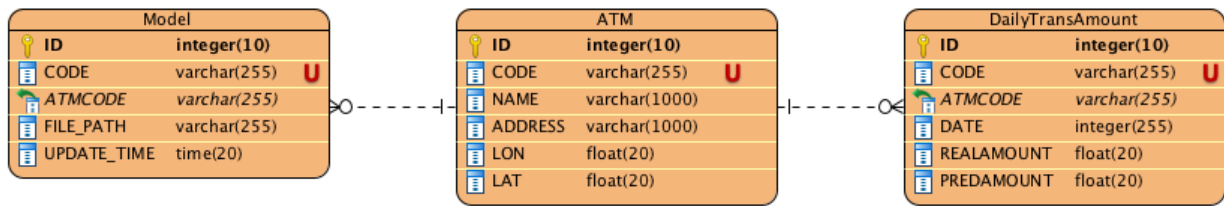
Hàm Controller	Luồng xử lý	Giả mã
errorHandler errorHandler(err, req, res, next) <i>middlewares/errorHandler.js</i> Mục đích: Chuẩn hóa lỗi backend thành HTTP status và JSON response.	Express chạy middleware errorHandler -> đọc header/body/quyền user -> hợp lệ thì next() -> không hợp lệ trả lỗi 400/401/403	<pre>FUNCTION errorHandler(err, req, res, next) THỰC HIỆN console.error(err.stack) TRẢ VỀ HTTP 500 CÙNG DỮ LIỆU { message: err.message OR "Lỗi server nội bộ" } END FUNCTION</pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
validate validate(schema) <i>middlewares/validate.js</i> Mục đích: Validate body theo schema trước khi controller xử lý.	Express chạy middleware validate -> đọc header/body/quyền user -> hợp lệ thì next() -> không hợp lệ trả lỗi 400/401/403	<pre>FUNCTION validate(schema) RETURN MIDDLEWARE FUNCTION(req, res, next) TRY parsed = schema.parse({ body: req.body, query: req.query, params: req.params }) IF Object.prototype.hasOwnProperty.call(parsed, "body") THEN req.body = parsed.body END IF IF Object.prototype.hasOwnProperty.call(parsed, "query") THEN req.query = parsed.query END IF ... END FUNCTION</pre>
runHealthCheck async runHealthCheck() <i>config/health-check.js</i> Mục đích: Kiểm tra biến môi trường, kết nối CSDL và tình trạng schema.	Chạy tác vụ runHealthCheck -> đọc cấu hình môi trường -> kiểm tra kết nối/trạng thái hệ thống -> log kết quả và kết thúc an toàn	<pre>FUNCTION runHealthCheck() THỰC HIỆN console.log("--- VOCABOOST SYSTEM DIAGNOSTIC ---") requiredEnv = ["DB_SERVER", "DB_NAME", "JWT_SECRET"] missing = requiredEnv.filter(HÀM CALLBACK(k)) IF missing.length > 0 THEN THỰC HIỆN console.error("✗ MISSING ENV VARS: ", missing.join(", ")) ELSE THỰC HIỆN console.log("☑ Environment Variables: OK") END IF TRY ... END FUNCTION</pre>
runIntegrationTest async runIntegrationTest()	Chạy tác vụ runIntegrationTest -> đọc cấu hình môi trường -> kiểm tra kết nối/trạng thái hệ thống	<pre>FUNCTION runIntegrationTest() THỰC HIỆN console.log("🚀 STARTING INTEGRATION TEST...")</pre>

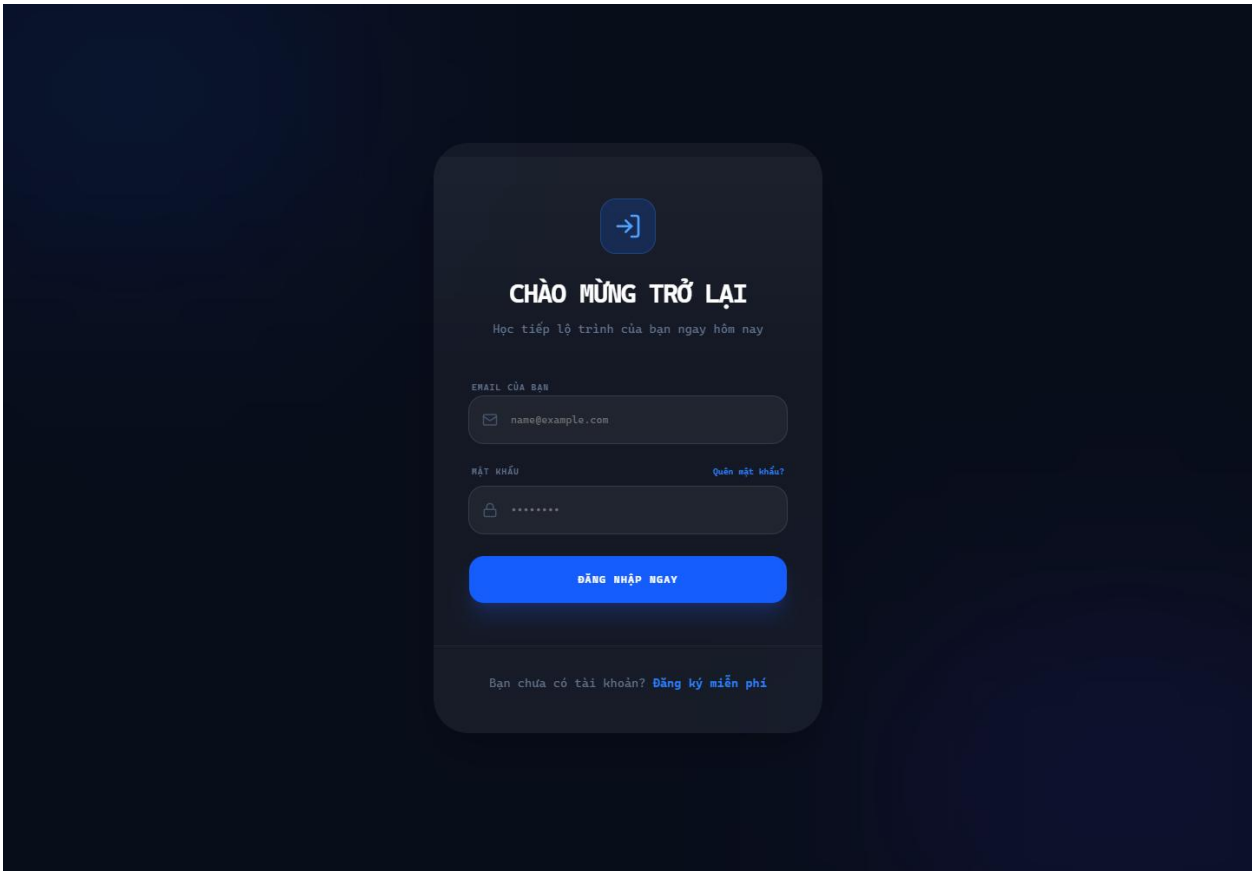
<i>config/integration-test.js</i> Mục đích: Chạy kiểm tra tích hợp nhanh cho cấu hình và truy vấn CSDL.	-> log kết quả và kết thúc an toàn	<pre>TRY THỰC HIỆN console.log("1.Testing Registration...") email = `test_\${ Date.now()   }@example.com` THỰC HIỆN GỌI BẮT ĐỒNG BỘ axios.post(`\${ API_URL }/auth/register`, { fullName: "Test User", email: email, password: "password123" }) THỰC HIỆN console.log(" ☒ Registration successful") THỰC HIỆN console.log("2.Testing Login...") loginRes = GỌI BẮT ĐỒNG BỘ axios.post(`\${ API_URL }/auth/login`, { email: email, password: "password123" }) token = loginRes.data.token ... END FUNCTION</pre>
gracefulShutdown async gracefulShutdown(signal) <i>index.js</i> Mục đích: Đóng server an toàn khi nhận tín hiệu dừng ứng dụng.	Chạy tác vụ gracefulShutdown -> đọc cấu hình môi trường -> kiểm tra kết nối/trạng thái hệ thống -> log kết quả và kết thúc an toàn	<pre>FUNCTION gracefulShutdown(signal) THỰC HIỆN console.log(`Received \${   signal }.Shutting down gracefully...`) TRY pool = GỌI BẮT ĐỒNG BỘ poolPromise THỰC HIỆN GỌI BẮT ĐỒNG BỘ pool.close() THỰC HIỆN console.log("Database pool closed.") THỰC HIỆN process.exit(0) CATCH err THỰC HIỆN console.error("Error during shutdown: ", err) THỰC HIỆN process.exit(1) END TRY END FUNCTION</pre>

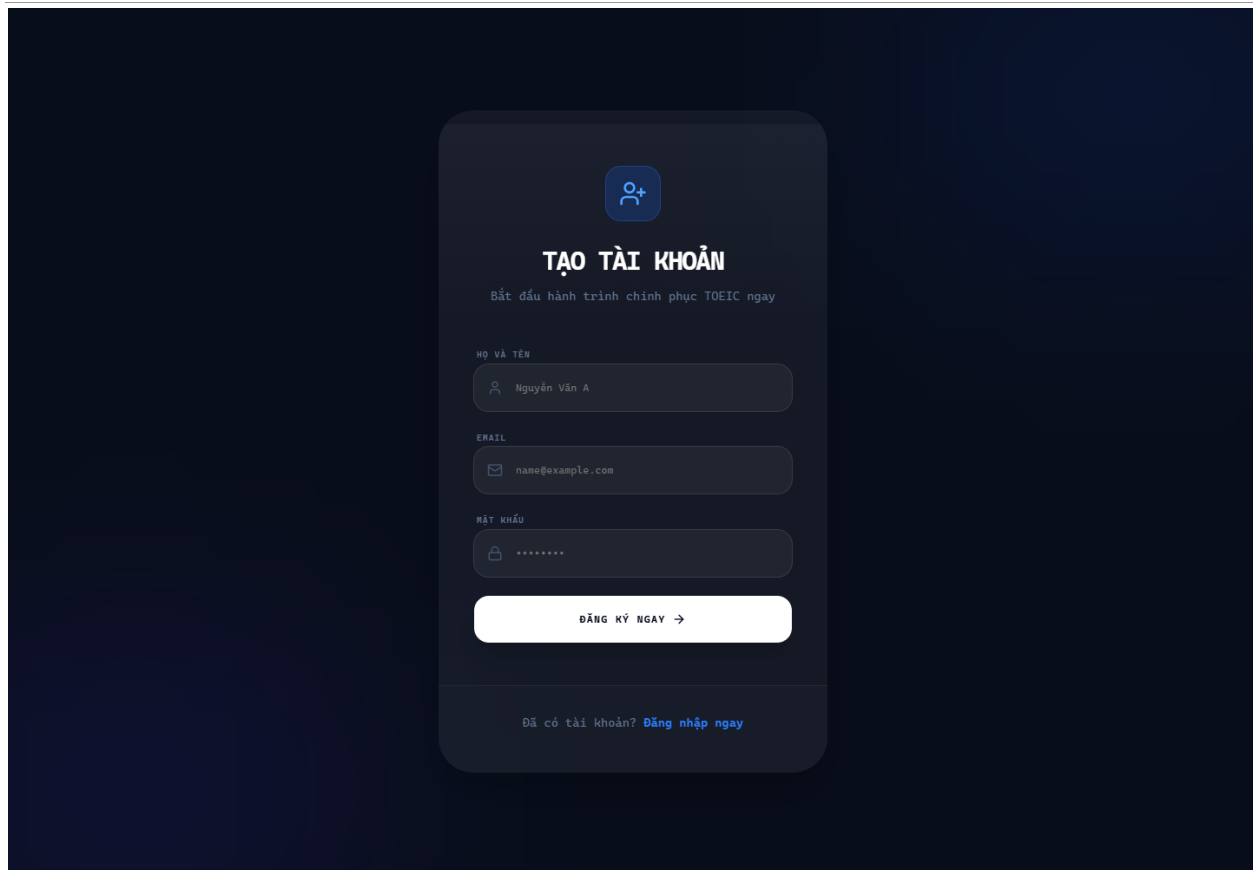
5. THIẾT KẾ DỮ LIỆU



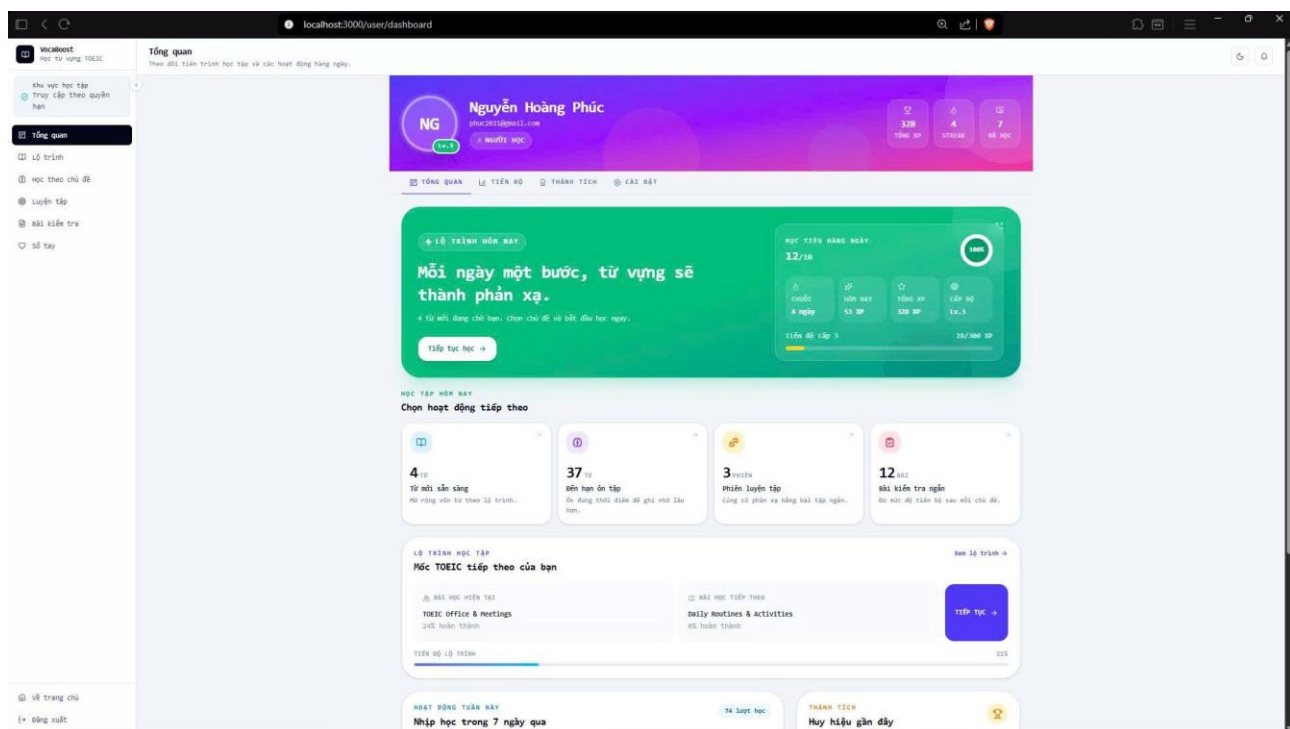
6. THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG

6.1 Giao diện đăng nhập, người dùng

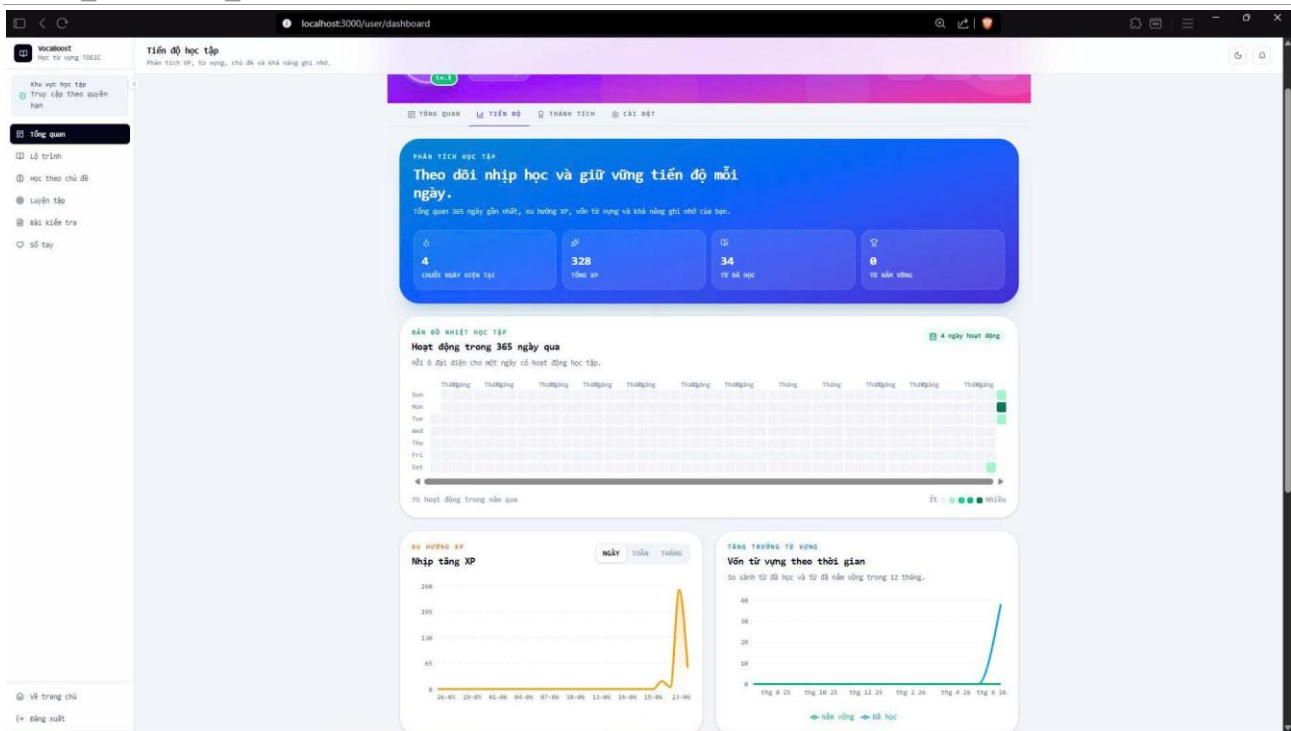




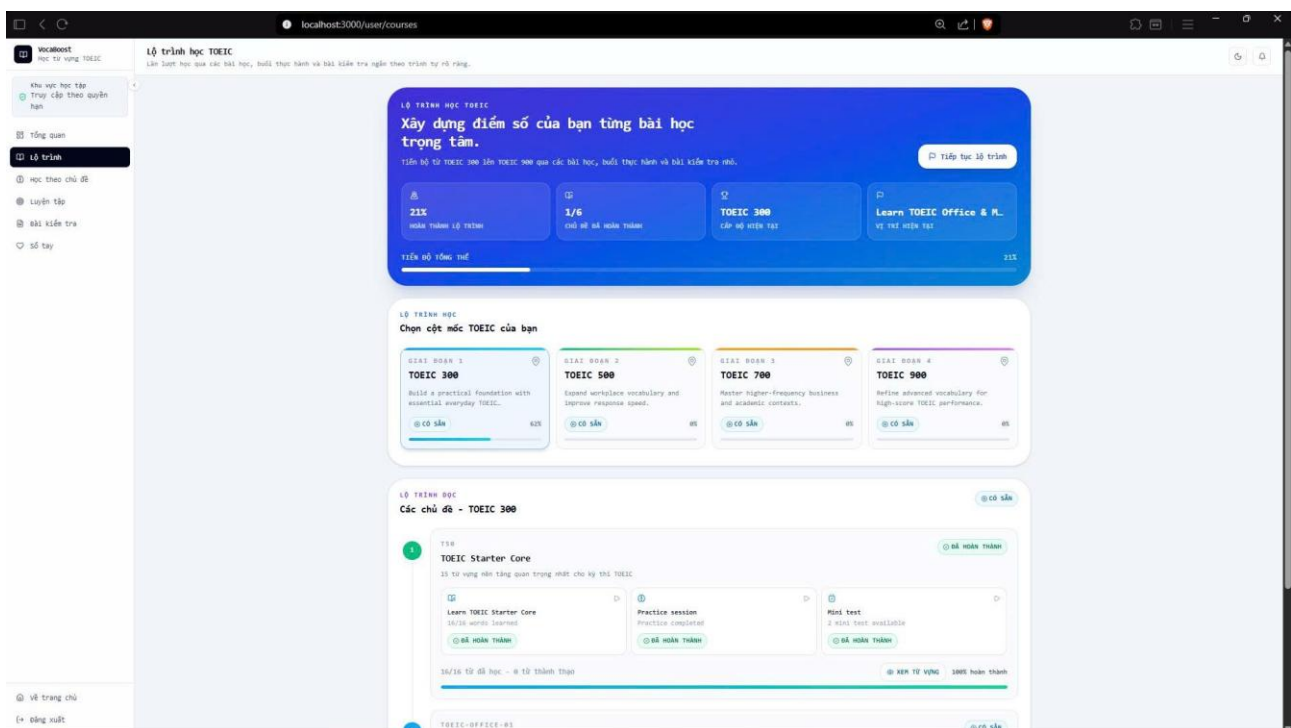
Hình 6.1.1: Giao diện đăng nhập, đăng kí hệ thống



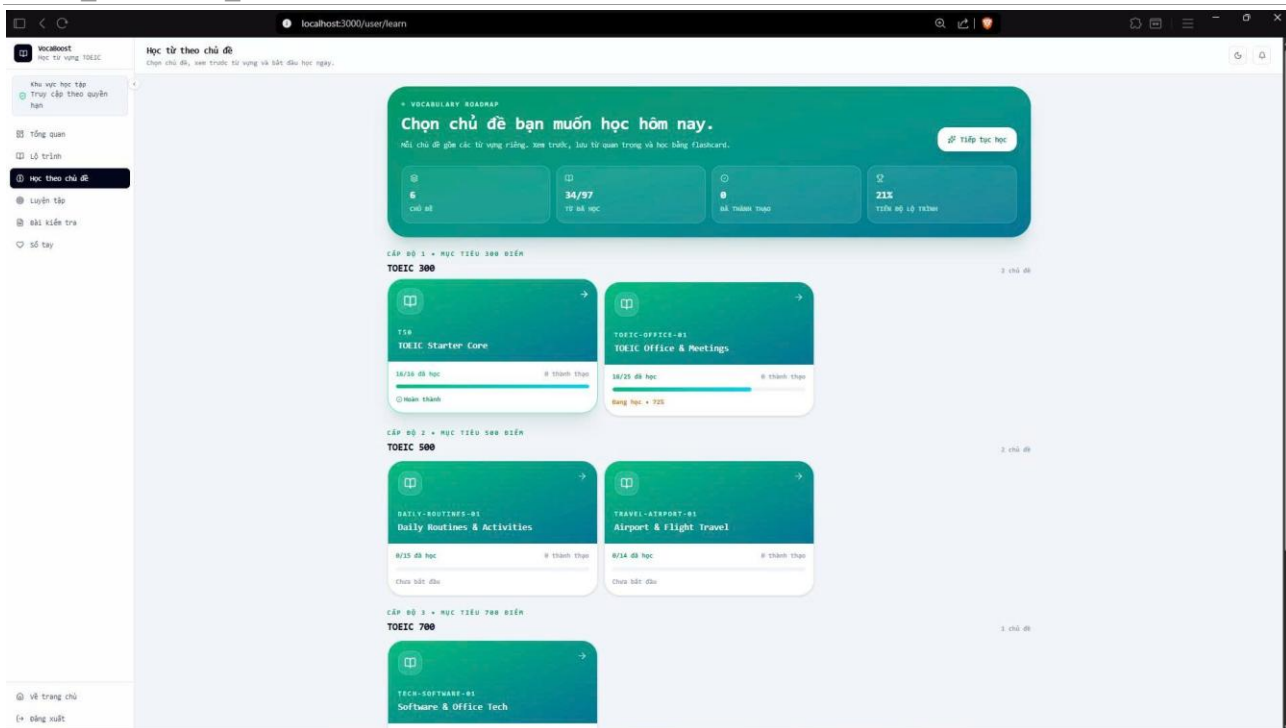
<2024_VocaBoost_VKU>: Thiết kế chi tiết



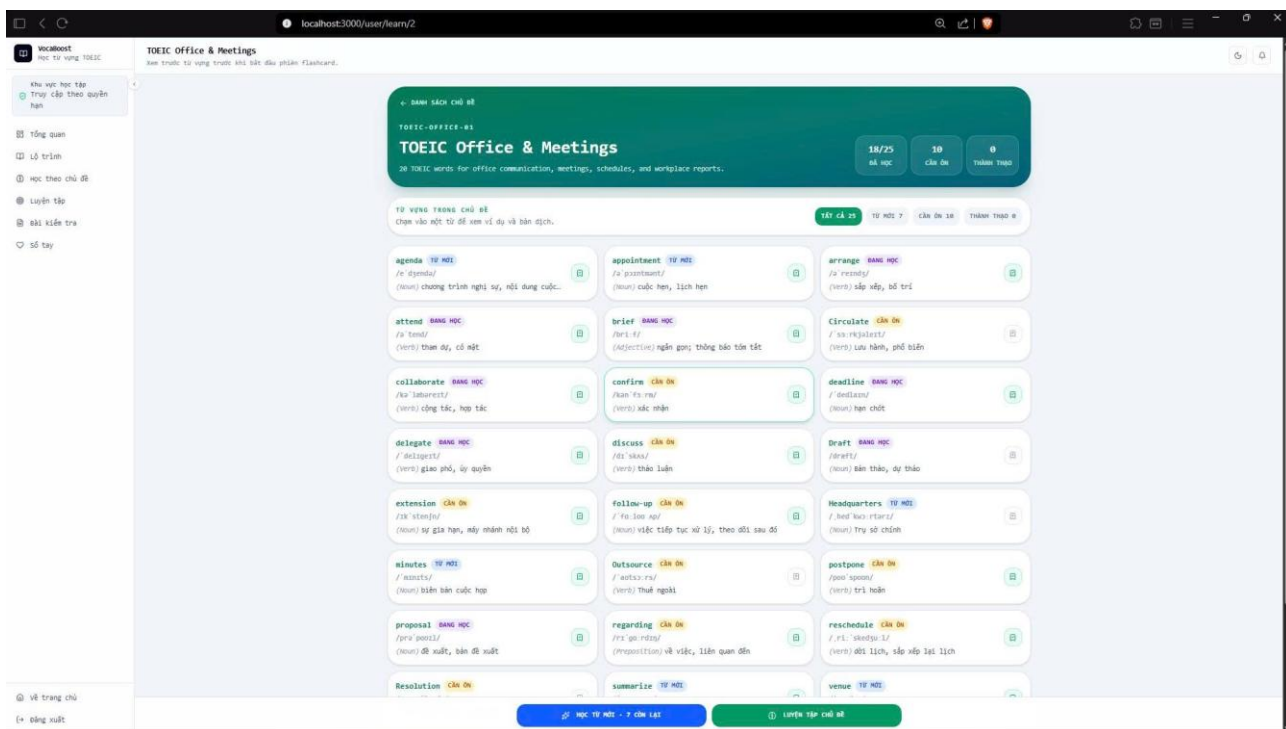
Hình 6.1.2: Giao diện bảng điều khiển học tập với mục tiêu ngày, XP, chuỗi học, hoạt động và lộ trình tiếp theo.



Hình 6.1.3: Giao diện danh sách lộ trình/chủ đề giúp học viên chọn nội dung theo thứ tự học phù hợp.

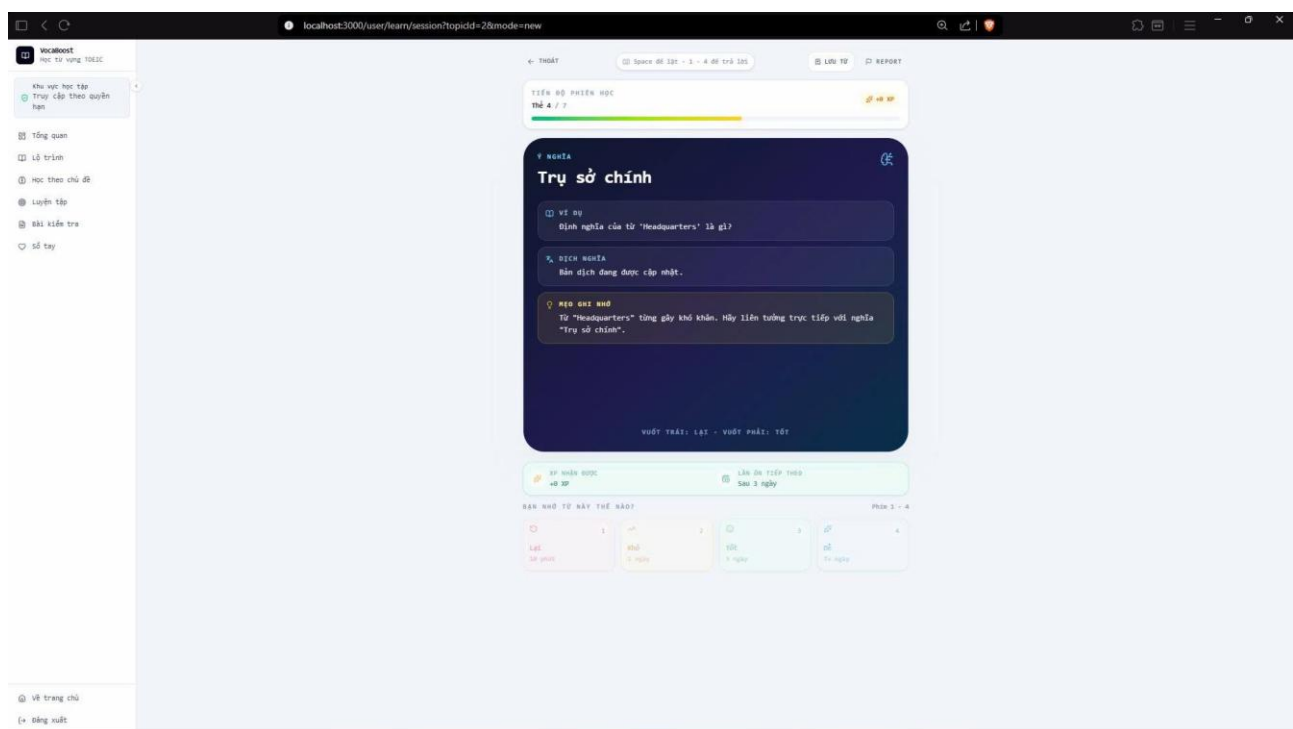
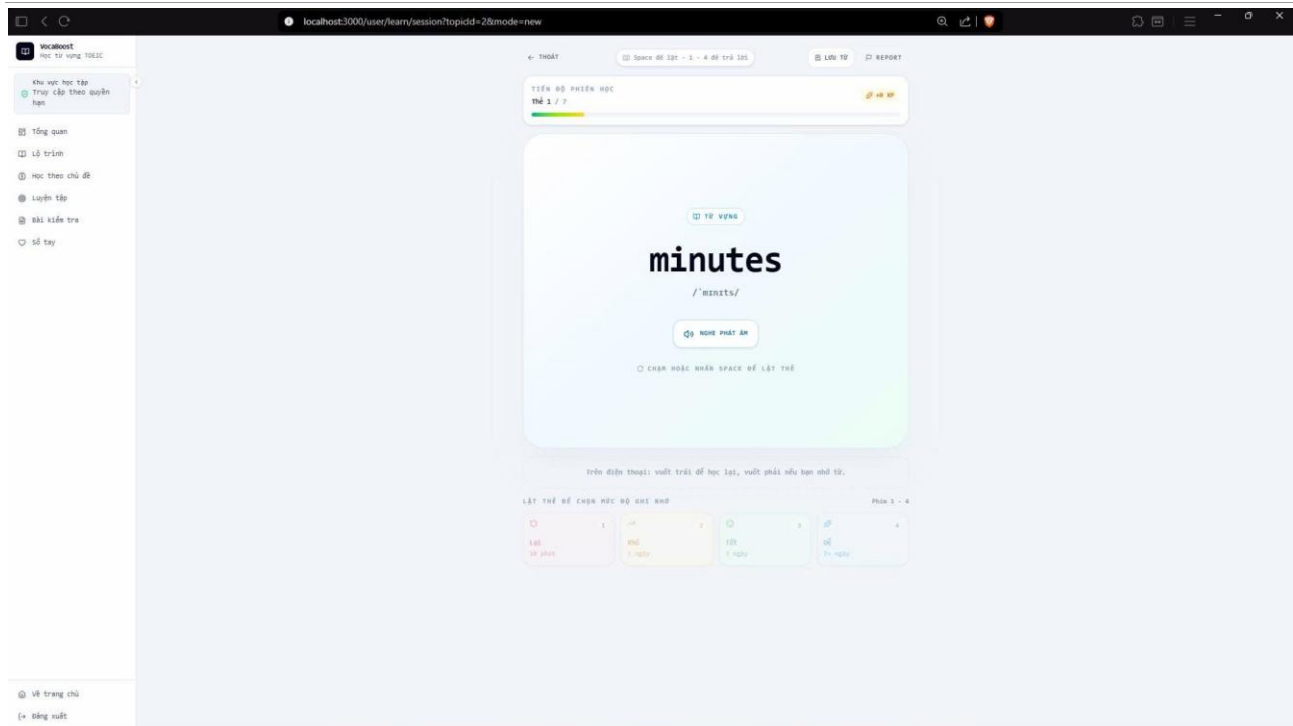


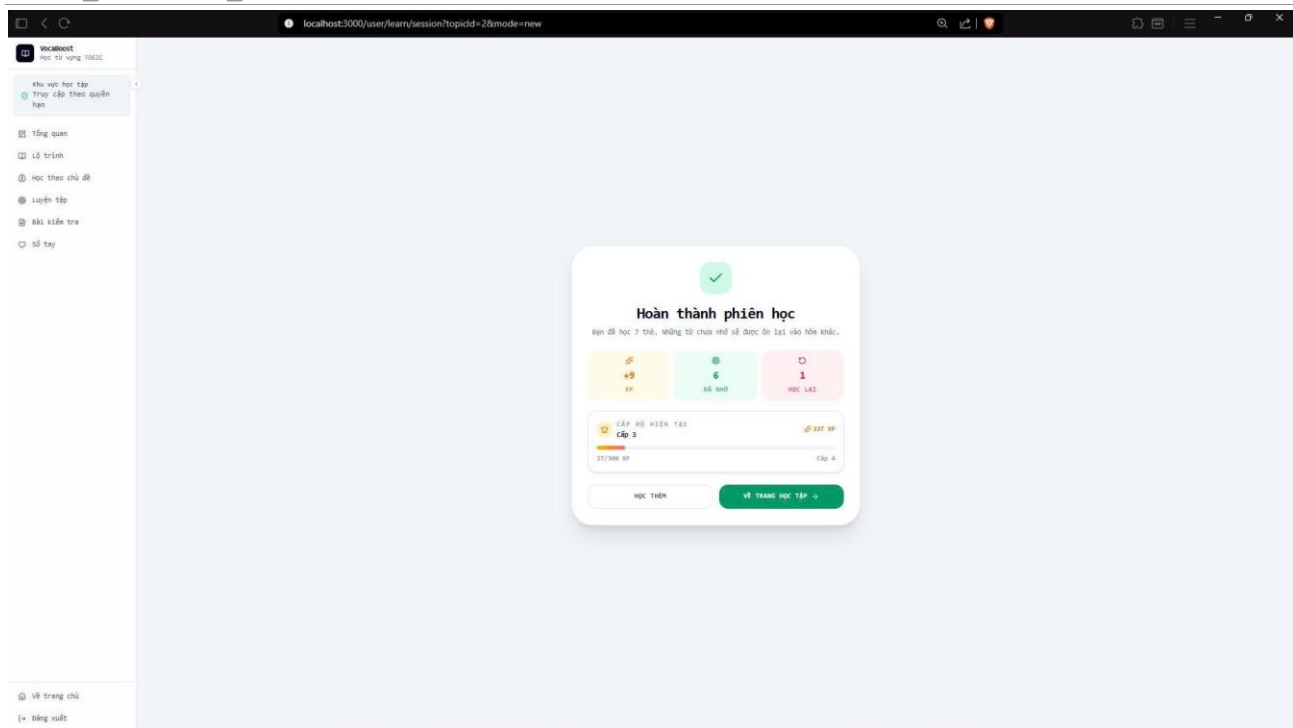
Hình 6.1.4: Khu vực chọn chủ đề và bắt đầu phiên học từ vựng bằng flashcard



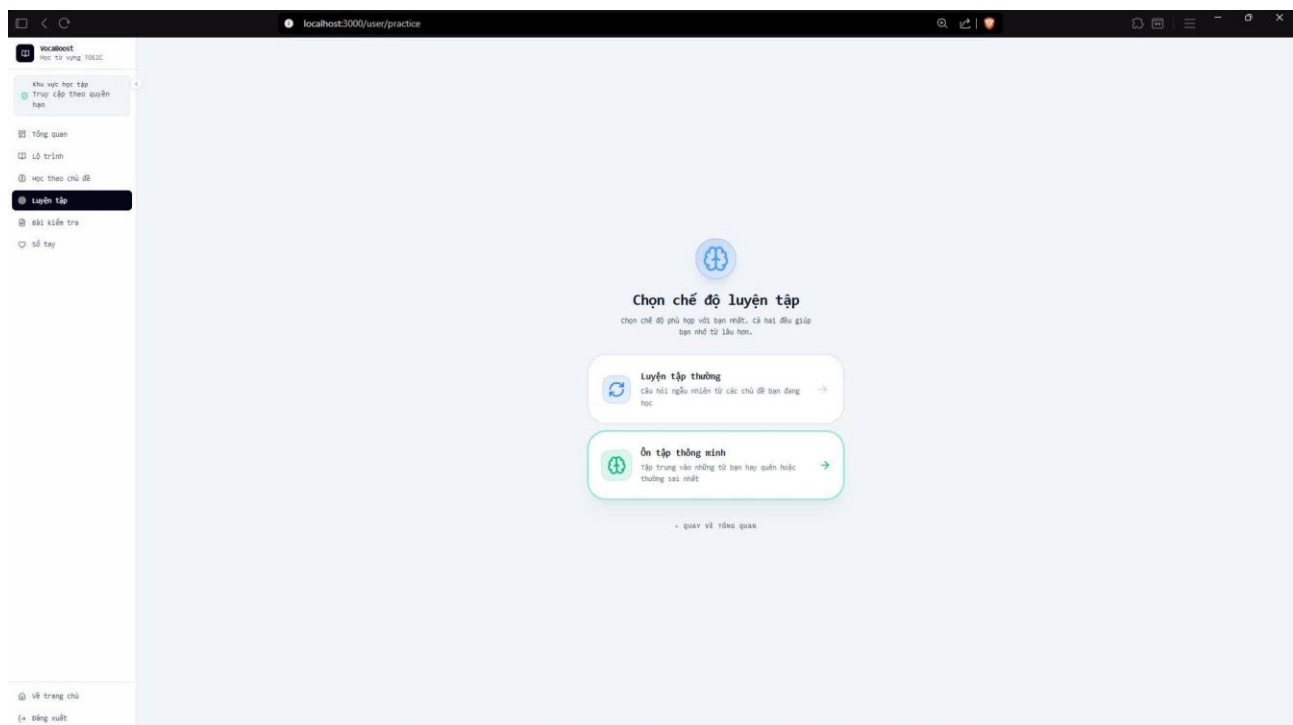
Hình 6.1.5: Giao diện xem các từ vựng trong chủ đề

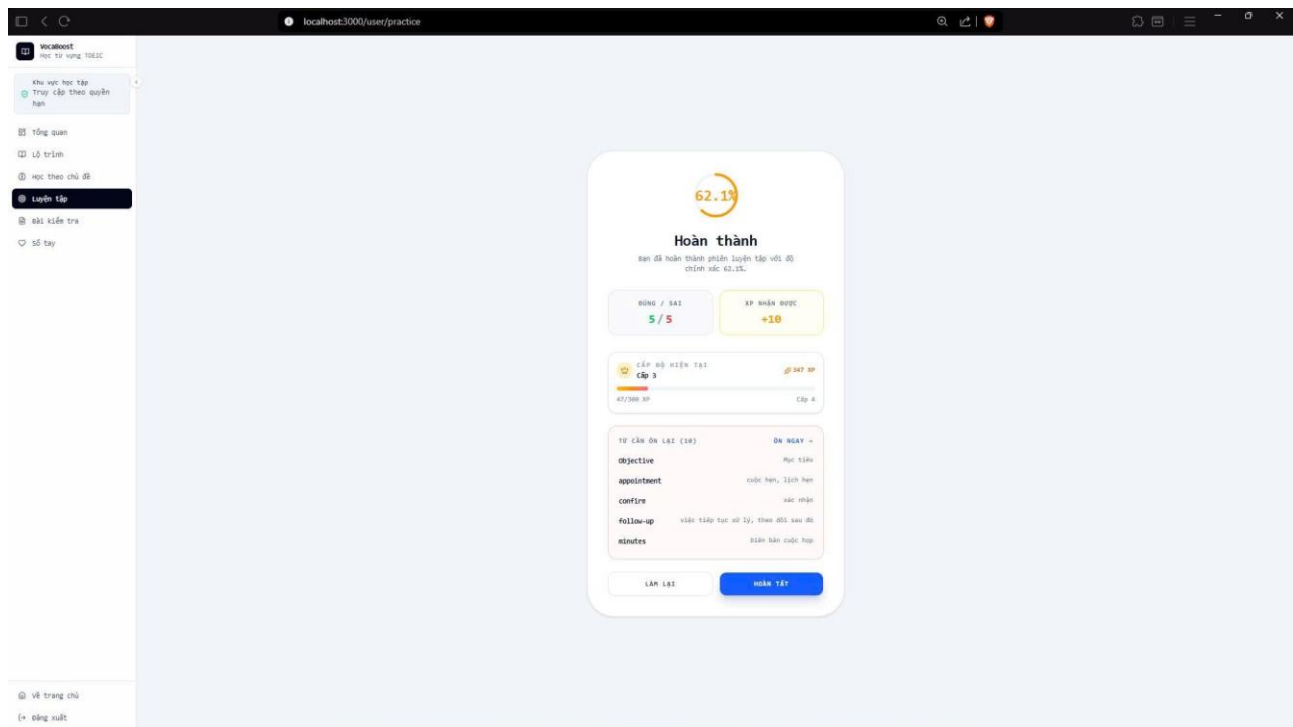
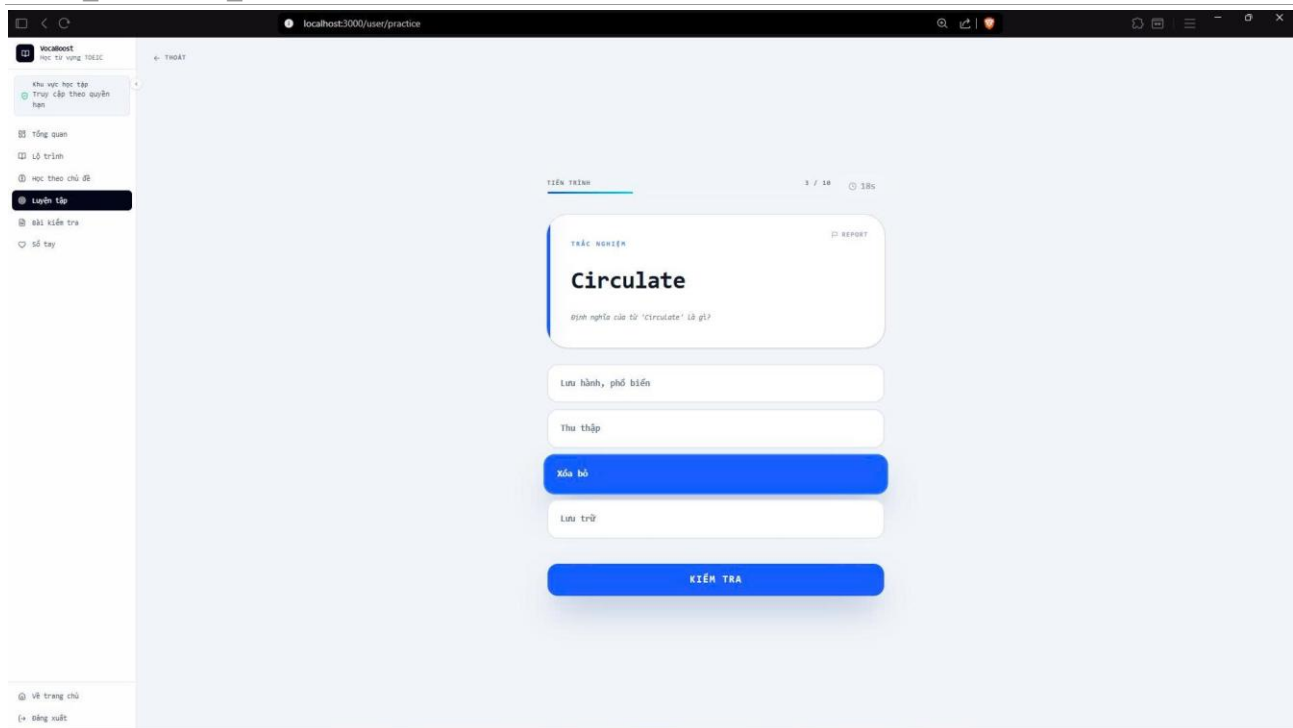
<2024_VocaBoost_VKU>: Thiết kế chi tiết





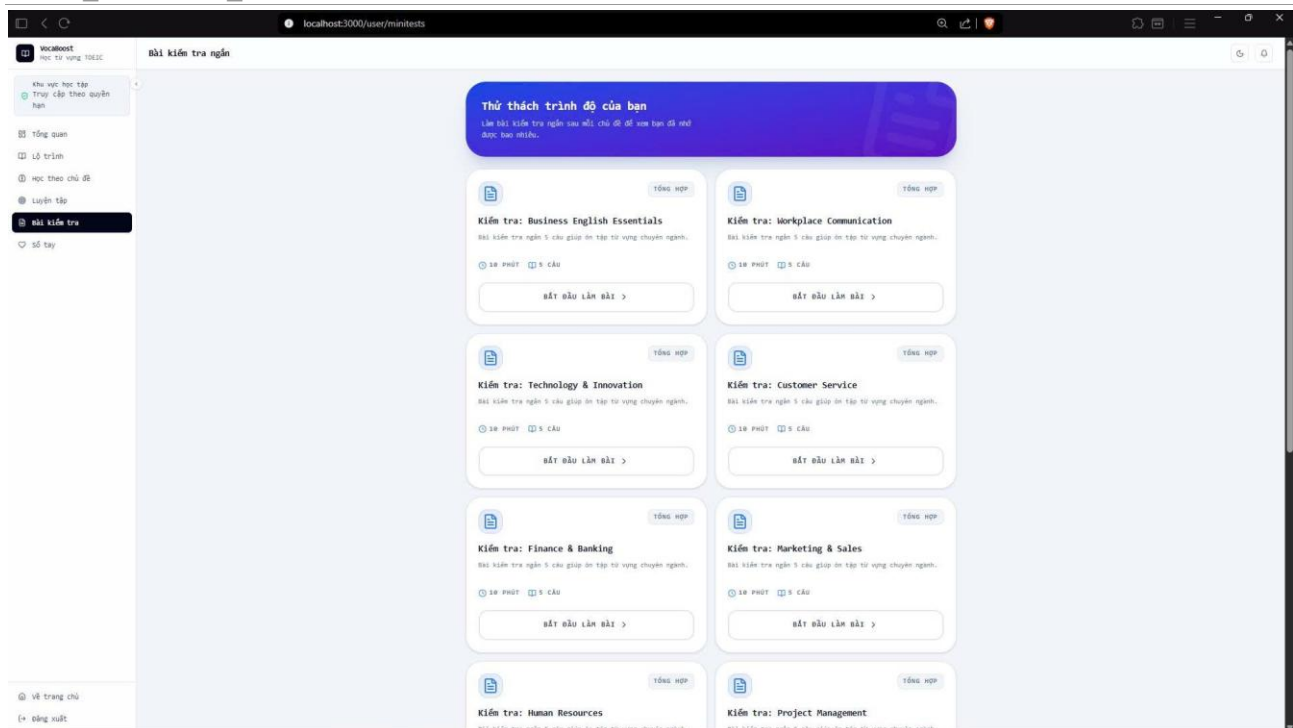
Hình 6.1.6: Giao diện học bằng flashcard



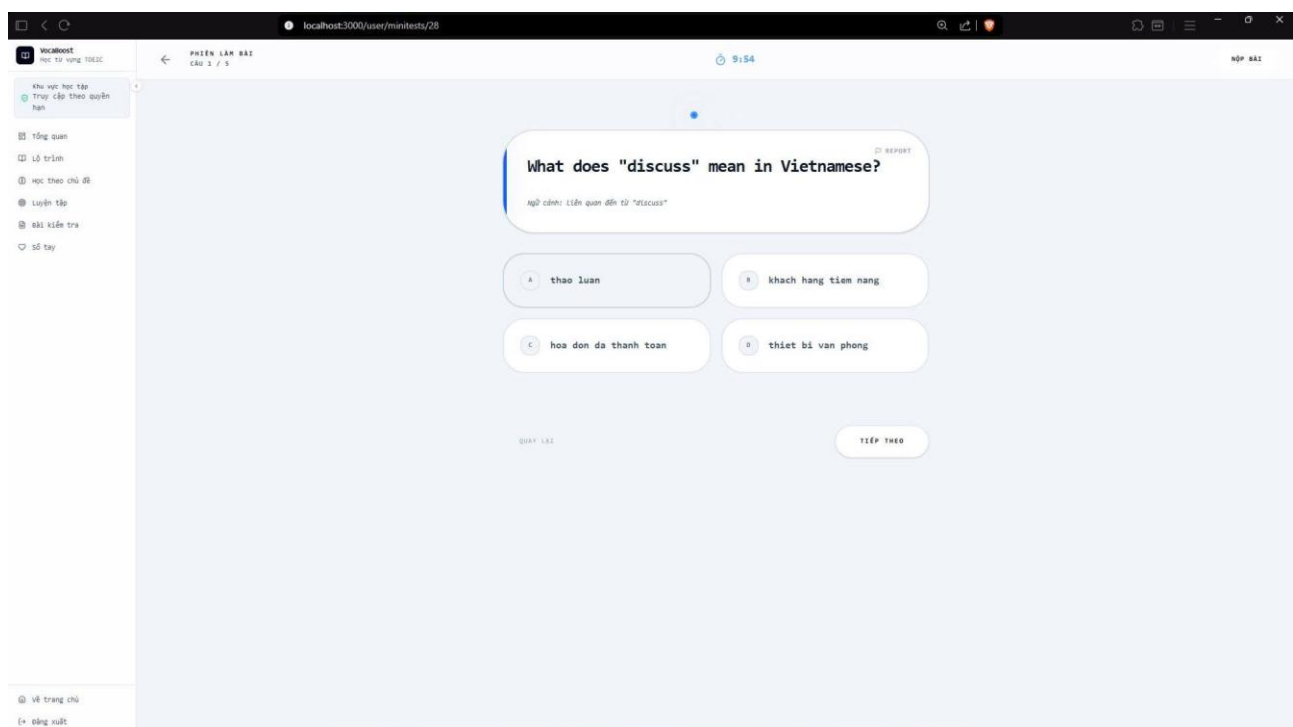


Hình 6.1.7: Màn hình chọn chế độ luyện tập và ôn tập thông minh theo dữ liệu SRS.

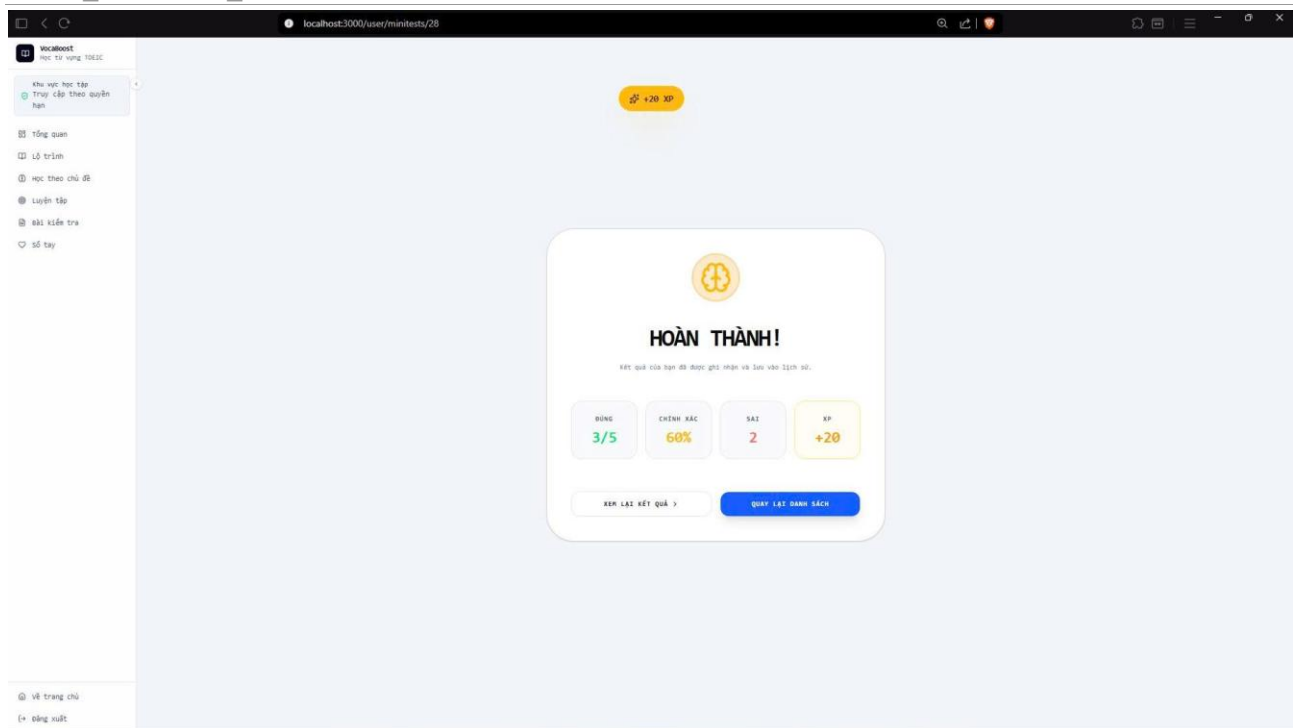
<2024_VocaBoost_VKU>: Thiết kế chi tiết



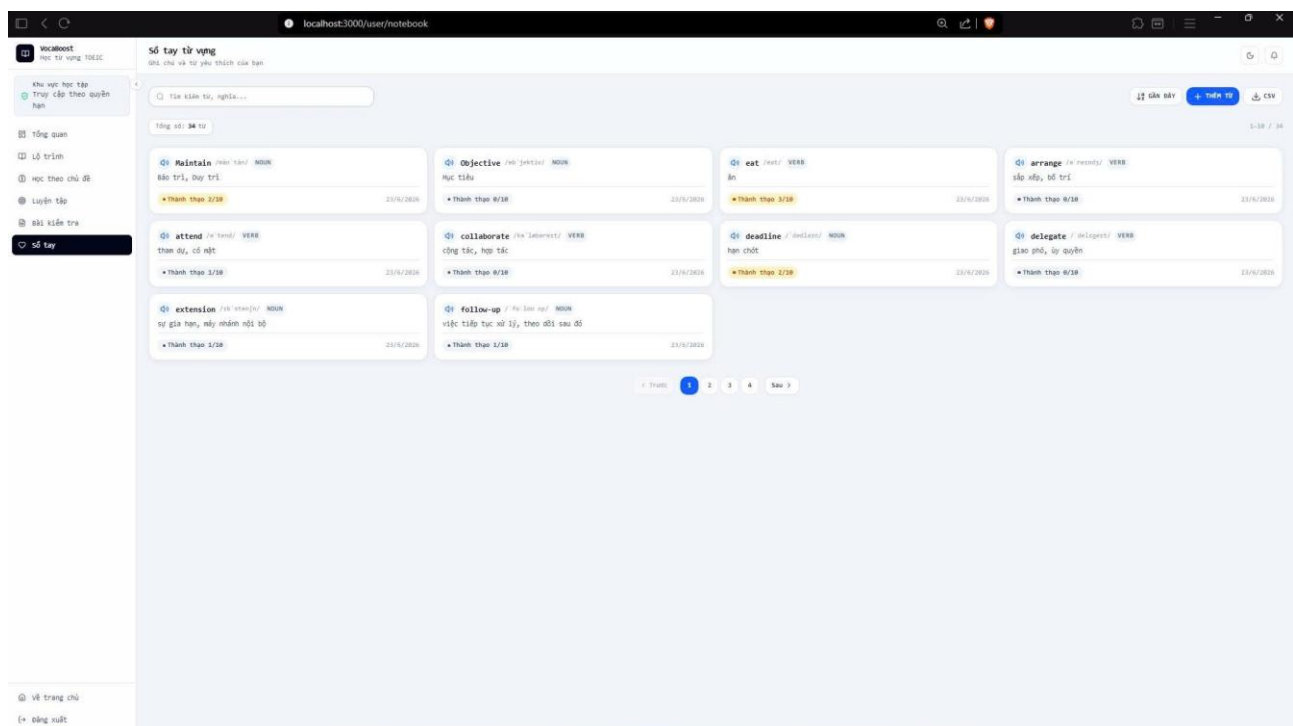
Hình 6.1.8: Danh sách mini test để đánh giá mức độ ghi nhớ và theo dõi kết quả.



<2024_VocaBoost_VKU>: Thiết kế chi tiết

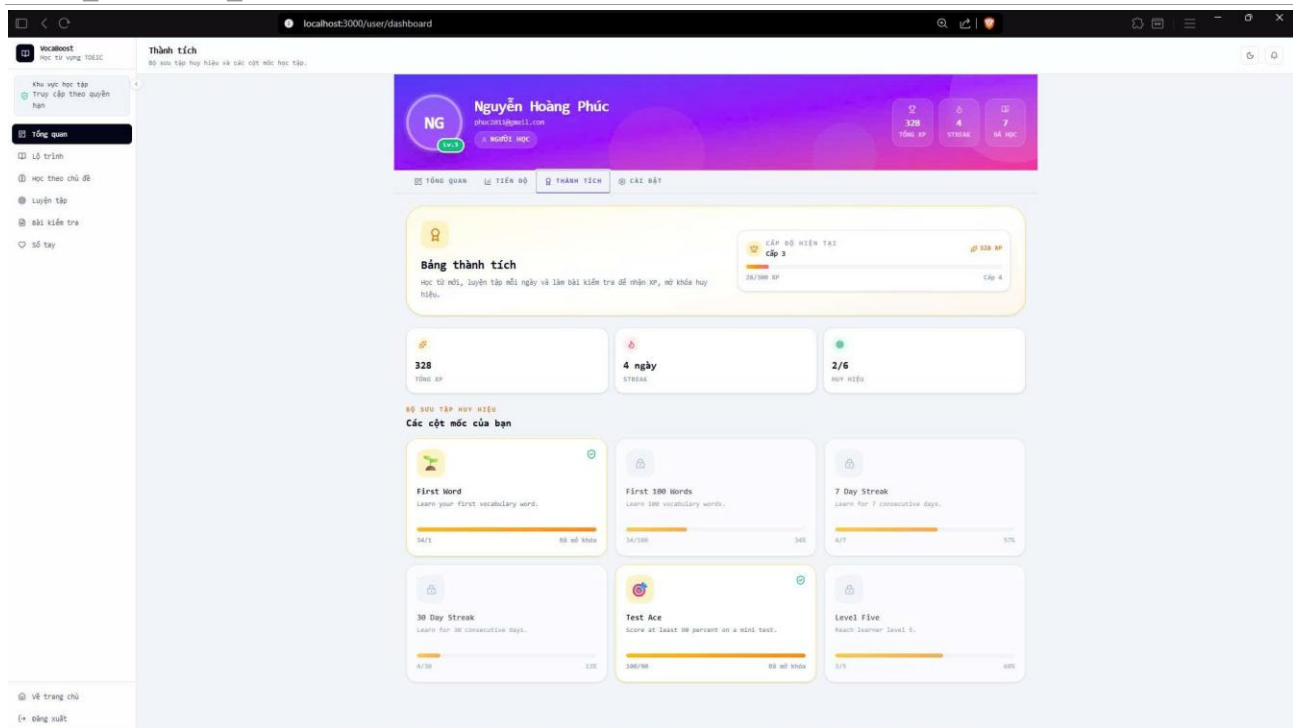


Hình 6.1.9: Giao diện luyện tập minitest



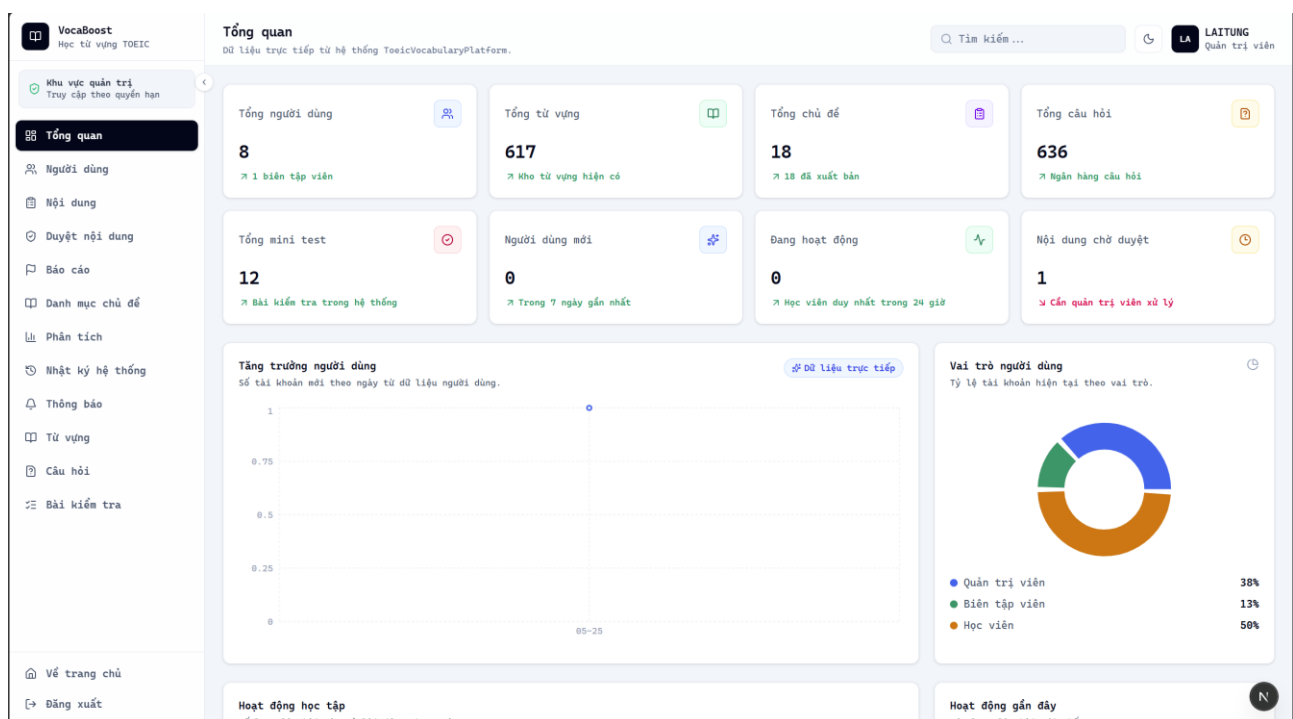
Hình 6.1.10: Nơi lưu, tìm kiếm và quản lý những từ vựng học viên muốn ghi nhớ riêng.

<2024_VocaBoost_VKU>: Thiết kế chi tiết



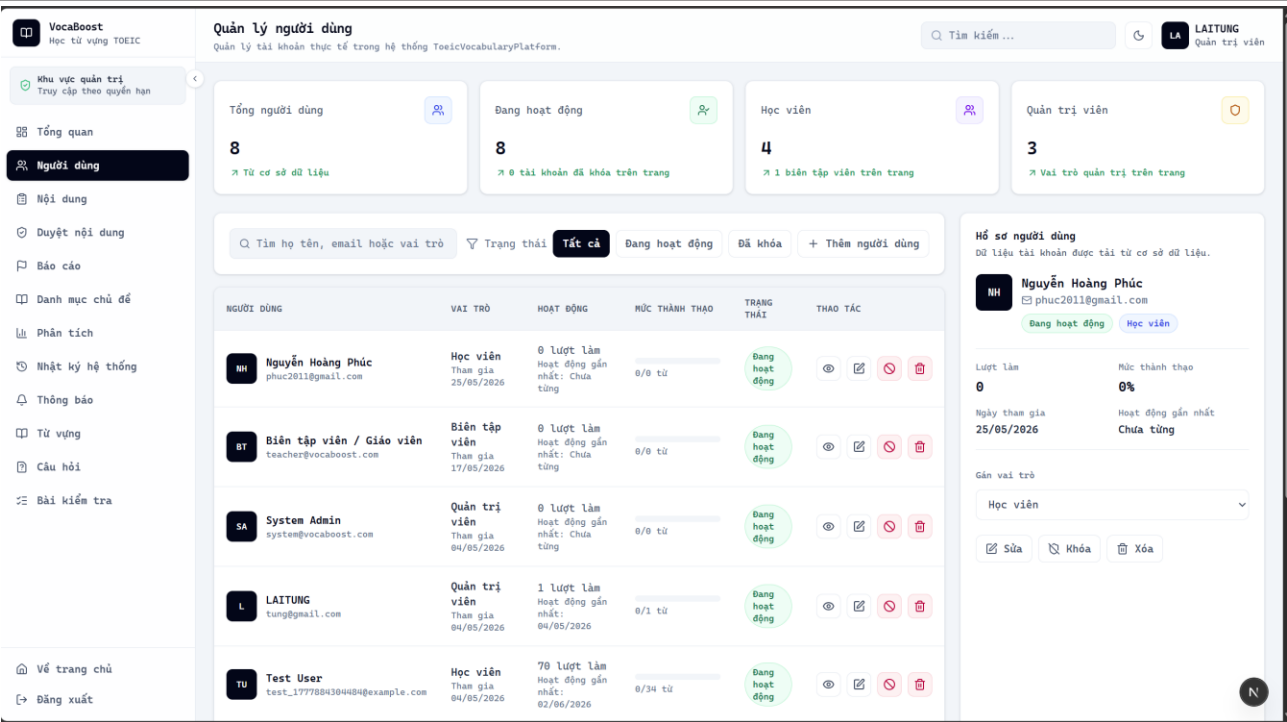
Hình 6.1.11: Các huy hiệu, cột mốc và tiến độ gamification của học viên.

6.2 Giao diện dành cho quản trị (admin)

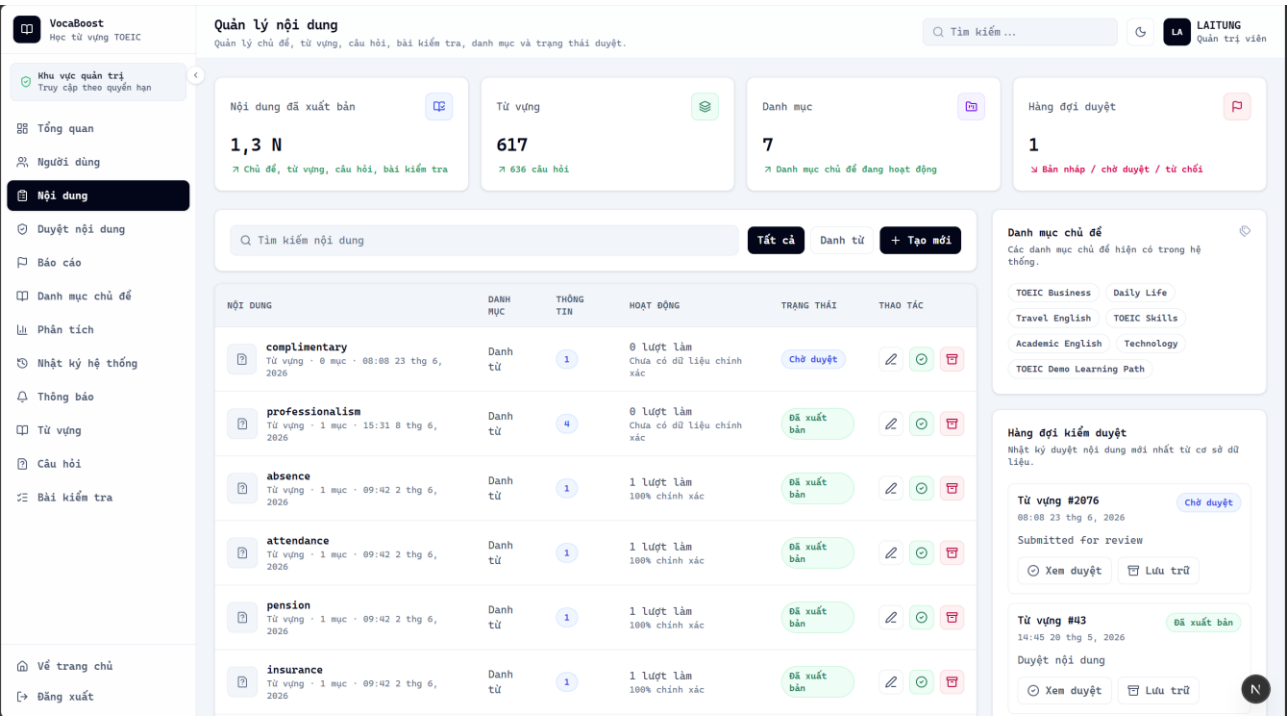


Hình 6.2.1: Dashboard hệ thống với KPI người dùng, nội dung, hoạt động học và trạng thái dịch vụ.

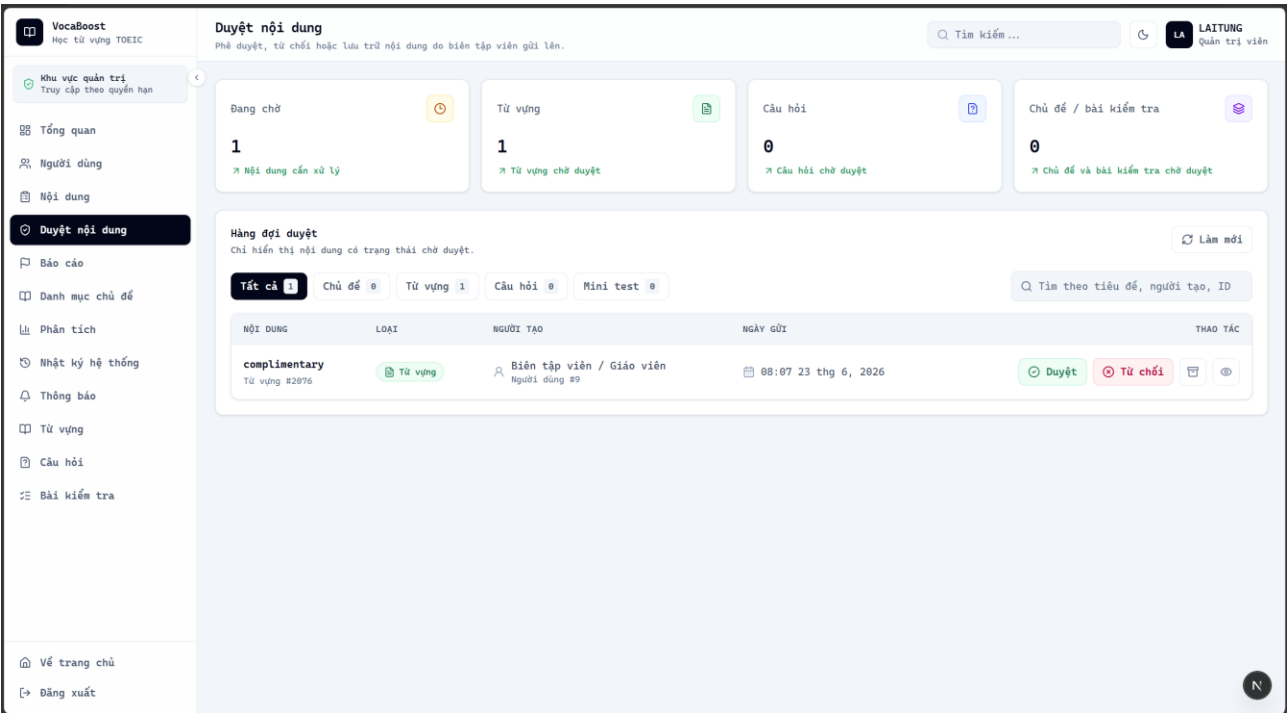
<2024_VocaBoost_VKU>: Thiết kế chi tiết



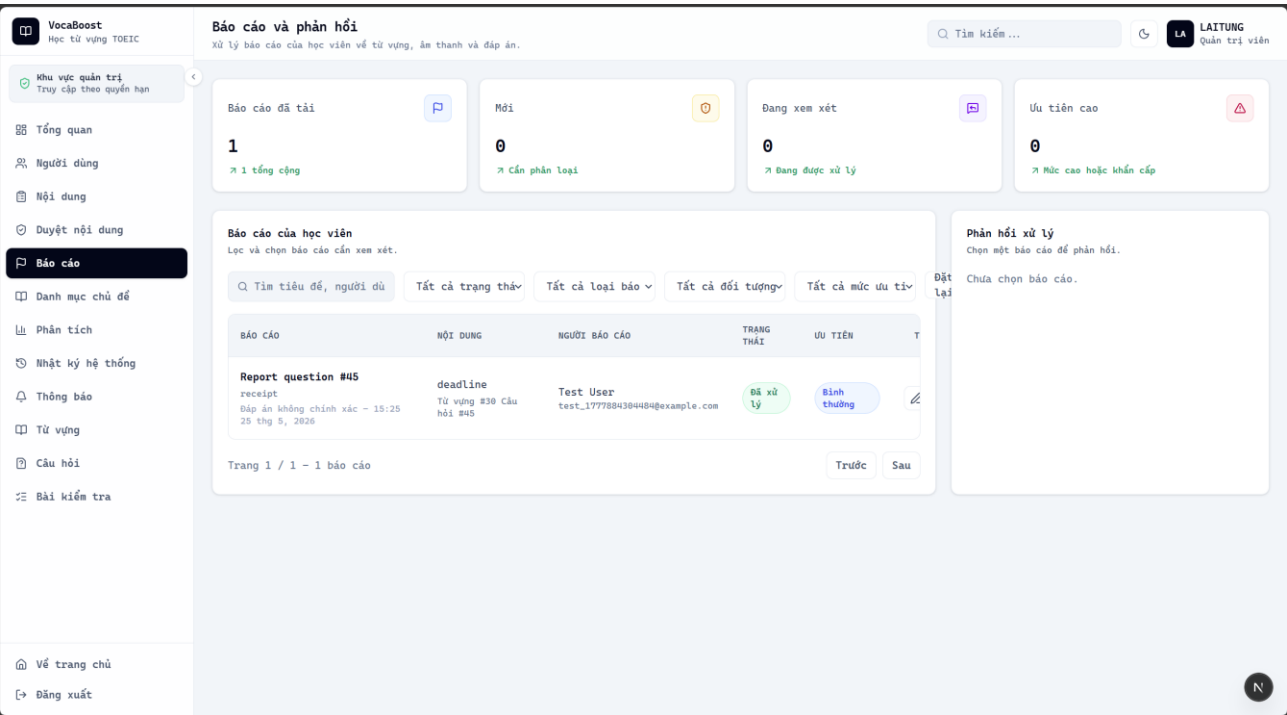
Hình 6.2.2: Danh sách tài khoản, vai trò, trạng thái hoạt động và các thao tác quản trị.



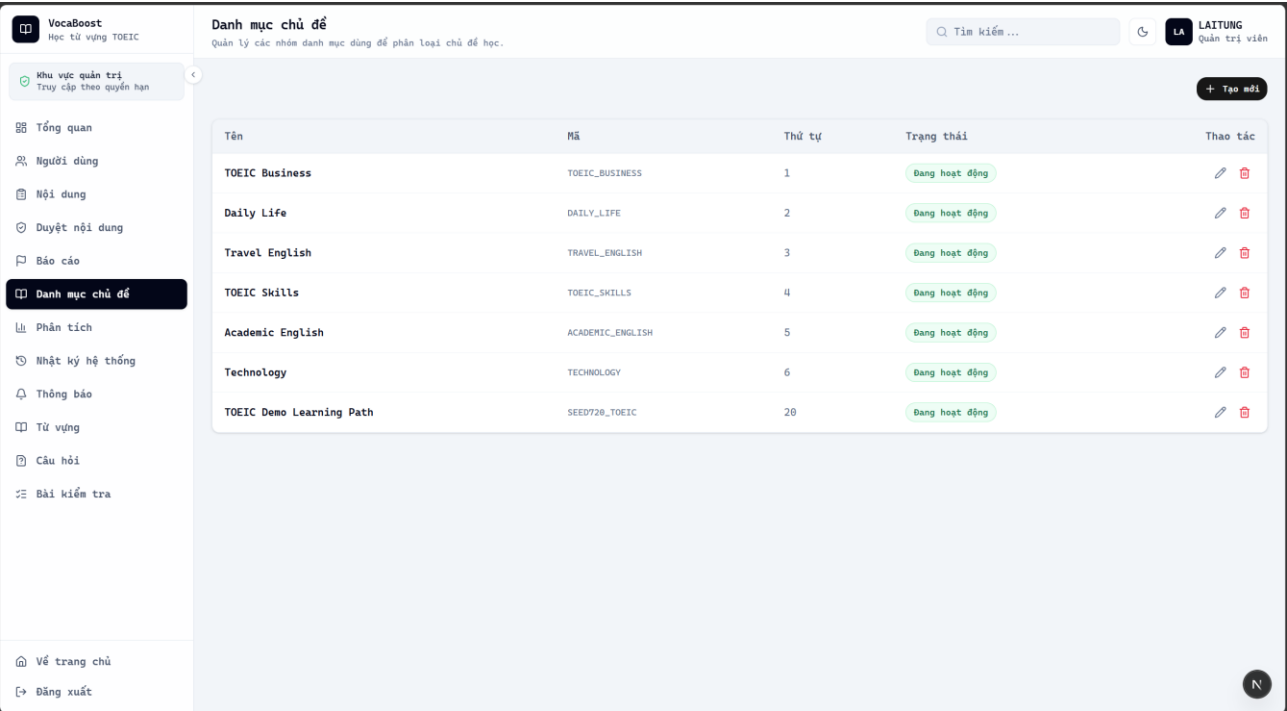
Hình 6.2.3: Giao diện tổng hợp chủ đề và nội dung học để quản trị trạng thái xuất bản.



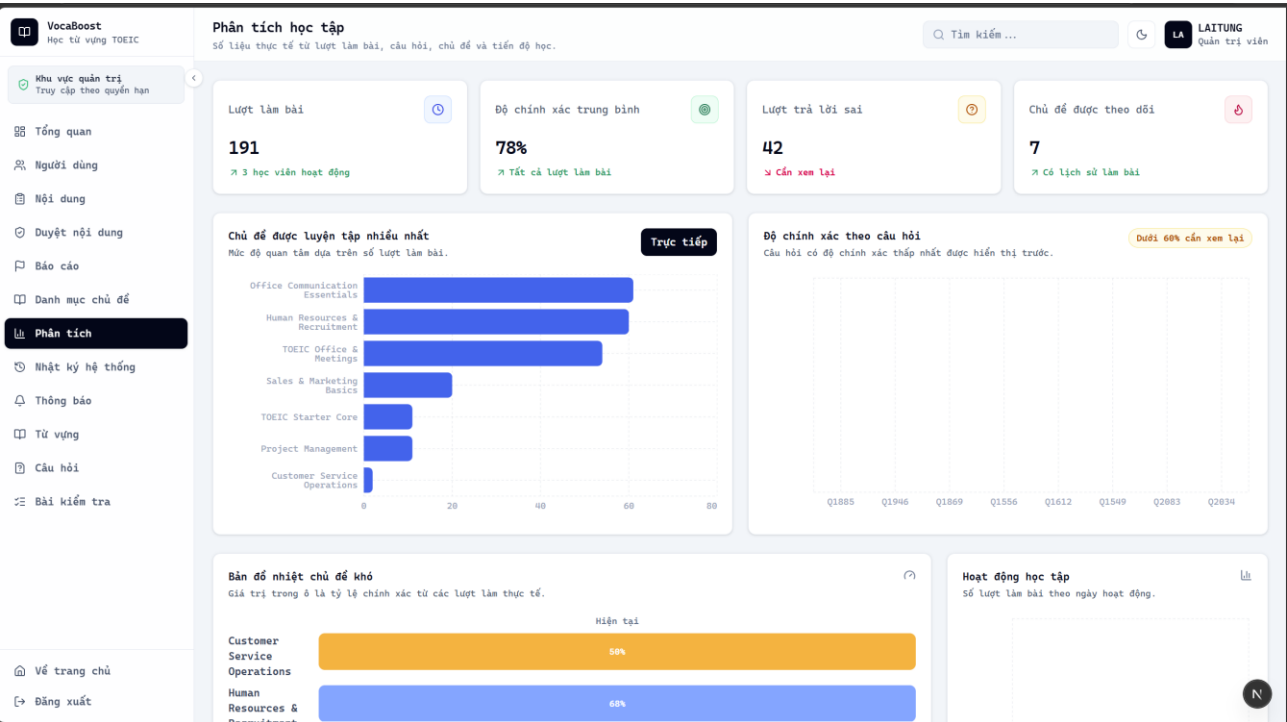
Hình 6.2.4: Giao diện hàng đợi kiểm duyệt nội dung do biên tập viên gửi lên.



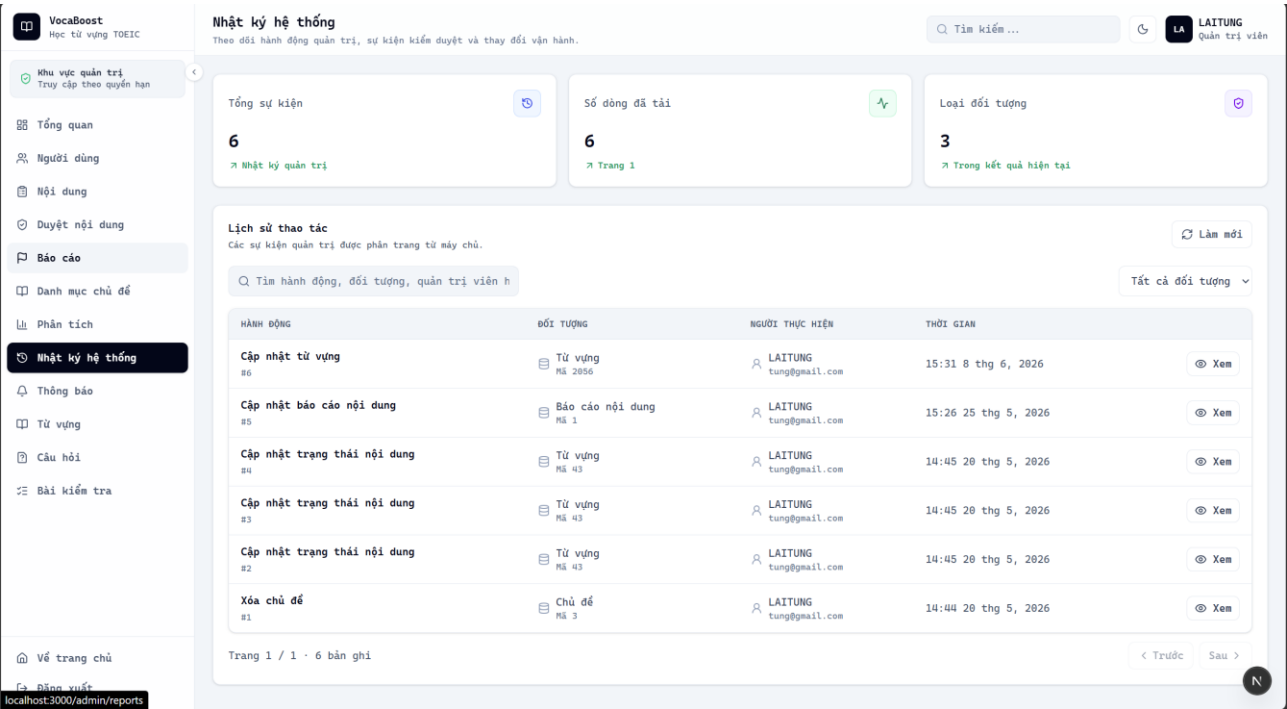
Hình 6.2.5: Giao diện danh sách phản hồi/báo lỗi của học viên và quy trình xử lý.



Hình 6.2.6: Giao diện Quản lý các nhóm danh mục dùng để phân loại chủ đề học.

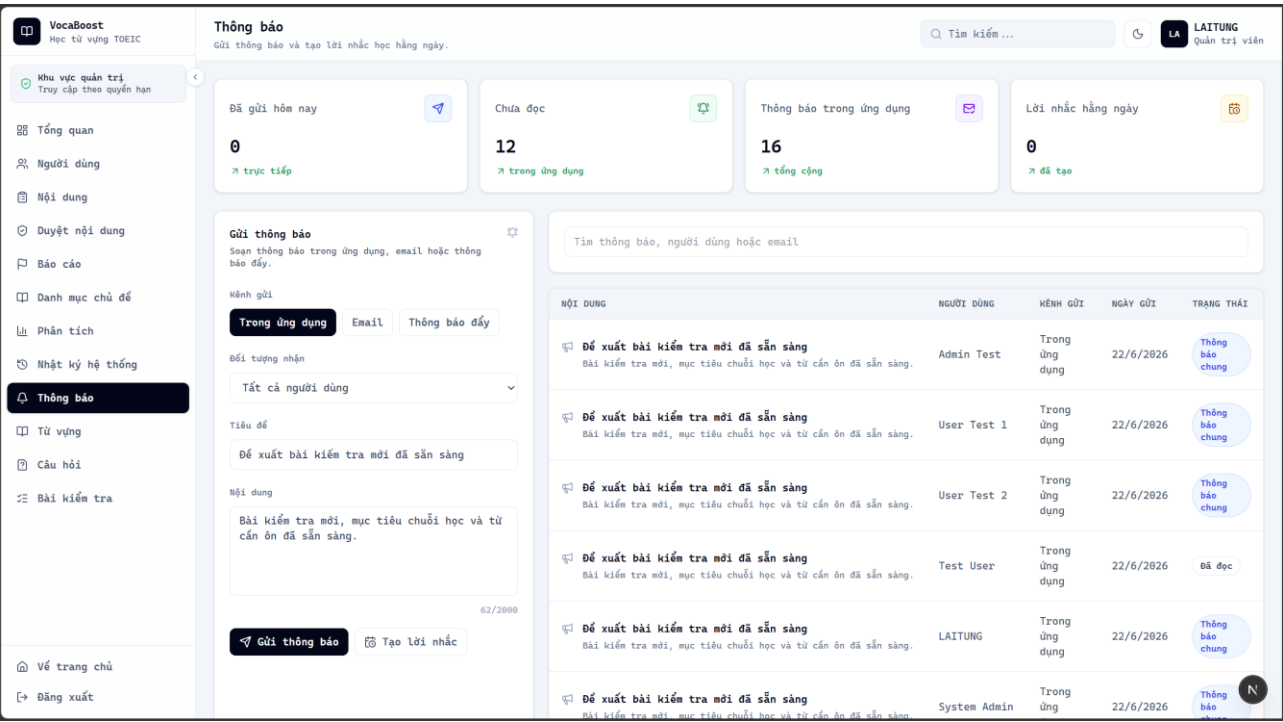


Hình 6.2.7: Biểu đồ phân tích người dùng, học tập và dữ liệu nội dung toàn hệ thống.

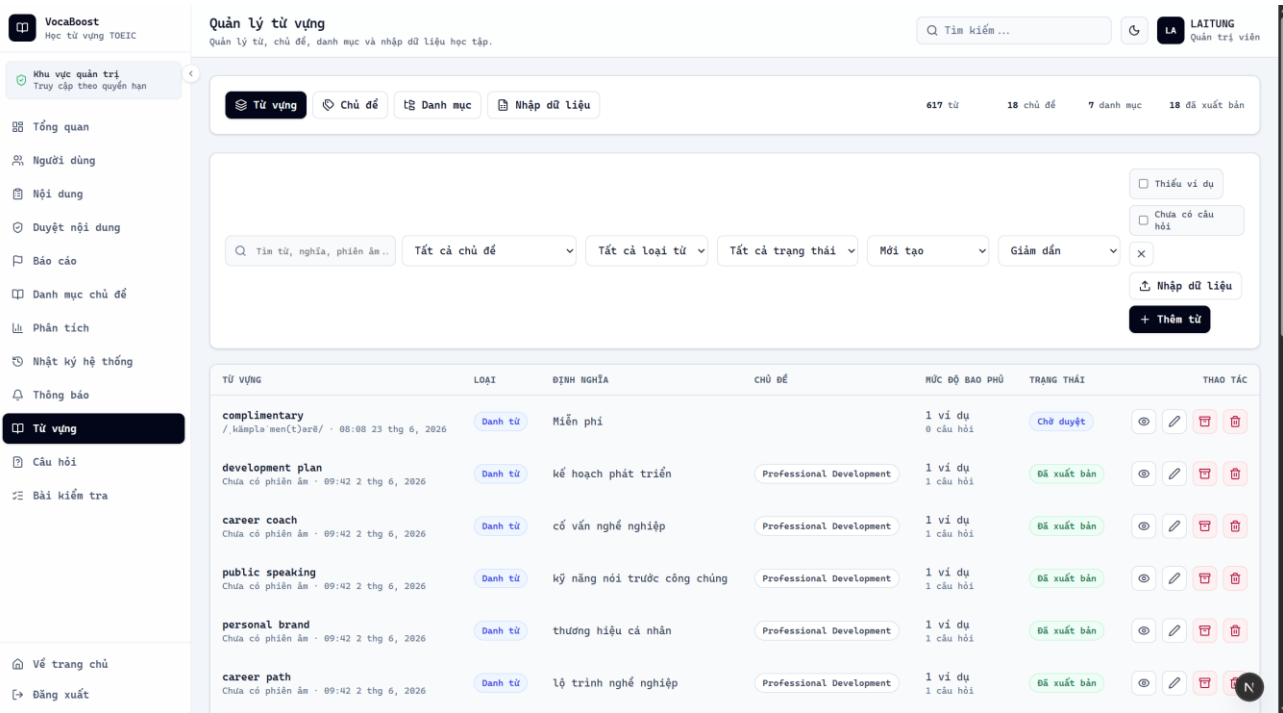


Hình 6.2.8: Giao diện theo dõi các hành động quản trị và thay đổi dữ liệu quan trọng.

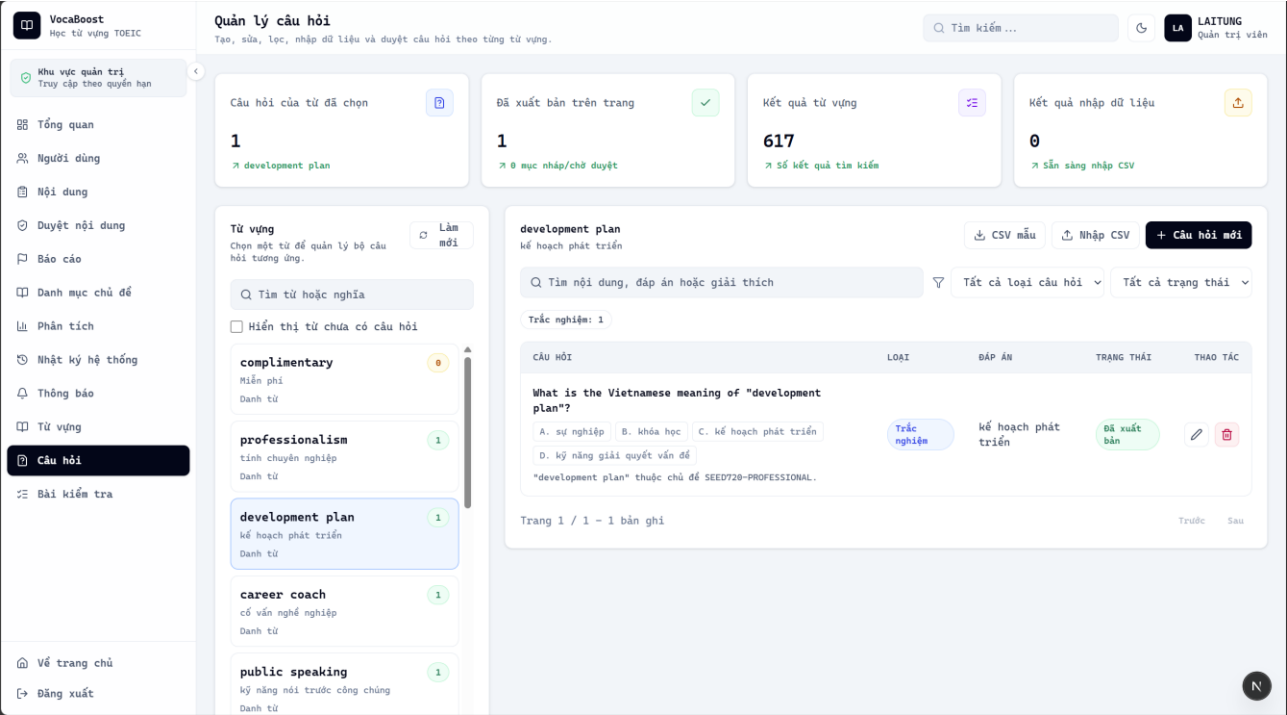
<2024_VocaBoost_VKU>: Thiết kế chi tiết



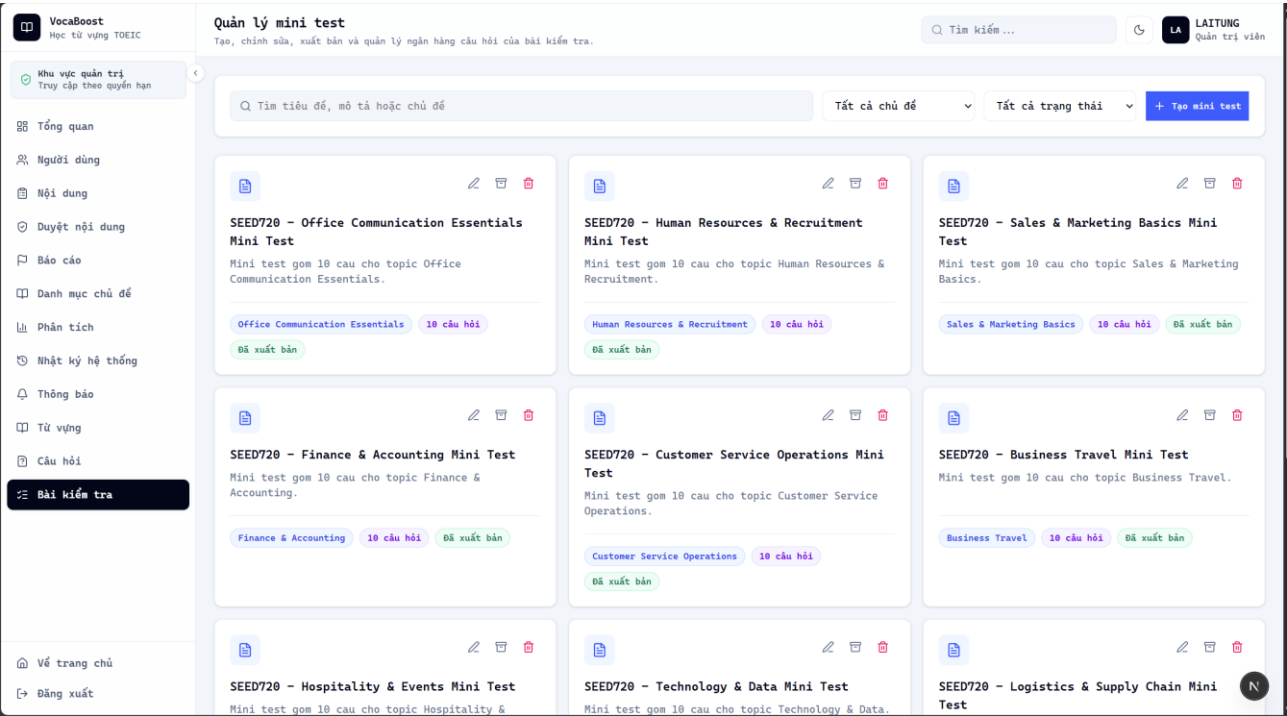
Hình 6.2.9: Giao diện gửi thông báo chung và tạo lời nhắc học hằng ngày.



Hình 6.2.10: Giao diện bảng dữ liệu từ vựng với tìm kiếm, lọc, nhập hàng loạt và CRUD.

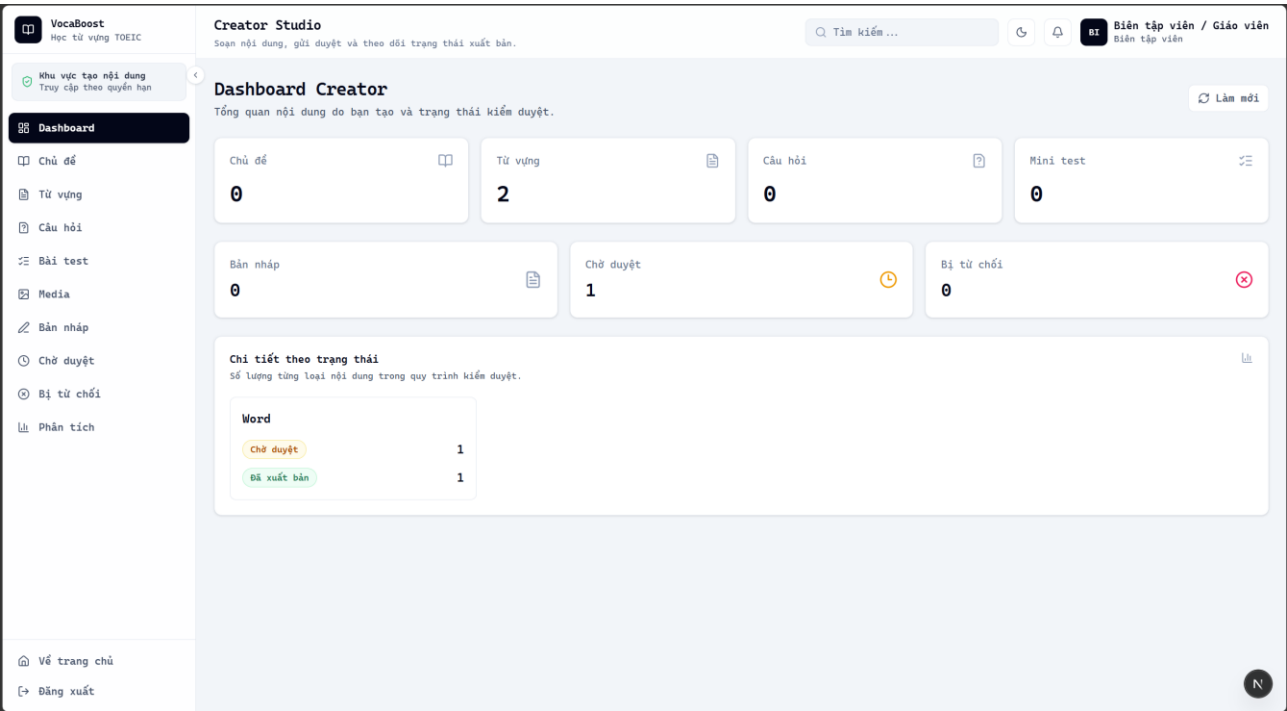


Hình 6.2.11: Giao diện quản lý câu hỏi luyện tập gắn với từ vựng và chủ đề.

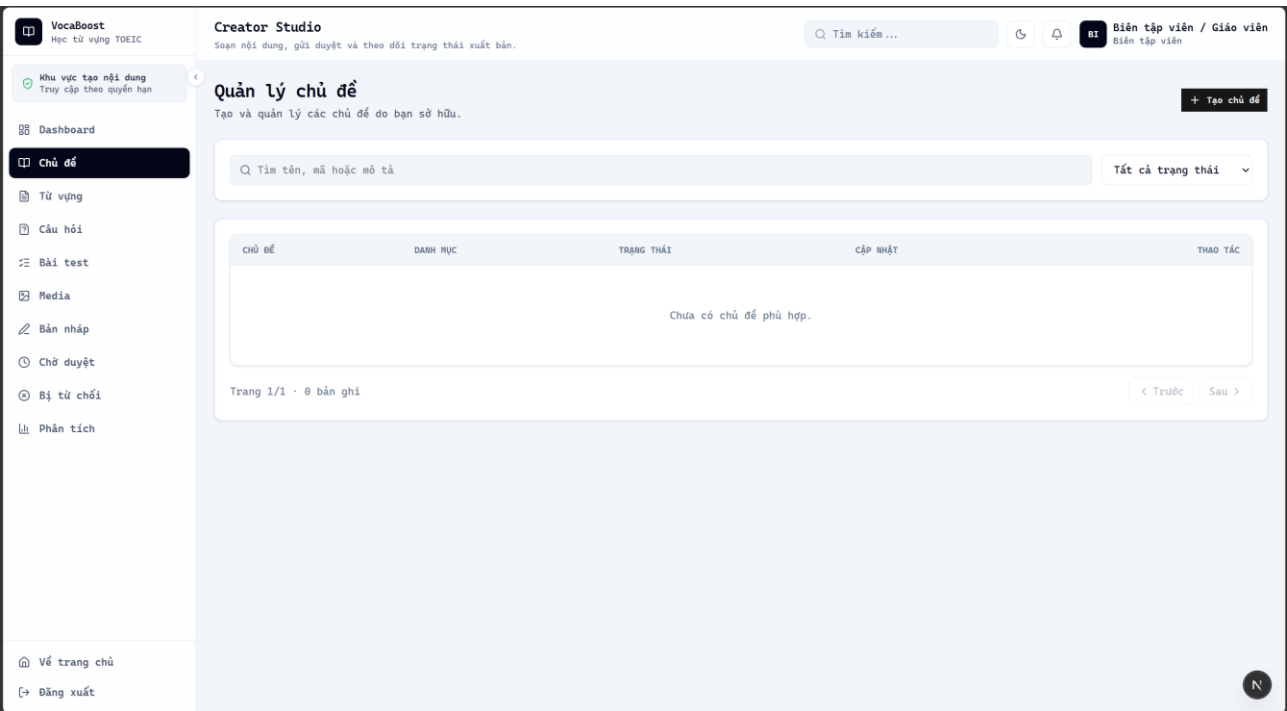


Hình 6.2.12: Giao diện tạo, chỉnh sửa, xuất bản và lưu trữ các bài kiểm tra ngắn.

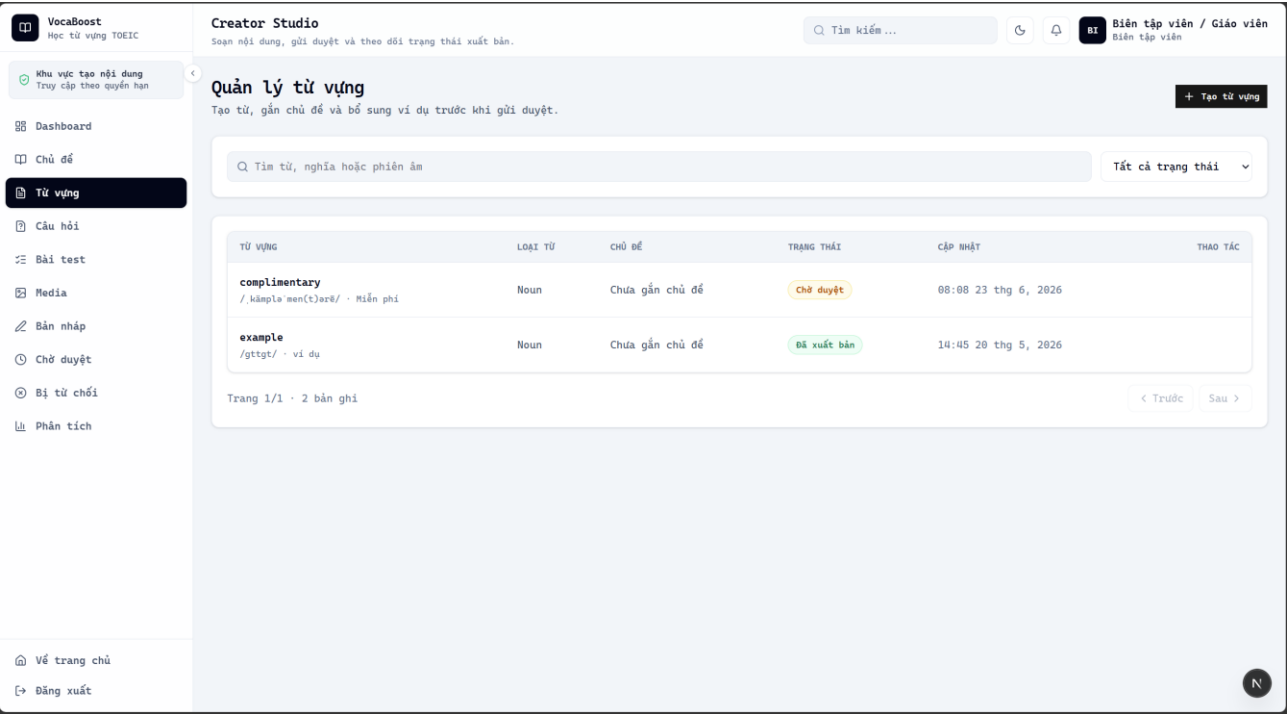
6.3 Giao diện dành cho Creator



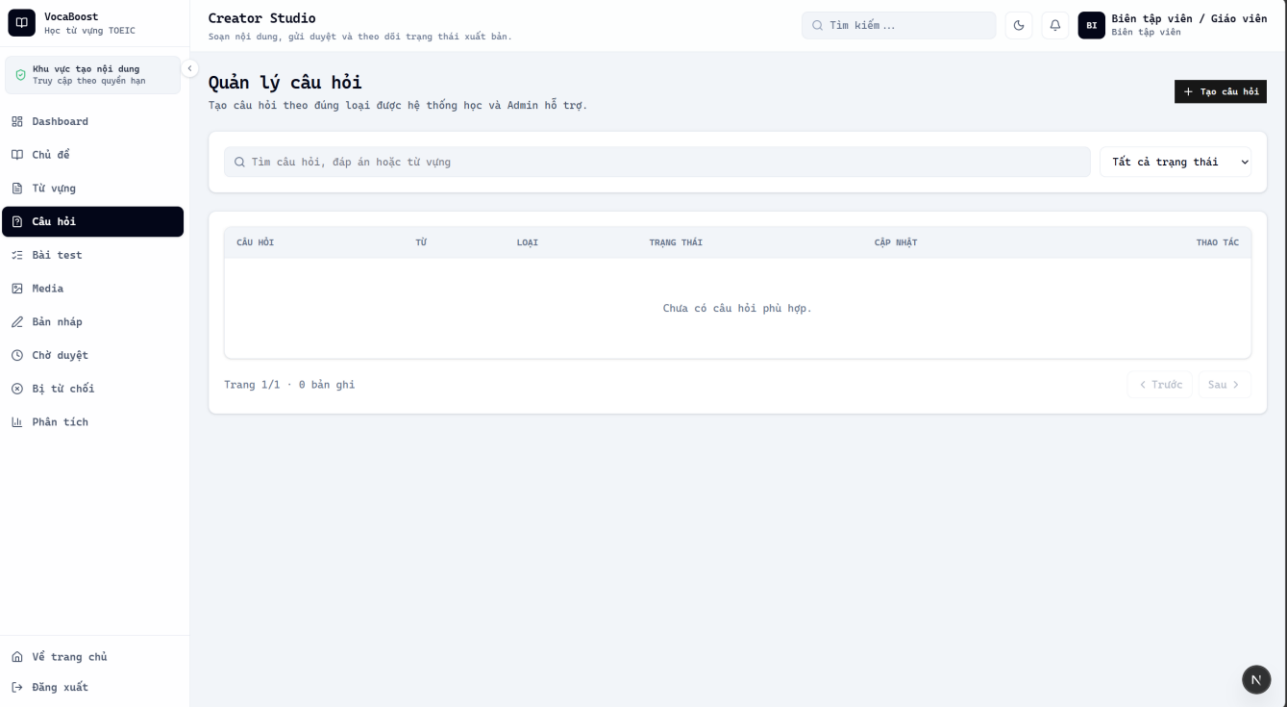
Hình 6.3.1: Tổng quan số lượng chủ đề, từ vựng, câu hỏi, bài test và trạng thái nội dung.



Hình 6.3.2: Giao diện tạo các chủ đề



Hình 6.3.3: Giao diện tạo các từ vựng



The screenshot shows a modal window titled "Tạo câu hỏi" (Create Question) with a close button (X) in the top right corner. The form is divided into several sections:

- TỪ VỰNG *** (Vocabulary): A dropdown menu showing "complimentary - Miễn phí".
- LOẠI CÂU HỎI** (Question Type): A dropdown menu with "Trắc nghiệm" (Multiple Choice) selected. A list of options is visible: "Trắc nghiệm", "Điền vào chỗ trống", "Kéo và thả", "Nghe và chép", and "Kiểm tra flashcard".
- NỘI DUNG CÂU HỎI *** (Question Content): A large text area for entering the question content.
- CÁC LỰA CHỌN** (Options): Four input fields labeled "Lựa chọn 1", "Lựa chọn 2", "Lựa chọn 3", and "Lựa chọn 4", each with a red "X" icon to its right. Below these is a "Thêm lựa chọn" (Add option) button.
- ĐÁP ÁN ĐÚNG *** (Correct Answer): A dropdown menu with "Chọn đáp án đúng" (Select correct answer) selected.
- GIẢI THÍCH** (Explanation): A large text area for providing an explanation.

At the bottom right of the modal, there are two buttons: "Hủy" (Cancel) and "Tạo câu hỏi" (Create Question).

Hình 6.3.4: Giao diện tạo và quản lý các câu hỏi.

The screenshot shows the "Creator Studio" interface for "Quản lý mini test" (Manage Mini Test). The header includes the "VocaBoost" logo, the text "Học từ vựng TOEIC", and a search bar with "Tìm kiếm...". The user is logged in as "Biên tập viên / Giáo viên" (Editor / Teacher).

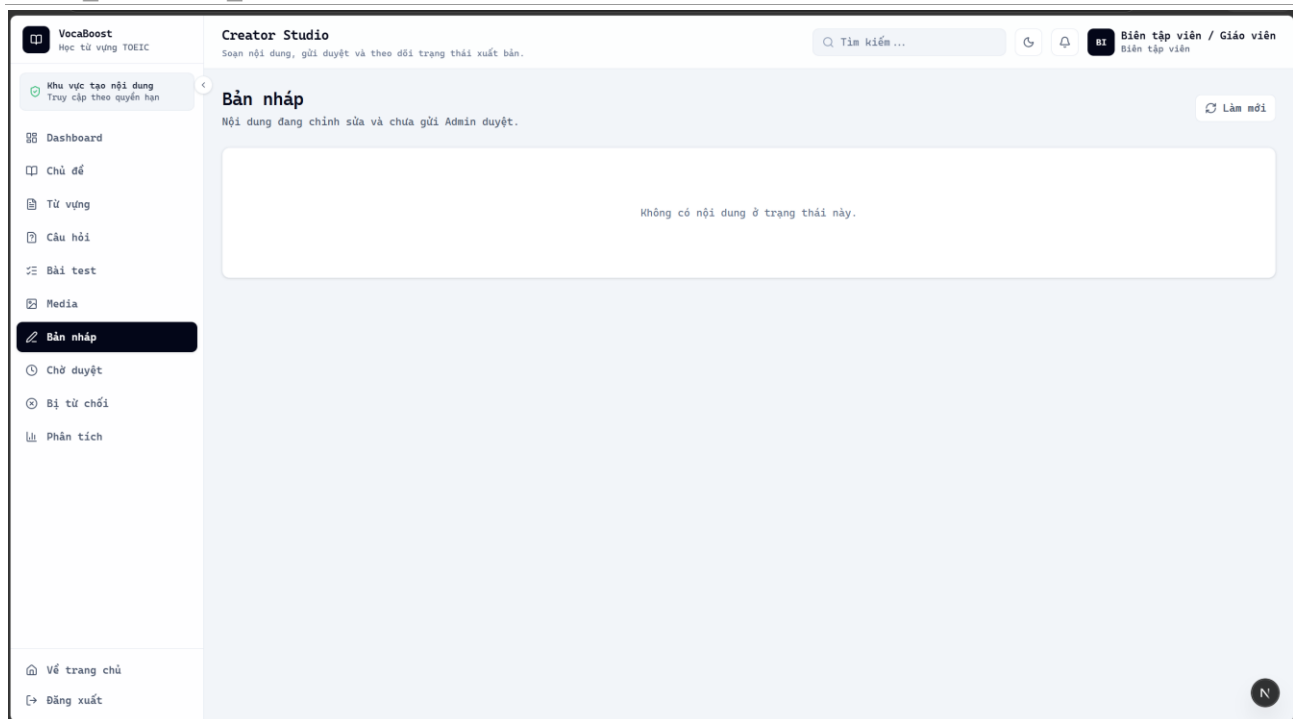
The main content area is titled "Quản lý mini test" and includes the subtitle "Tạo bài kiểm tra từ các câu hỏi đã được Admin xuất bản." (Create a test from questions published by Admin). A "+ Tạo mini test" (Create mini test) button is in the top right corner.

Below the subtitle is a search bar with the placeholder "Tìm tiêu đề hoặc mô tả" (Find title or description) and a dropdown menu labeled "Tất cả trạng thái" (All status). The main area displays a message: "Chưa có mini test phù hợp." (No suitable mini test found).

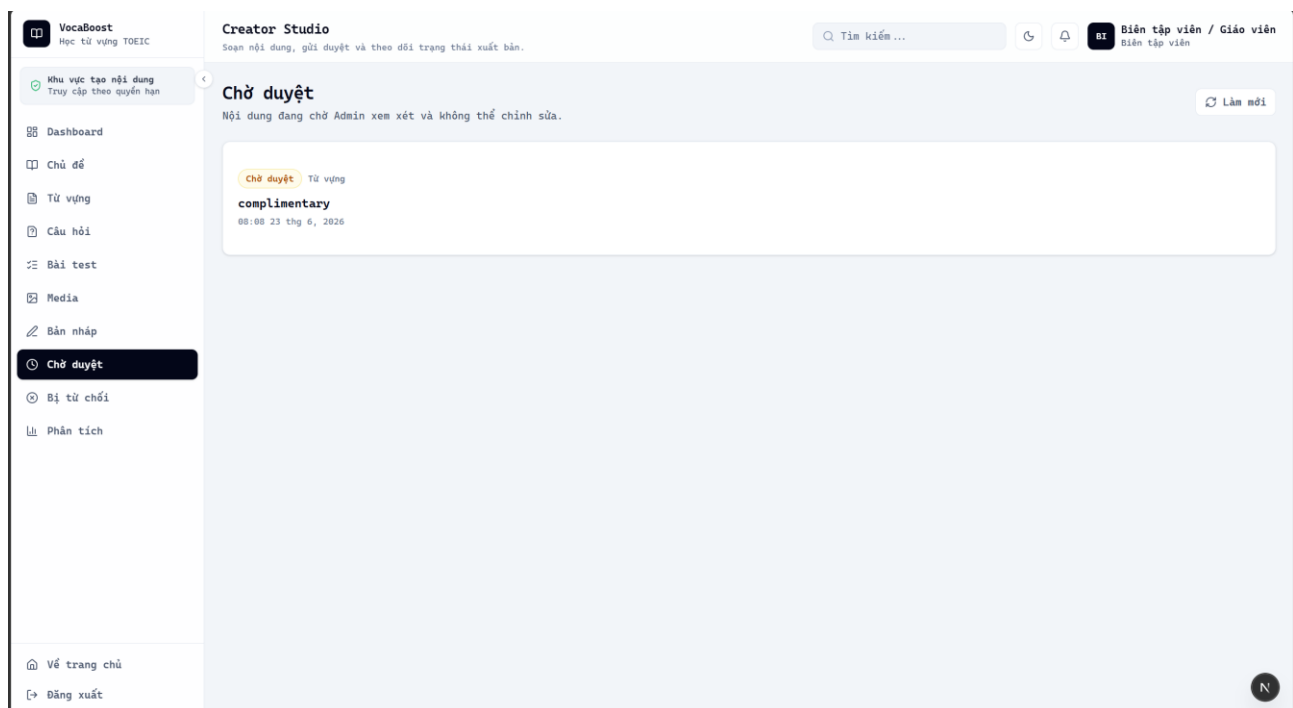
At the bottom, there is a pagination bar showing "Trang 1/1 · 0 bản ghi" (Page 1/1 · 0 records) and navigation buttons "< Trước" (Previous) and "Sau >" (Next).

A sidebar on the left contains a menu with the following items: Dashboard, Chủ đề (Topic), Từ vựng (Vocabulary), Câu hỏi (Questions), **Bài test** (Test), Media, Bản nháp (Draft), Chờ duyệt (Pending review), Bị từ chối (Rejected), and Phân tích (Analysis). At the bottom of the sidebar are links for "Về trang chủ" (Home) and "Đăng xuất" (Logout).

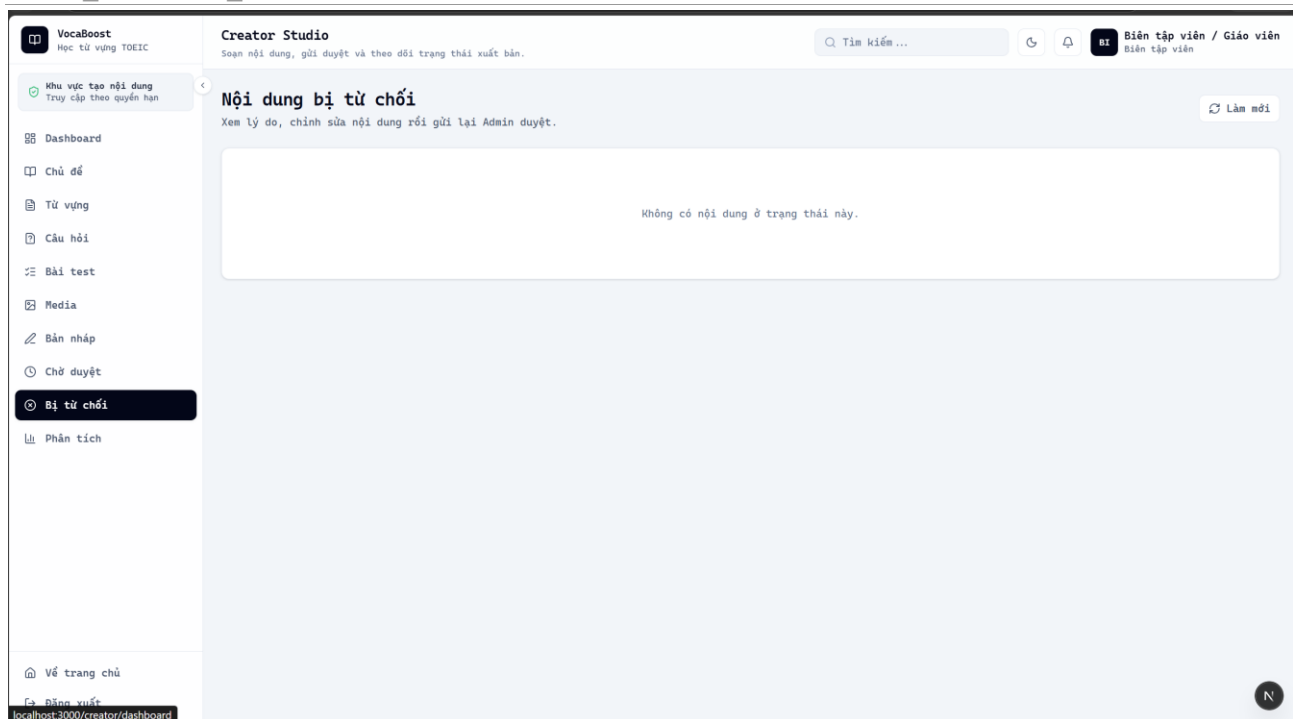
Hình 6.3.5: Giao diện tạo và quản lý minitest



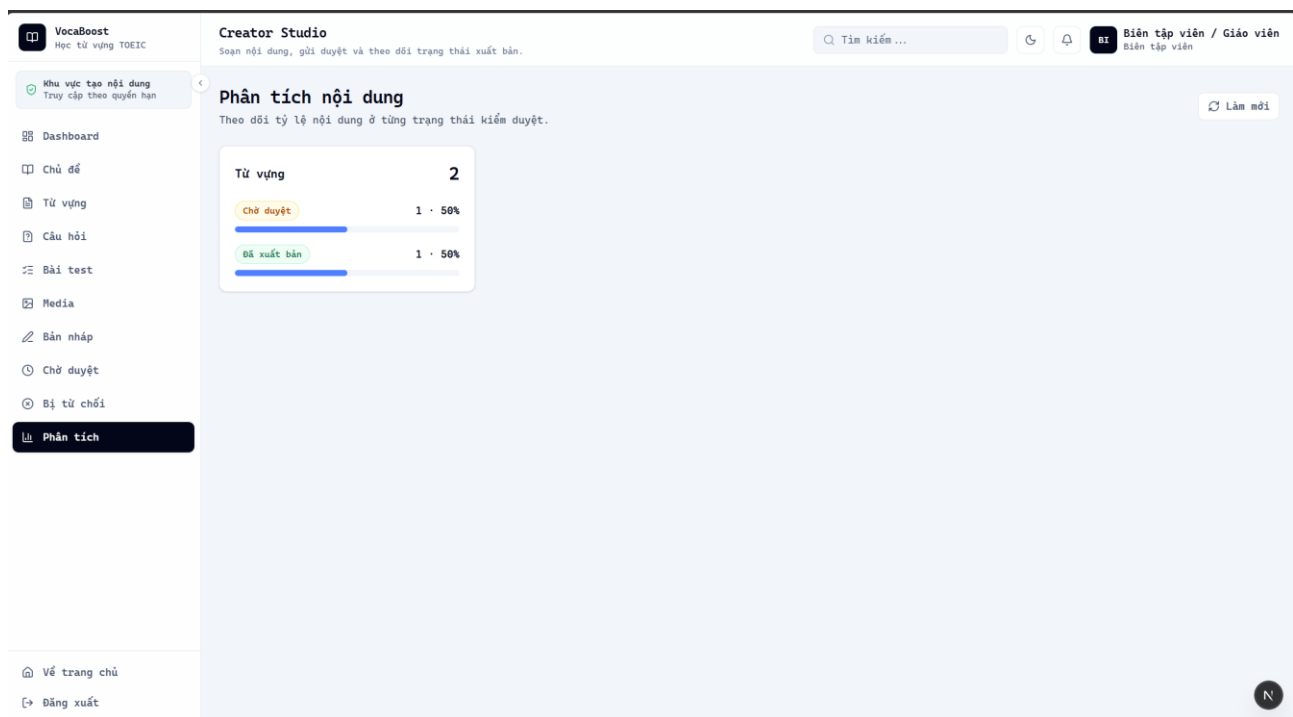
Hình 6.3.6: Tập hợp nội dung đang soạn, chưa gửi sang quy trình kiểm duyệt.



Hình 6.3.7: Giao diện xem trạng thái chờ admin duyệt.



Hình 6.3.8: Giao diện xem nội dung bị từ chối duyệt.



Hình 6.3.9: Các chỉ số giúp đánh giá quy mô và hiệu quả nội dung đã tạo.